BAYMAX

AN INTELLIGENT INTEGRATED SYSTEM FOR SMART MEDICAL ASSISTANCE AND HEALTHCARE SERVICES

submitted in the partial fulfillment of the requirements
for the award of the degree in

BACHELOR OF COMPUTER APPLICATIONS

By

VETRI SURIYA P	(234031101137)
VIGNESH B	(234031101138)
VIGNESH M	(234031101139)
VIGNESH M	(234031101140)
VIGNESHWARAN D	(234031101141)

DEPARTMENT OF COMPUTER APPLICATIONS

Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status
(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

APRIL 2026

DECLARATION

We **VETRI SURIYA P, VIGNESH B, VIGNESH M, VIGNESH M, VIGNESHWARAN D** hereby declare that the Project Report entitled **"BAYMAX – SMART MEDICAL ASSISTANT"** is done by me under the guidance of **Mr. N. SURYA** is submitted in partial fulfillment of the requirements for the award of the degree in BACHELOR OF COMPUTER APPLICATIONS.

SIGNATURE OF THE CANDIDATE

1.

2.

3.

Date: **4.**

Place: **5.**

Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status
(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the Bonafide work of Mr./Ms. **VETRI SURYA.P, VIGNESH.B, VIGNESH.M, VIGNESH.M, VIGNESHWARAN.D** Reg. No **234031101137, 234031101138, 234031101139, 234031101140, 234031101141** who carried out the project entitled "**BAYMAX – SMART MEDICAL ASSISTANT**" under our supervision from December 2025 to March 2026.

Internal Guide **Head of the Department**

Submitted for Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEADGEMENT

We would first like to thank our beloved Founder Chancellor **Dr. A.C. SHANMUGAM, B.A., B.L.** and President **Er. A.C.S. ARUNKUMAR, B.Tech., M.B.A.,** and Secretary **Thiru A. RAVIKUMAR** for all the encouragement and support extended to us during the tenure of this project and also our years of studies in his wonderful University.

We express our heartfelt thanks to our Vice Chancellor Prof. Dr. S. GEETHALAKSHMI in providing all the support of our Project (Project Phase-II).

We extend our sincere gratitude to our **Deputy Head of Department, Mrs. V. SARALA DEVI** for her unwavering support and valuable guidance throughout the duration of our project.

We express our heartfelt thanks to our **Head of the Department, Dr. VIJI VINOD,** who has been actively involved and very influential from the start till the completion of our project.

Our sincere thanks to our Project Coordinator **MR. SURYA** and Project guide **Dr. M. SWEDHA** for their continuous guidance and encouragement throughout this work, which has made the project a success.

We would also like to thank all the teaching and non-teaching staffs of Data Science department, for their constant support and the encouragement given to us while we went about to achieving my project goals.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
API	-	Application Programming Interface Application Programming Interface
API Gateway	-	Application Programming Interface Gateway
AWS	-	Amazon Web Services
BCA	-	Bachelor of Computer Applications
CPU	-	Central Processing Unit
CRUD	-	Create, Read, Update, Delete
CSS	-	Cascading Style Sheets
DB	-	Database
DFD	-	Data Flow Diagram
ER	-	Entity Relationship
Git	-	Global Information Tracker
GPS	-	Global Positioning System
HTML	-	HyperText Markup Language
HTTP	-	HyperText Transfer Protocol
HTTPS	-	HyperText Transfer Protocol Secure
IDE	-	Integrated Development Environment
JSON	-	JavaScript Object Notation
JS	-	JavaScript
JWT	-	JSON Web Token
ML	-	Machine Learning
MVC	-	Model View Controller

NoSQL	-	Not Only Structured Query Language
NPM	-	Node Package Manager
OS	-	Operating System
RAM	-	Random Access Memory
REST	-	Representational State Transfer
SOS	-	Save Our Souls (Emergency Signal)
SPA	-	Single Page Application
UI	-	User Interface
UML	-	Unified Modeling Language
URL	-	Uniform Resource Locator
UX	-	User Experience
VS Code	-	Visual Studio Code

ABSTRACT

Accessing timely healthcare services is often challenging due to lack of medical awareness, difficulty in finding suitable doctors, long waiting times, and scattered healthcare resources. BAYMAX is an intelligent, user-centric system designed to simplify healthcare access by integrating doctor recommendation, appointment booking, pharmacy assistance, navigation, health guidance, and emergency support into a single platform. The system allows users to enter their illness, based on which it recommends nearby specialized doctors and displays the number of patients currently waiting. Users can select a doctor and book appointments online, reducing waiting time and manual effort.

The system also identifies nearby pharmacies where prescribed medicines are available and displays their prices, eliminating the need to search multiple medical stores. Integrated map services provide navigation routes to hospitals and pharmacies. A medicine library is included to offer information on medicine usage and requirements. Premium features such as video consultation enable remote interaction with preferred doctors, while prescription-based medicine reminders ensure timely medication intake.

Additional intelligent modules include illness suggestion through user prompts and a personal health trainer that provides basic fitness and diet guidance using a conversational bot. An SOS mode supports emergency situations by displaying emergency contact numbers and alerting nearby emergency services.
A personalized health dashboard can be included to present an overview of appointments, reminders, and health activities. By combining intelligent recommendations, healthcare navigation, medical knowledge, fitness support, and emergency assistance, the Healthcare Assistant aims to enhance healthcare accessibility, improve patient convenience, and support efficient medical service delivery.

EXISTING PROJECT

Digital healthcare platforms have improved access to medical services by connecting patients with doctors, hospitals, pharmacies, and diagnostic centres through online systems. One of the widely used healthcare platforms in India is **Apollo 24|7**, developed by Apollo Hospitals Group. This platform provides an integrated healthcare ecosystem where users can consult doctors online, book appointments, order medicines, and access health records through web and mobile applications. Apollo 24|7 offers several healthcare services such as **online doctor consultation, appointment booking, medicine ordering, lab test scheduling, and digital health record management**. Patients can search for doctors based on specialization, location, or availability, and the system also supports video consultations, allowing remote interaction with medical professionals. In addition, the platform provides an online pharmacy where users can purchase medicines and healthcare products.

It also enables users to book laboratory tests and receive reports digitally. Another key feature is the **digital health record system**, which stores prescriptions, reports, and treatment history in a centralized location. However, Apollo 24|7 mainly focuses on consultation and appointment services and lacks intelligent features such as **AI-based symptom analysis, doctor recommendation based on illness input, real-time patient waiting information, healthcare navigation support, and emergency assistance modules**. These limitations highlight the need for a more intelligent healthcare system. The proposed system **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance** aims to address these gaps by integrating AI-based health assistance, doctor recommendation, appointment booking, pharmacy services, and emergency support into a single platform to improve healthcare accessibility and user convenience.

BASE RESEARCH

Year of Publishing: 2020

Authors Name: Apollo Hospitals Group

Title: Apollo 24|7 Digital Healthcare Platform

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Healthcare accessibility remains a major challenge in many regions due to factors such as limited medical awareness, difficulty in identifying appropriate doctors, long waiting times, and fragmented healthcare services. Patients often need to visit multiple locations to consult doctors, purchase medicines, book diagnostic tests, and manage medical records. With the advancement of digital technologies, healthcare services are increasingly being integrated into online platforms to improve accessibility and efficiency. Digital healthcare systems allow users to consult doctors remotely, book appointments, order medicines, and maintain digital health records through web or mobile applications. These systems help reduce manual effort, save time, and provide better connectivity between patients and healthcare providers.

Despite these advancements, many existing healthcare platforms mainly focus on appointment booking or online consultation services and lack intelligent features that assist users in identifying suitable medical professionals based on symptoms or health conditions. Additionally, users may face difficulties in locating nearby pharmacies, receiving timely medication reminders, or accessing emergency healthcare assistance quickly.

To address these challenges, the proposed system **"BAYMAX – An Intelligent Integrated System for Smart Medical Assistance"** is developed. BAYMAX is designed as a comprehensive healthcare platform that integrates doctor recommendation, appointment booking, pharmacy services, health guidance, and emergency support into a single intelligent system. The platform allows users to enter their symptoms or illness and receive recommendations for specialized doctors along with relevant healthcare services.

The system also provides additional features such as nearby pharmacy information, medicine availability, digital health records, and AI-based health assistance. By combining intelligent healthcare recommendations with essential medical services,

BAYMAX aims to improve healthcare accessibility, reduce waiting time, and enhance user convenience. Overall, the proposed system focuses on creating a **smart, user-friendly, and integrated healthcare assistance platform** that supports patients in making informed healthcare decisions while simplifying the process of accessing medical services.

1.2 PROBLEM STATEMENT

In the modern healthcare environment, patients often face multiple challenges while accessing medical services. These challenges include difficulty in identifying the right doctor for a specific illness, long waiting times in hospitals, lack of real-time information about doctor availability, and the need to visit multiple platforms for different healthcare services such as consultation, pharmacy, and medical records.

Existing digital healthcare systems provide partial solutions by offering appointment booking and online consultations. However, they lack intelligent assistance features that can guide users based on their symptoms and provide personalized healthcare recommendations. Additionally, users often need to manually search for nearby pharmacies, compare medicine prices, and navigate healthcare facilities, which can be time-consuming and inefficient.

There is also a lack of integrated systems that combine healthcare services such as doctor recommendation, pharmacy assistance, emergency support, and health guidance into a single platform. This fragmentation reduces efficiency and increases the complexity of accessing healthcare services.

Therefore, there is a need for a smart, integrated, and intelligent healthcare system that can provide real-time assistance, reduce manual effort, and improve overall healthcare accessibility. The proposed system BAYMAX aims to address these challenges by providing a unified platform that integrates multiple healthcare services with intelligent decision-support capabilities.

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

A literature survey is an important part of any research or development project. It involves studying existing systems, research papers, and technologies related to the proposed system. The purpose of this chapter is to analyse existing healthcare platforms and digital medical services to understand their functionalities and limitations.

Studying these systems helps in identifying gaps in current healthcare solutions and justifies the need for the proposed system. In the healthcare sector, several digital platforms have been developed to improve medical accessibility and patient convenience. These systems provide services such as doctor consultation, appointment booking, online pharmacy, and digital health records. However, many of these platforms still lack intelligent healthcare assistance features that can guide users based on their symptoms or health conditions.

2.2 EXISTING DIGITAL HEALTHCARE PLATFORMS

Digital healthcare platforms have become increasingly popular due to the growing need for convenient medical services. One of the widely used platforms in India is **Apollo 24|7**, which allows users to consult doctors online, book appointments, order medicines, and schedule lab tests. Such platforms provide an integrated healthcare environment that connects patients with healthcare providers through web and mobile applications.

These systems help reduce the need for physical hospital visits and allow users to access healthcare services from their homes. However, most existing platforms mainly focus on appointment scheduling and consultation services and do not provide advanced intelligent healthcare guidance.

2.3 ROLE OF ARTIFICIAL INTELLIGENCE IN HEALTHCARE

Artificial Intelligence (AI) has significantly transformed the healthcare industry by enabling intelligent decision-support systems and predictive analysis. AI-based

healthcare applications can analyse symptoms, suggest possible medical conditions, and assist users in identifying appropriate medical professionals.

These systems help improve efficiency and provide quick preliminary guidance to users seeking medical assistance. AI-powered technologies can also provide chatbot-based health guidance, medication reminders, and personalized healthcare recommendations. By analysing user inputs and medical data, AI systems can support healthcare services and enhance accessibility for patients. However, it is important to note that AI systems are designed only to **assist and support decision-making**, not to replace professional medical expertise.

AI predictions are based on algorithms and available data, and therefore the results may not always be completely accurate. Users should avoid relying solely on AI-generated suggestions and must consult qualified healthcare professionals for proper diagnosis and treatment. Thus, AI should be considered as a **supportive healthcare tool** that helps provide preliminary guidance and improves healthcare accessibility, while final medical decisions should always be made by trained medical practitioners.

2.4 LIMITATIONS OF EXISTING SYSTEMS

Although many healthcare platforms provide digital medical services, they still have several limitations. Most systems focus mainly on doctor consultation and appointment booking and do not include intelligent features such as AI-based symptom analysis, illness-based doctor recommendation, real-time waiting information, or emergency healthcare assistance. Additionally, users often need to search manually for suitable doctors or nearby pharmacies, which can be time-consuming. These limitations highlight the need for an integrated and intelligent healthcare platform that can provide better medical assistance and guidance.

2.5 NEED FOR THE PROPOSED SYSTEM

Based on the analysis of existing systems and technologies, it is clear that there is a need for a more intelligent and integrated healthcare platform. The proposed system **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance** aims to address these limitations by combining AI-based healthcare

guidance with essential medical services such as doctor recommendation, appointment booking, pharmacy support, and emergency assistance. The proposed system focuses on providing a smart healthcare environment that improves accessibility, reduces waiting time, and helps users make informed healthcare decisions.

2.6 SUMMARY OF LITERATURE SURVEY

This chapter presented a review of existing healthcare platforms, technologies, and research related to digital healthcare systems. The study analysed current healthcare platforms such as Apollo 24|7 and discussed how digital systems help users access medical services like doctor consultation, appointment booking, pharmacy services, and digital health records.

In addition to analyzing existing healthcare platforms, the literature survey also explored various technological frameworks and architectures used in modern digital healthcare systems. It was observed that most existing solutions focus primarily on providing isolated services such as online consultation or medicine delivery, rather than offering a fully integrated healthcare ecosystem. This fragmentation often leads to inefficiencies and increased effort for users when managing different aspects of their healthcare needs.

The literature survey also examined the role of Artificial Intelligence in healthcare, highlighting how AI can assist in symptom analysis, health guidance, and intelligent medical recommendations. At the same time, it emphasized that AI should be considered a supportive tool rather than a complete replacement for professional medical diagnosis. The analysis of existing systems revealed several limitations, including the lack of intelligent healthcare assistance, limited integration of services, and difficulty in quickly identifying appropriate medical support. These gaps highlight the need for a more advanced and integrated healthcare solution.

Therefore, the proposed system **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance** is developed to address these limitations by combining AI-based healthcare guidance with doctor recommendation, appointment booking, pharmacy support, and emergency assistance within a single platform.

CHAPTER 3
AIM AND SCOPE OF PRESENT INVESTIGATION

3.1 INTRODUCTION

This chapter describes the aim, objectives, and scope of the proposed system **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance**. It explains the purpose of developing the system and the areas in which it can be applied. The chapter also outlines the limitations and future possibilities of the project. The present investigation focuses on developing a **smart healthcare assistance platform** that integrates multiple healthcare services such as doctor recommendation, appointment booking, pharmacy support, and health guidance within a single system. By using modern web technologies and intelligent assistance features, the system aims to improve healthcare accessibility and user convenience.

3.2 AIM OF THE PROJECT

The main aim of the project **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance** is to develop a centralized healthcare platform that provides intelligent medical assistance and simplifies access to healthcare services. The system aims to help users identify suitable doctors based on their symptoms, book appointments easily, access nearby pharmacy information, and receive basic health guidance through an integrated digital platform. The project also focuses on reducing waiting time, improving healthcare accessibility, and providing a user-friendly environment where patients can manage their healthcare needs efficiently.

3.3 OBJECTIVES OF THE INVESTIGATION

The present investigation is carried out with the following objectives:

3.3.1 Technical Objectives

- To design and develop a web-based healthcare assistance platform.
- To implement a user-friendly interface using modern web technologies.
- To integrate a database system for storing user and healthcare information.
- To implement intelligent healthcare assistance features.

3.3.2 Functional Objectives

- To enable users to search and identify doctors based on illness or specialization.
- To provide an online appointment booking system.
- To display nearby pharmacies and medicine availability.
- To maintain digital health records for users.
- To provide emergency healthcare support information.

3.3.3 Social Objectives

- To improve healthcare accessibility for users.
- To reduce the time required to find suitable medical services.
- To assist users in making better healthcare decisions.

3.4 SYSTEM FEATURES

The proposed system BAYMAX incorporates several advanced features designed to enhance healthcare accessibility and improve user experience. The system provides intelligent doctor recommendation by allowing users to enter symptoms or illness details and receive suggestions for specialized doctors, thereby reducing the need for manual searching.

It includes a real-time waiting count feature that displays the number of patients waiting for a doctor, helping users make better decisions and save time. The platform supports online appointment booking, enabling users to schedule consultations without physically visiting hospitals. It also integrates pharmacy services by providing information about nearby pharmacies, medicine availability, and price comparison.

An AI-based health assistant is included to offer basic healthcare guidance through user queries, assisting users in understanding their symptoms. The system maintains digital health records, allowing users to store and access medical history and prescriptions in a centralized manner.

Additionally, it provides an emergency SOS feature that offers quick access to emergency contacts and nearby hospitals. Navigation support is integrated to help users locate healthcare facilities easily, and a medicine reminder system ensures

timely medication intake. The overall design of the system focuses on providing a simple, user-friendly interface that can be easily used by individuals with minimal technical knowledge.

3.5 SCOPE OF THE PRESENT INVESTIGATION

The scope of the project defines the boundaries and operational coverage of the system. The proposed system focuses on developing a **web-based healthcare assistance platform** that integrates several healthcare services into a single application.

The system allows users to:

- Search for doctors based on illness or specialization
- Book medical appointments online
- Access nearby pharmacy information
- View medicine details and availability
- Maintain digital health records
- Receive healthcare guidance through intelligent assistance

The platform is designed to be accessible through web browsers and can be used by patients to manage healthcare services conveniently.

3.6 LIMITATIONS

Although the proposed system provides several useful healthcare features, it has certain limitations. The system provides **basic health guidance and assistance**, but it cannot replace professional medical diagnosis or treatment. Users must consult qualified healthcare professionals for accurate medical advice.

Additionally, the system depends on internet connectivity for accessing healthcare services. Real-time integration with hospital databases and medical institutions may also be limited in the current version.

3.7 FUTURE SCOPE

The system can be further improved by adding advanced features in the future.

Possible enhancements include:

- Integration with hospital management systems
- Development of a mobile application version

- Real-time integration with pharmacy and hospital databases
- Multilingual support for wider accessibility
- Advanced AI-based health prediction and recommendation systems
- Integration with wearable health monitoring devices

These improvements can make the system more efficient and provide a more comprehensive healthcare assistance platform.

3.8 SUMMARY

This chapter explained the aim, objectives, scope, limitations, and future possibilities of the proposed system. The investigation focuses on developing an intelligent healthcare assistance platform that integrates multiple healthcare services into a single system.

In addition to defining the core objectives, this chapter highlights the importance of integrating modern technologies such as artificial intelligence and real-time data processing into healthcare systems. The proposed system is designed to address common challenges faced in traditional healthcare, including difficulty in identifying suitable doctors, lack of immediate guidance, and inefficient appointment management.

The scope of the system extends beyond basic consultation services by incorporating features such as symptom-based doctor recommendation, online appointment scheduling, pharmacy assistance, and emergency support. These features collectively contribute to creating a comprehensive and user-centric healthcare ecosystem.

The limitations discussed in this chapter provide a realistic understanding of the system boundaries, such as dependency on internet connectivity, availability of accurate data, and constraints in AI-based predictions. Recognizing these limitations helps in identifying areas for improvement and ensures transparency in system capabilities.

By combining intelligent guidance with digital healthcare services, the proposed system **BAYMAX** aims to improve healthcare accessibility, reduce waiting time, and provide a convenient platform for users to manage their healthcare needs.

CHAPTER 4
EXPERIMENTAL OR MATERIAL AND METHOD
ALGORITHM USED

4.1 INTRODUCTION

This chapter describes the technical components, tools, and methods used in the development of the proposed system **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance**. It explains the hardware and software requirements, system architecture, development methodology, modules implementation, and security mechanisms used in the system. The project is developed using modern web technologies and follows a **client–server architecture**, where the frontend provides the user interface and the backend processes user requests and communicates with the database. The system integrates healthcare services such as doctor recommendation, appointment booking, pharmacy information, and health assistance into a single platform.

4.2 MATERIALS USED

4.2.1 Hardware Requirements

The system requires standard computing hardware to develop and run the application.

Table 1: Hardware Requirements and Specification

Component	Specification
Processor	Intel Core i3 or higher
RAM	Minimum 8 GB
Storage	Minimum 10 GB free space
Operating System	Windows 10 / 11
Internet	Stable internet connection
Camera & Microphone (Optional)	Optional – Required only for video consultation and voice communication with doctors

This configuration is sufficient for developing and running the web-based healthcare application.

4.2.2 Software Requirements

The system uses several software technologies to develop different components of the application.

Frontend Technologies

- **HTML** – Used for structuring web pages
- **CSS** – Used for styling and layout design
- **JavaScript** – Used for adding interactivity
- **React.js** – Used for building dynamic user interfaces

Backend Technologies

- **Node.js** – Used for server-side programming
- **Express.js** – Used for handling API requests and server operations

Database

- **MongoDB** – Used for storing user data, medical information, and system records

Development Tools

- **Visual Studio Code** – Code editor used for development
- **Git & GitHub** – Used for version control and project management
- **Web Browser (Chrome / Edge)** – Used for testing the application

These technologies were selected because they are scalable, efficient, and widely used in modern web application development.

4.3 SYSTEM ARCHITECTURE

The system architecture of **BAYMAX** is designed to ensure efficient communication between different components while maintaining scalability and performance. The architecture follows a three-tier model consisting of the presentation layer, application layer, and data layer.

The presentation layer is responsible for handling user interactions through a web-based interface developed using **React.js**, which provides dynamic rendering and a responsive user experience.

The application layer, built using **Node.js** and **Express.js**, acts as the core processing unit of the system, handling API requests, business logic, and communication between the frontend and database. The data layer utilizes MongoDB, which stores structured and unstructured healthcare data in a flexible and scalable format.

The architecture also supports modular design, allowing each functional component such as user management, doctor consultation, pharmacy services, and AI assistance to operate independently while still being integrated within the system. This modularity improves maintainability and allows future enhancements to be implemented without affecting existing functionalities.

Additionally, RESTful APIs are used to enable seamless communication between client and server, ensuring efficient data exchange and faster response times. The system is designed with scalability in mind, allowing it to handle increasing numbers of users and data without performance degradation. Load balancing and efficient database queries further enhance system performance. Security is also integrated into the architecture through authentication mechanisms and encrypted communication protocols.

Overall, the architecture ensures reliability, efficiency, and flexibility, making the system suitable for real-world healthcare applications.

The proposed system follows a **client-server architecture**.

Client Side

The client side represents the user interface of the system. It is developed using React.js and allows users to interact with the platform through web browsers. Users can search for doctors, book appointments, view medicines, and access healthcare services through this interface.

Server Side

The server side processes requests sent by the client. It handles user authentication, appointment management, doctor data retrieval, and communication with the database. The backend is developed using Node.js and Express.js.

Database Layer

The database stores all system data including user information, doctor details, appointment records, and medicine data. MongoDB is used as the database because it provides flexibility and efficient data storage for web applications.

The architecture ensures:

- Efficient communication between frontend and backend
- Secure storage of user data
- Scalable system performance

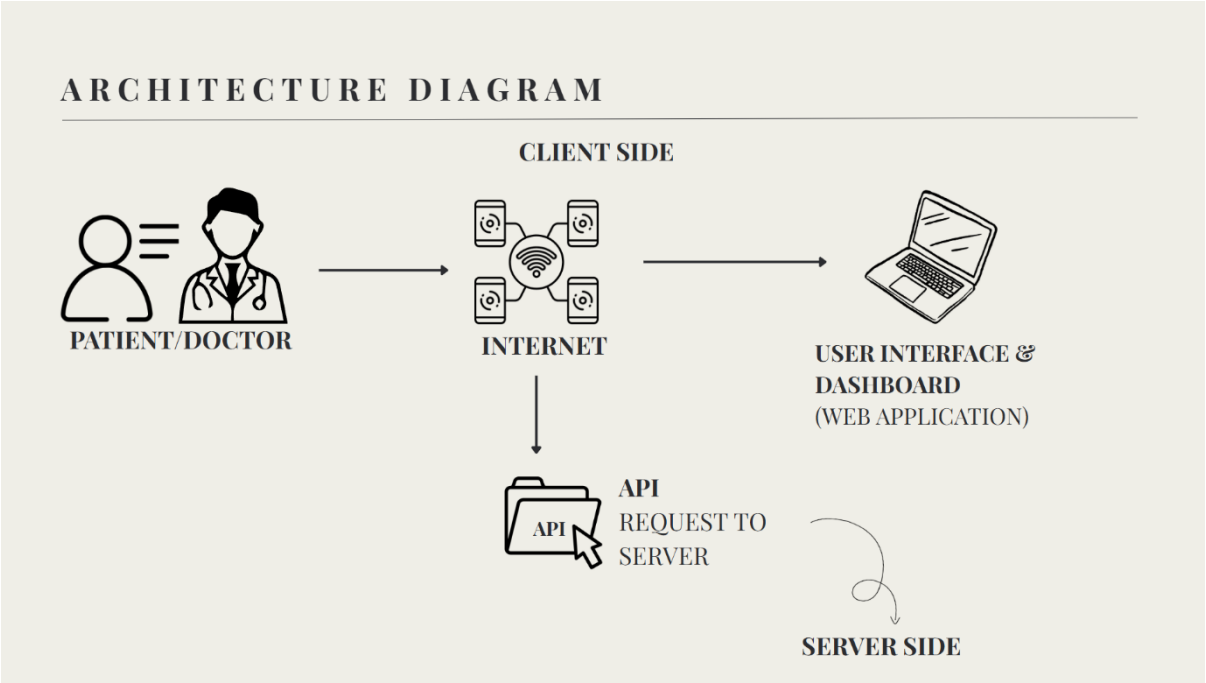

Fig 1: Architecture Diagram

4.4 EXPERIMENTAL METHODOLOGY

The system development process followed several stages.

Step 1: Requirement Analysis

- Identify system requirements
- Study existing healthcare platforms

- Define project objectives

Step 2: System Design

- Design system architecture
- Create database structure
- Plan modules and functionalities

Step 3: Implementation

- Develop frontend components
- Implement backend APIs
- Connect system with the database

Step 4: Testing

- Perform module testing
- Verify system functionality
- Fix errors and improve performance

Step 5: Deployment

- Run the application on a web server
- Test the system in real-time conditions

4.5 MODULES IMPLEMENTATION

Each module in the BAYMAX system is designed to perform specific functions while contributing to the overall efficiency of the platform. The user management module ensures secure authentication and profile management, enabling personalized user experiences. The doctor consultation module not only allows users to search and book appointments but also provides detailed doctor profiles, including specialization, experience, and availability, which helps users make informed decisions.

The pharmacy module enhances convenience by providing real-time information about medicine availability and pricing, reducing the effort required to visit multiple medical stores. The health records module ensures that all user medical data is stored securely and can be accessed anytime, promoting better healthcare management and continuity of treatment. The AI health assistant module uses intelligent processing to analyse user inputs and provide preliminary guidance, making healthcare information more accessible.

The emergency assistance module plays a critical role in providing immediate support during urgent situations by displaying emergency contacts and nearby healthcare facilities. These modules are interconnected and communicate efficiently through the backend system, ensuring smooth data flow and consistent performance. The modular approach also allows the system to be easily extended with additional features in the future.

The system is divided into several functional modules.

4.5.1 User Management Module

This module manages user registration, login authentication, and user profile information. It ensures that only authorized users can access the system.

4.5.2 Doctor Consultation Module

This module allows users to search for doctors based on specialization or illness and book appointments online.

4.5.3 Pharmacy Module

This module provides information about medicines and allows users to search for medicines available in nearby pharmacies.

4.5.4 Health Records Module

This module stores user medical records, prescriptions, and health information in a centralized database.

4.5.5 AI Health Assistant Module

This module provides basic health guidance by analysing user inputs and offering preliminary healthcare suggestions.

4.5.6 Emergency Assistance Module

This module provides emergency contact numbers and quick access to nearby healthcare services.

4.5.7 Data Flow Diagram

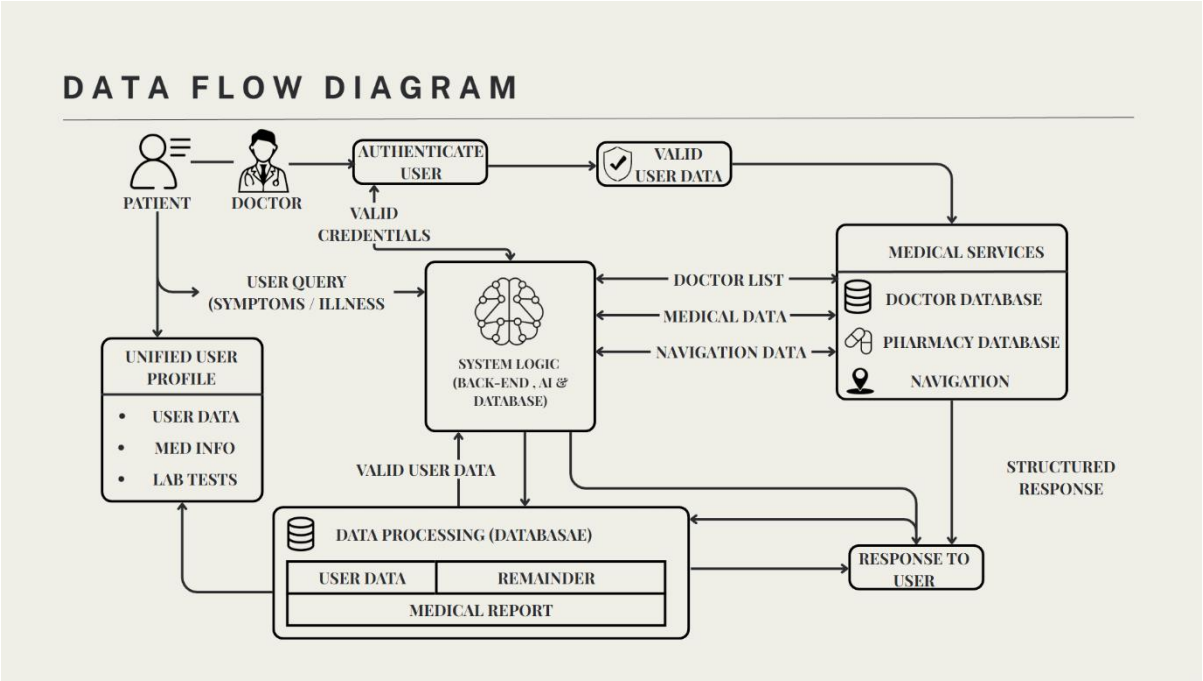

Fig 2: Data Flow Diagram

4.6 ALGORITHM USED

The system uses a simple **health query processing algorithm** to assist users.

Health Assistance Algorithm

Step 1: User enters symptoms or health-related query.

Step 2: System validates user input.

Step 3: The query is processed by the system.

Step 4: Relevant healthcare information is retrieved from the database.

Step 5: System suggests appropriate doctors or healthcare services.

Step 6: Results are displayed to the user.

This algorithm helps users quickly identify suitable healthcare services.

4.7 SECURITY MECHANISMS USED

The system implements several security mechanisms to protect user data.

- **User authentication system** for login verification
- **Role-based access control** for system users
- **Secure database access** to prevent unauthorized data access
- **Encrypted communication (HTTPS)** for data protection

These mechanisms ensure the safety of user information and system data.

4.8 TESTING METHODS

Several testing methods were employed to verify the performance, reliability, and functionality of the BAYMAX healthcare system. These testing approaches ensure that all modules operate correctly and that the system delivers accurate and consistent results under different conditions. Unit testing was conducted to validate individual components, while integration testing ensured that different modules interact seamlessly without errors. System testing was performed to evaluate the complete system's behaviour, and user acceptance testing helped confirm that the application meets user requirements and expectations. Overall, these comprehensive testing strategies enhance the robustness, efficiency, and dependability of the system.

4.8.1 Unit Testing

Each module of the system was tested individually to ensure that it performs its intended function correctly. Components such as user authentication, doctor search, and appointment booking were tested separately to identify and fix errors at an early stage.

4.8.2 Integration Testing

Integration testing was performed to verify the interaction between different components of the system. The communication between the frontend, backend, and database was tested to ensure smooth data flow and proper API functionality.

4.8.3 System Testing

The entire system was tested as a complete unit to ensure that all features work together without any issues. This includes verifying user workflows such as login, booking appointments, accessing health records, and using the AI assistant.

4.8.4 Performance Testing

Performance testing was conducted to evaluate the system's response time, speed, and stability under different conditions. The system was tested with multiple user requests to ensure it can handle real-time operations efficiently without performance degradation.

4.9 USE CASE DIAGRAM

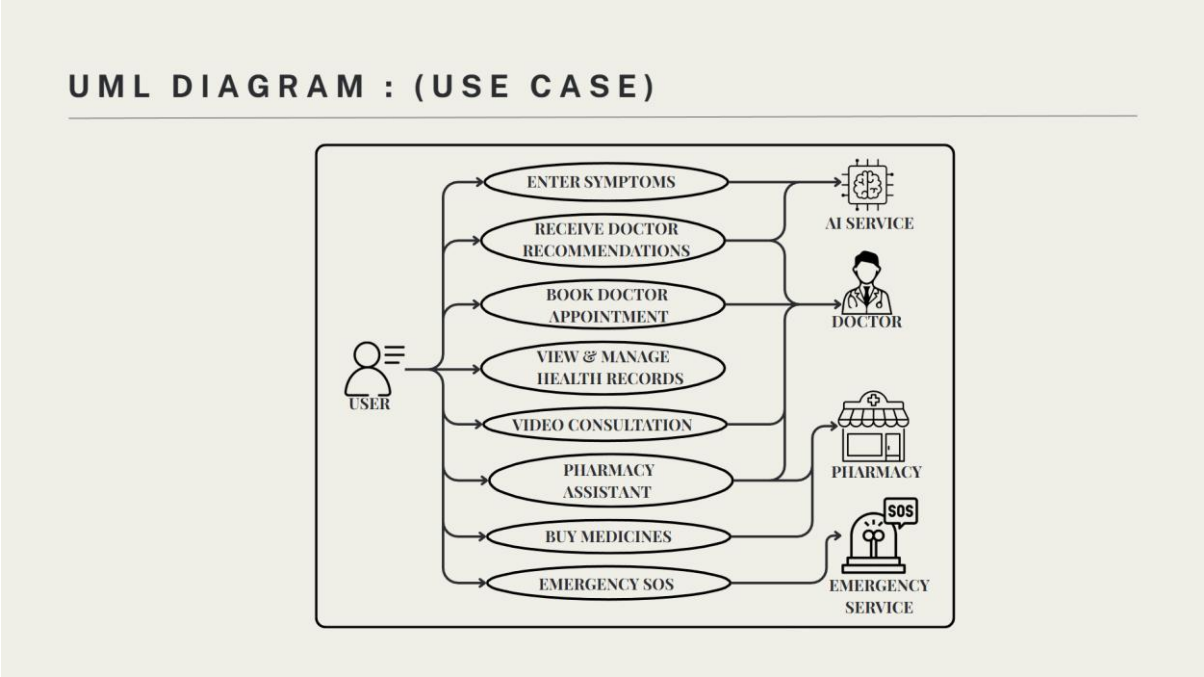

Fig 3: UML Diagram: Use Case

4.10 DATABASE DESIGN

The database design plays a crucial role in managing and storing system data efficiently in the BAYMAX platform. The system uses MongoDB, a NoSQL database, which allows flexible and scalable data storage. The database is structured into multiple collections to handle different types of information required by the system. The user collection stores details such as user name, email, password, and profile information.

The doctor collection maintains information about doctors, including their specialization, experience, availability, and consultation details. The appointment collection stores booking details such as user ID, doctor ID, date, and time of appointment. The medicine collection contains information about various medicines, including name, type, price, and availability.

The health records collection stores medical history, prescriptions, and other health-related data of users.

Additionally, the emergency contacts collection contains important emergency numbers and hospital details. This database structure ensures efficient data retrieval, scalability for handling large datasets, flexibility for future expansion, and secure storage of sensitive healthcare information, thereby supporting smooth system operation.

4.12 FEASIBILITY STUDY

The feasibility study evaluates the practicality and viability of the proposed system from different perspectives. The technical feasibility of the BAYMAX system is high, as it is developed using widely used and reliable technologies such as React.js, Node.js, and MongoDB, which are capable of supporting scalable and efficient applications. These technologies are compatible with standard hardware requirements and can be easily deployed in real-world environments.

From an operational perspective, the system is designed to be user-friendly and easy to use, requiring minimal technical knowledge. This ensures that users from different backgrounds can access healthcare services without difficulty. The economic feasibility of the system is also favourable, as it utilizes open-source technologies and tools, reducing development and maintenance costs.

The system is also feasible from a scheduling perspective, as the modular development approach allows the project to be completed within the given academic timeline. Additionally, legal and ethical considerations are taken into account by ensuring data privacy and security. The system does not replace professional medical advice but acts as a supportive tool, maintaining ethical standards in healthcare applications. Overall, the feasibility study confirms that the proposed system is practical, cost-effective, and suitable for implementation.

4.12 SUMMARY

This chapter explained the materials, technologies, system architecture, development methodology, modules implementation, and security mechanisms used in developing the proposed system.

CHAPTER 5
SYSTEM DESIGN

5.1 INTRODUCTION

The system design phase plays a crucial role in transforming the conceptual model of the proposed system into a structured and implementable solution. It defines how different components of the system interact with each other to achieve the desired functionality.

The BAYMAX system is designed as a user-friendly, scalable, and efficient healthcare platform that integrates multiple services such as doctor consultation, pharmacy assistance, health records, and AI-based health guidance. The design focuses on providing a seamless user experience while ensuring system performance, security, and reliability.

This chapter presents the design aspects of the system, including user interface design, system workflow, database structure, and API architecture.

5.2 UI/UX DESIGN PRINCIPLES

The user interface of the BAYMAX system is designed to provide a clean, simple, and intuitive experience for users. Since the system is intended for healthcare services, it is important that users of all age groups can easily navigate and access the features.

The following design principles were followed:

- ***Simplicity***

The The interface is kept minimal with clear navigation options, ensuring that users can easily find doctors, medicines, and healthcare services without confusion, thereby improving accessibility, reducing user effort, and enhancing overall interaction efficiency.

- ***Consistency***

Uniform design elements such as colors, fonts, and buttons are used throughout the application to provide a consistent user experience, ensuring improved usability, better visual clarity, and enhanced user satisfaction across all system interfaces.

- ***Accessibility***

The system is designed to be accessible to users with basic technical knowledge. Important features such as doctor search and emergency services are easily reachable.

- ***Visual Hierarchy***

Important information such as doctor availability, waiting count, and emergency options are highlighted using visual elements to attract user attention.

- ***Responsive Design***

The application is designed to work across different screen sizes, ensuring usability on desktops, tablets, and mobile devices.

5.3 SYSTEM WORKFLOW

The workflow of the BAYMAX system describes how user requests are processed and how the system responds to them.

Workflow Steps:

1. The user accesses the web application through a browser
2. The user enters a query (e.g., symptoms, doctor search, or medicine search)
3. The request is sent to the backend server via API
4. The server processes the request and interacts with the database
5. Relevant data is retrieved (doctor details, medicines, etc.)
6. The system processes the results and sends them back to the frontend
7. The frontend displays the results to the user

This workflow ensures smooth communication between different system components and provides quick responses to user queries.

5.4 DATABASE DESIGN OVERVIEW

The database design is an essential part of the system as it manages all the data required for the application. The BAYMAX system uses MongoDB, a NoSQL database, which allows flexible and scalable data storage.

Key Collections:

- ***User Collection***
  Stores user details such as name, email, password, and profile information.
- ***Doctor Collection***
  Contains doctor details including specialization, experience, availability, and consultation details.
- ***Appointment Collection***
  Stores appointment details such as user ID, doctor ID, date, and time.
- ***Medicine Collection***
  Contains information about medicines, including name, type, price, and availability.
- ***Health Records Collection***
  Stores medical history, prescriptions, and health-related data of users.

The database structure ensures efficient data retrieval, scalability, and secure storage of sensitive healthcare information.

5.5 API DESIGN

The BAYMAX system uses RESTful APIs to enable communication between the frontend and backend.

Table 2: API Endpoint

API Endpoint	Method	Description
/api/users/register	POST	Register new user
/api/users/login	POST	Authenticate user
/api/doctors	GET	Fetch doctor list
/api/appointments	POST	Book appointment
/api/medicines	GET	Fetch medicine data
/api/chat	POST	AI health assistant

These APIs ensure efficient data exchange and enable modular development of the system.

5.6 SYSTEM ARCHITECTURE OVERVIEW (DESIGN PERSPECTIVE)

The system follows a three-tier architecture:

- ***Presentation Layer***

Developed using React.js, this layer handles user interaction and displays data.

- ***Application Layer***

Built using Node.js and Express.js, it processes business logic and handles API requests.

- ***Data Layer***

MongoDB is used to store and manage application data efficiently.

This layered architecture ensures scalability, maintainability, and better performance.

5.7 REAL-TIME SYSTEM IMPLEMENTATION

The BAYMAX system is designed to support real-time interactions to enhance user experience and improve healthcare accessibility. Real-time features are essential in healthcare applications where timely information plays a critical role in decision-making.

One of the key real-time features implemented in the system is the **doctor waiting count mechanism**. This feature dynamically updates the number of patients waiting for a doctor, allowing users to select doctors with shorter waiting times. This significantly reduces waiting time and improves patient satisfaction.

The system also supports real-time appointment updates. When a user books an appointment, the information is immediately reflected in the database and becomes visible to the doctor dashboard. This ensures synchronization between patients and healthcare providers.

Additionally, the AI health assistant responds to user queries instantly, providing quick preliminary guidance based on user input. This real-time interaction improves accessibility and allows users to receive immediate health-related suggestions.

The use of asynchronous API calls and efficient backend processing ensures that the system responds quickly without delays. These real-time capabilities make the BAYMAX system more interactive, efficient, and suitable for modern healthcare applications.

5.8 SUMMARY

This chapter presented the design aspects of the BAYMAX system, including user interface design, system workflow, database structure, and API architecture. The design focuses on creating a user-friendly and efficient healthcare platform that integrates multiple services into a single system.

Furthermore, the design emphasizes modular architecture, allowing different components of the system such as doctor recommendation, appointment booking, pharmacy services, and AI health assistance to function independently while remaining interconnected.

This modularity improves maintainability and enables future enhancements without affecting the overall system performance.

The user interface design plays a crucial role in ensuring accessibility and ease of use. A clean and intuitive layout helps users quickly navigate through various healthcare services such as searching for doctors, booking appointments, and accessing medical information.

The inclusion of real-time features, such as doctor waiting count and nearby pharmacy availability, enhances user decision-making and reduces delays in receiving care.

In addition, the system design incorporates secure data handling practices, ensuring that sensitive user information such as medical records and personal details is protected. Authentication mechanisms and role-based access control further strengthen system security and reliability.

The integration of intelligent features such as AI-based health suggestions and chatbot assistance adds significant value to the system by providing preliminary guidance and improving user engagement.

The structured design ensures that the system is scalable, secure, and capable of handling real-world healthcare requirements effectively.

CHAPTER 6
RESULTS AND DISCUSSION / PERFORMANCE ANALYSIS

6.1 INTRODUCTION

This chapter presents the results obtained after implementing the proposed system **BAYMAX – An Intelligent Integrated System for Smart Medical Assistance**. It explains how the developed modules perform in real-time conditions and evaluates the overall efficiency, usability, and performance of the system.

The purpose of this chapter is to analyse whether the proposed system successfully achieves the objectives defined in the earlier chapters. The system was tested with different user inputs and scenarios to ensure that all modules function correctly and provide the expected outputs.

6.2 IMPLEMENTATION RESULTS

After the development and integration of all modules, the system was tested under various conditions. The results show that the system operates correctly and provides the required healthcare services.

6.2.1 User Management Module

The user management module allows users to register and log in to the system securely. The system successfully validates user credentials and ensures that only authorized users can access the platform.

Result: The module ensures secure user authentication and efficient account management.

6.2.2 Doctor Consultation Module

The doctor consultation module enables users to search for doctors based on specialization or illness and book appointments online. The system successfully displays doctor details and allows users to schedule appointments.

Result: Users can easily find suitable doctors and book appointments without visiting hospitals physically

6.2.3 Pharmacy Module

The pharmacy module allows users to search for medicines and view information about available medicines. The system successfully displays medicine details and provides pharmacy-related information.

Result: Users can quickly access medicine information and locate nearby pharmacies.

6.2.4 Health Records Module

The Health Records Module is designed to securely store and manage user medical records, prescriptions, and other essential healthcare information within the system database. It enables users to access their health records anytime and from anywhere, ensuring convenience and continuity of care.

Result: The system provides a centralized platform for managing digital health records.

6.2.5 AI Health Assistant Module

The AI health assistant allows users to enter symptoms or health-related queries and receive preliminary guidance. The system analyses the input and provides relevant healthcare suggestions.

Result: The AI assistant helps users obtain basic health guidance, though final medical advice should always be obtained from healthcare professionals.

6.2.6 Emergency Assistance Module

The Emergency Assistance Module is designed to provide immediate support during critical situations by offering essential emergency healthcare contact information and quick access to nearby medical services. It enables users to easily locate hospitals, clinics, and pharmacies in their vicinity, ensuring timely medical attention when needed. The module may also include emergency helpline numbers, ambulance services, and first-aid guidance to assist users in urgent scenarios.

Result: The system helps users quickly find emergency support when required.

6.3 SYSTEM TESTING RESULTS

Different testing methods were used to evaluate the system functionality.

Table 3: Module Testing Result

Module	Status
User Management	Passed
Doctor Consultation	Passed
Pharmacy Module	Passed
Health Records	Passed
AI Health Assistant	Passed
Emergency Assistance	Passed

The testing results show that all modules function correctly and meet the system requirements.

6.4 PERFORMANCE ANALYSIS

Performance analysis was conducted to evaluate the system's efficiency, response time, and reliability.

Response Time

The system responds quickly to user requests, and most operations such as doctor search, appointment booking, and medicine search are completed within a few seconds.

Usability

The system provides a simple and user-friendly interface, allowing users with minimal technical knowledge to use the platform easily.

Reliability

The system performs consistently without major errors during testing and handles user requests effectively.

Scalability

The use of modern web technologies and database systems allows the platform to support multiple users simultaneously.

In addition to basic performance metrics, the system was evaluated under different usage scenarios to assess its robustness and efficiency. The system demonstrated stable performance even when handling multiple user requests simultaneously, indicating good concurrency handling capability. Database queries were optimized to ensure quick retrieval of information such as doctor details and medicine availability, which significantly reduced response time.

The system also showed efficient memory usage and minimal latency during operations, ensuring a smooth user experience. Stress testing was conducted to evaluate system behaviour under high load conditions, and the results indicated that the system can handle increased user traffic without major performance degradation. The use of modern web technologies and efficient backend processing contributes to the overall responsiveness and reliability of the system.

Furthermore, the system's modular architecture allows performance improvements to be implemented easily in specific components without affecting the entire system. These results confirm that the BAYMAX platform is capable of delivering consistent and efficient performance in real-world scenarios.

6.5 COMPARISON WITH EXISTING SYSTEMS

Table 4: Comparison Features

Feature	**Existing Systems**	**Proposed System (BAYMAX)**
Doctor Appointment	Available	Available
Medicine Purchase	Available	Available
AI Health Assistance	Limited	Available
Emergency Support	Limited	Available
Integrated Healthcare Services	Partial	Fully Integrated

The comparison shows that the proposed system provides more integrated healthcare services compared to many existing platforms.

6.6 ADVANTAGES

The proposed system BAYMAX offers several advantages over existing healthcare platforms by providing a more integrated and intelligent solution. One of the major advantages is the availability of multiple healthcare services within a single platform, eliminating the need for users to switch between different applications.

The system significantly improves time efficiency by enabling users to quickly find doctors, book appointments, and access medicines without delays. The inclusion of AI-based assistance enhances decision-making by providing preliminary healthcare guidance based on user input.

The platform also improves accessibility, as it can be accessed from anywhere using a web browser. The real-time waiting count feature helps in reducing waiting time by allowing users to choose doctors with fewer patients.

Additionally, the system offers an improved user experience through its simple and intuitive interface. The emergency support feature ensures quick access to critical services, thereby improving safety. Overall, these advantages make the system more effective, efficient, and user-friendly compared to traditional healthcare solutions.

6.7 LIMITATIONS AND CHALLENGES FACED

During the development and implementation of the BAYMAX healthcare system, several challenges were encountered. These challenges provided valuable learning experiences and helped improve the overall design of the system.

One of the primary challenges was handling real-time data updates, particularly in managing doctor availability and waiting counts. Ensuring accurate synchronization between multiple users required careful backend design and efficient database handling.

Another challenge was integrating AI-based health assistance. Since AI responses depend on external APIs and data models, maintaining accuracy and reliability of responses was a critical concern. The system was designed to provide only preliminary guidance and not replace professional medical advice.

Database management was also a significant challenge, especially in organizing healthcare data such as user records, appointments, and medical information in a scalable manner. MongoDB was chosen to address this issue due to its flexibility and performance.
Security was another important consideration. Protecting sensitive healthcare data required implementing authentication mechanisms, secure APIs, and proper access control.

Despite these challenges, the system was successfully developed and tested. The challenges faced during development helped in improving system efficiency, scalability, and reliability.

6.8 FUTURE ENHANCEMENTS AND INNOVATIONS

The BAYMAX healthcare system has been designed with scalability and flexibility in mind, allowing it to be enhanced with advanced technologies in the future. Several potential improvements can further increase the efficiency, intelligence, and usability of the system.

One of the major future enhancements is the integration of advanced Artificial Intelligence models for more accurate symptom analysis and disease prediction. By using machine learning algorithms trained on large healthcare datasets, the system can provide more precise recommendations and personalized healthcare insights.
Another important improvement is the development of a mobile application version of the system. A dedicated mobile application would allow users to access healthcare services anytime and anywhere, improving accessibility and convenience.
The system can also be enhanced by integrating real-time hospital and pharmacy databases. This would enable users to view live doctor availability, medicine stock status, and dynamic pricing, making the platform more reliable and practical for real-world usage.

Integration with wearable health devices such as smartwatches and fitness trackers is another potential enhancement. These devices can provide real-time health data such as heart rate, activity levels, and sleep patterns, which can be analysed by the system to offer personalized health recommendations.

Additionally, multilingual support can be implemented to make the system accessible to users from different regions and language backgrounds. This would significantly improve usability and inclusivity.
Future versions of the system may also include features such as online payment integration, telemedicine improvements, and electronic prescription management, making the platform more comprehensive.

Overall, these enhancements can transform the BAYMAX system into a fully advanced digital healthcare ecosystem capable of delivering intelligent, real-time, and personalized medical assistance.

6.9 DISCUSSION

The results indicate that the proposed system successfully achieves its objective of providing an integrated healthcare assistance platform. The system simplifies access to healthcare services and significantly reduces the effort required for users to find doctors, medicines, and other healthcare support. By offering a centralized and user-friendly interface, it enhances convenience and ensures that users can access essential services with ease.

The integration of intelligent assistance features further improves the overall user experience by providing quick responses, relevant suggestions, and preliminary health guidance. This helps users make informed decisions at an early stage, potentially reducing the severity of health issues.

Additionally, the system demonstrates reliable performance, secure data handling, and effective module integration, which contribute to its robustness and practicality.

Overall, the system proves to be technically feasible, efficient, and highly beneficial in improving healthcare accessibility. It addresses existing challenges in traditional healthcare systems and offers a scalable solution that can be further enhanced with advanced technologies and additional features in the future.

CHAPTER 7
SUMMARY AND CONCLUSION

7.1 SUMMARY OF THE WORK

The project **"BAYMAX – An Intelligent Integrated System for Smart Medical Assistance"** was developed with the objective of improving healthcare accessibility by integrating multiple medical services into a single digital platform. The system aims to simplify the process of finding doctors, booking appointments, accessing pharmacy services, and receiving basic healthcare guidance.

During the development of the project, several stages were completed including requirement analysis, system design, implementation, testing, and performance evaluation. The system was developed using modern web technologies such as **React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for the database**. These technologies provide scalability, efficiency, and flexibility for the application.

The platform includes several functional modules such as **User Management, Doctor Consultation, Pharmacy Services, Health Records Management, AI Health Assistant, and Emergency Assistance**. Each module was designed to provide specific healthcare services and improve the overall user experience. The AI health assistant helps users obtain basic health guidance by analysing symptoms and suggesting possible healthcare actions.

However, the system is designed only to provide **preliminary assistance and does not replace professional medical consultation**. The integration of digital healthcare services helps users save time, reduce effort, and access medical services more conveniently.

The testing results confirmed that the system functions correctly and efficiently. The platform provides a user-friendly interface and allows users to access healthcare services easily through a web-based environment.

7.2 CONCLUSION

The BAYMAX healthcare assistance system represents a significant step towards improving healthcare accessibility through the integration of modern digital technologies.

By combining multiple healthcare services such as doctor consultation, appointment booking, pharmacy assistance, and health guidance into a single platform, the system simplifies the process of accessing medical services and reduces the complexity faced by users.

The implementation of intelligent features such as AI-based health assistance and real-time doctor recommendation enhances the overall effectiveness of the system and supports users in making informed healthcare decisions.

The system successfully demonstrates how technology can be used to bridge the gap between patients and healthcare providers, improving efficiency and convenience. Moreover, the system is designed with scalability and flexibility in mind, allowing it to adapt to future technological advancements and increasing user demands.

The integration of secure data handling mechanisms ensures the protection of sensitive healthcare information, which is crucial in modern digital systems. Although the system provides valuable assistance, it is important to emphasize that it is not a replacement for professional medical diagnosis. Instead, it serves as a supportive tool that enhances healthcare accessibility and awareness.

In conclusion, the BAYMAX system is a practical, efficient, and innovative solution that enhances the healthcare experience for users. The system ensures data security, reliability, and scalability, making it suitable for real-world use. It also improves healthcare accessibility and awareness, especially for users with limited access to traditional services. Overall, the system represents a significant step toward the digital transformation of healthcare.

BIBLIOGRAPHY / REFERENCES

1. JOURNAL REFERENCES

1. Topol, E. (2019). *High-performance medicine: The convergence of human and artificial intelligence.* Nature Medicine, 25(1), 44–56.
2. Jiang, F., Jiang, Y., Zhi, H., Dong, Y., Li, H., Ma, S., Wang, Y., Dong, Q., Shen, H., & Wang, Y. (2017). *Artificial intelligence in healthcare: Past, present and future.* Stroke and Vascular Neurology, 2(4), 230–243.

2. BOOK REFERENCES

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach.* Pearson Education.
2. Reddy, C. K., & Aggarwal, C. C. (2015). *Healthcare Data Analytics.* Chapman and Hall/CRC.

INTERNET SOURCES

1. Apollo Hospitals – Apollo 24|7 Healthcare Platform
   https://www.apollo247.com
2. React Official Documentation
   https://react.dev
3. Node.js Official Documentation
   https://nodejs.org
4. MongoDB Official Documentation
   https://www.mongodb.com
5. Express.js Documentation
   https://expressjs.com

APPENDIX

APPENDIX A – SYSTEM SCREENSHOTS

This section contains screenshots of the developed **BAYMAX healthcare assistance system** to demonstrate the user interface and functionality of the application. The screenshots help in understanding how different modules of the system operate and how users interact with the platform.

Some of the important screenshots that can be included are:

1. **Home Page** – Displays the main interface of the BAYMAX platform with navigation options and healthcare services.

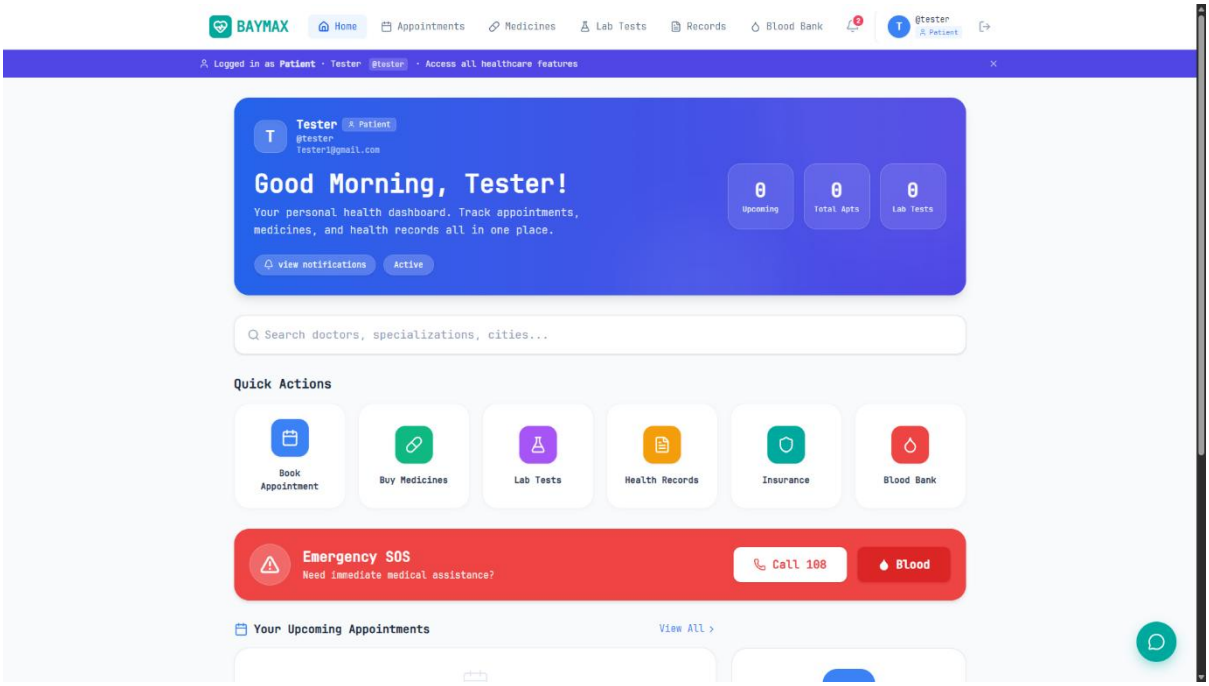

Fig 4: Home Page

2. **User Registration Page** – Allows new users to create an account in the system.

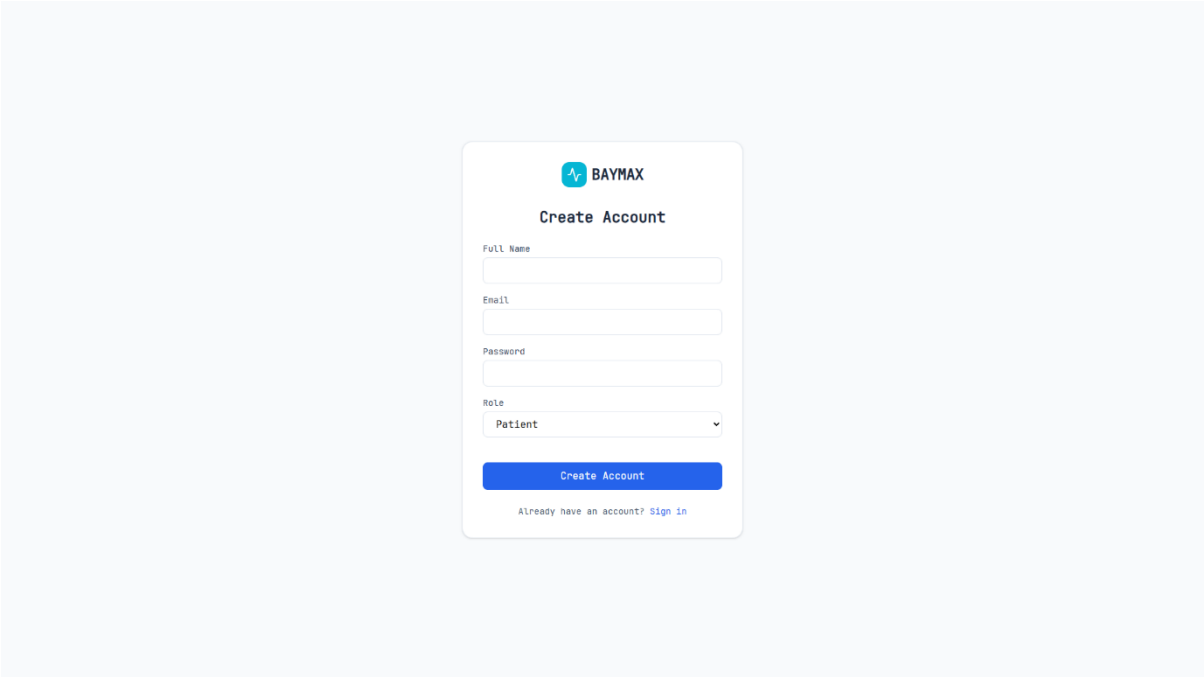

Fig 5: User Registration Page

3. **Login Page** – Enables registered users to log into the system securely.

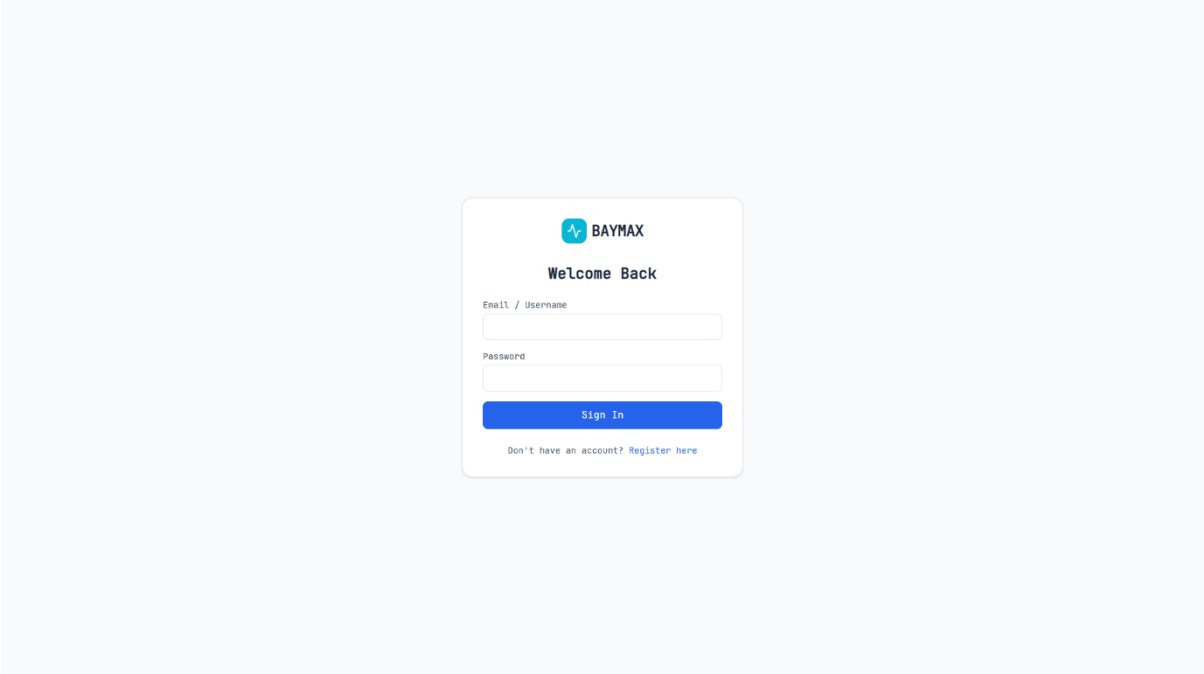

Fig 6: Login Page

4. **Patient Dashboard** – Shows the main dashboard where users can access healthcare services such as doctor consultation, medicine purchase, and health records.

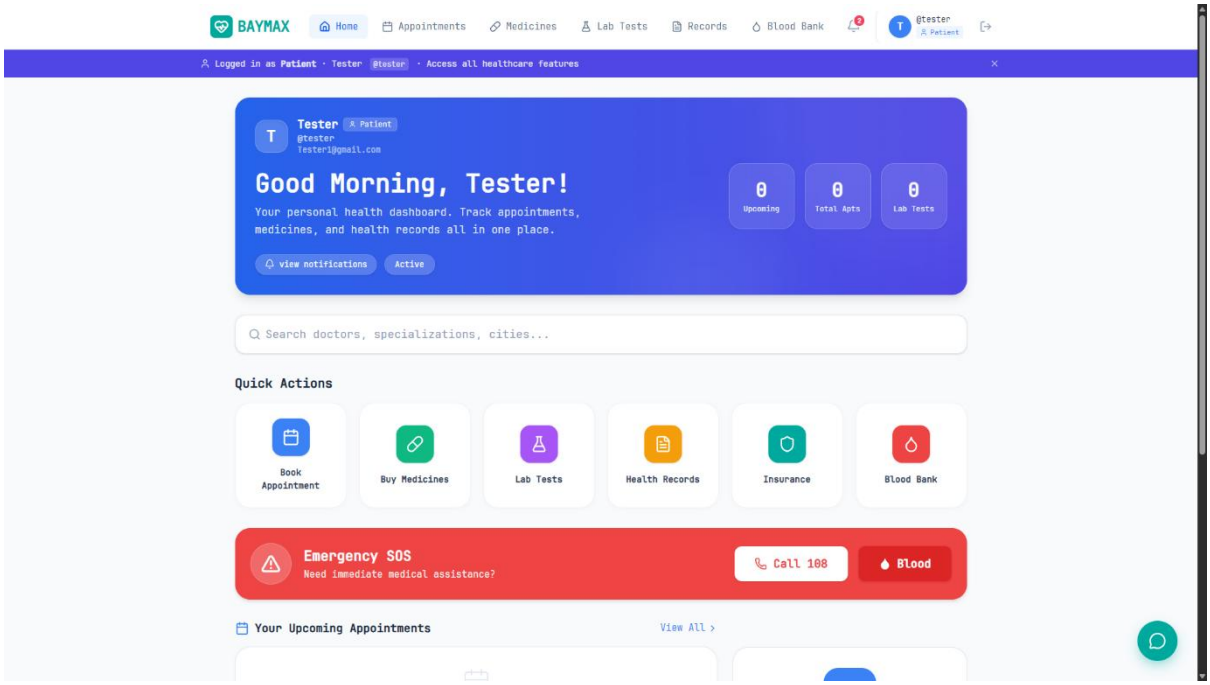

Fig 7: Patient Dashboard

5. **Doctor Appointment Page** – Displays the list of available doctors and allows users to book appointments.

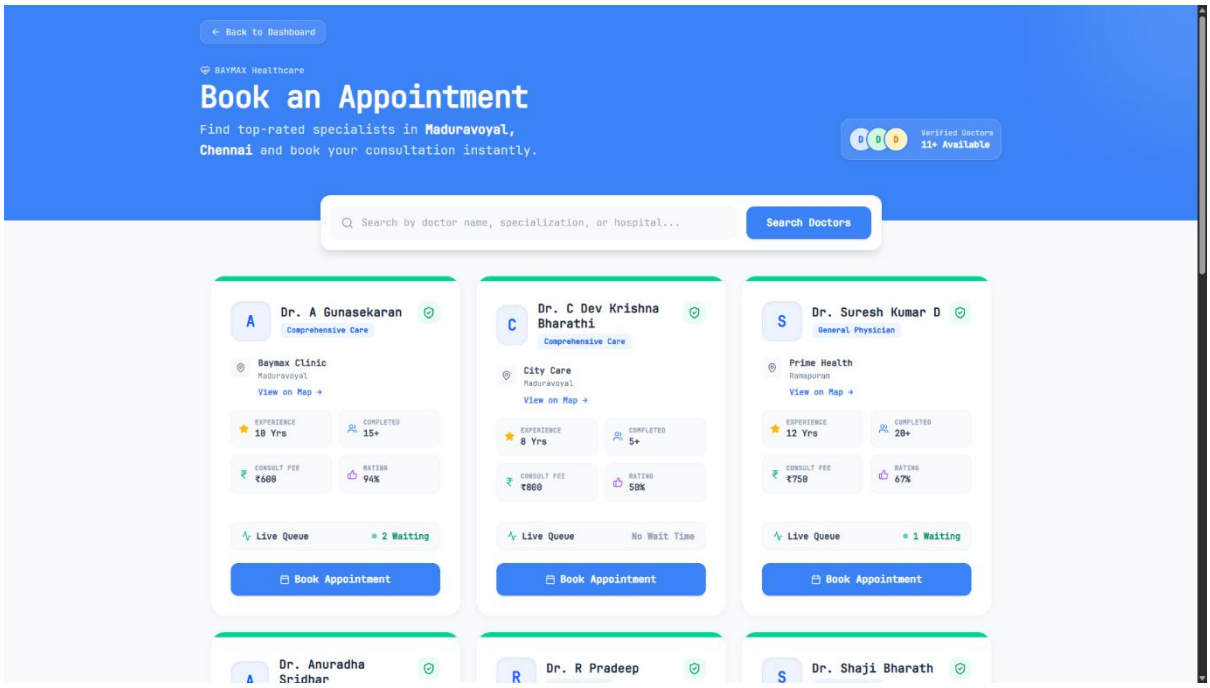

Fig 8: Doctor Appointment Page

6. **Buy Medicines Page** – Shows medicine details and pharmacy information.

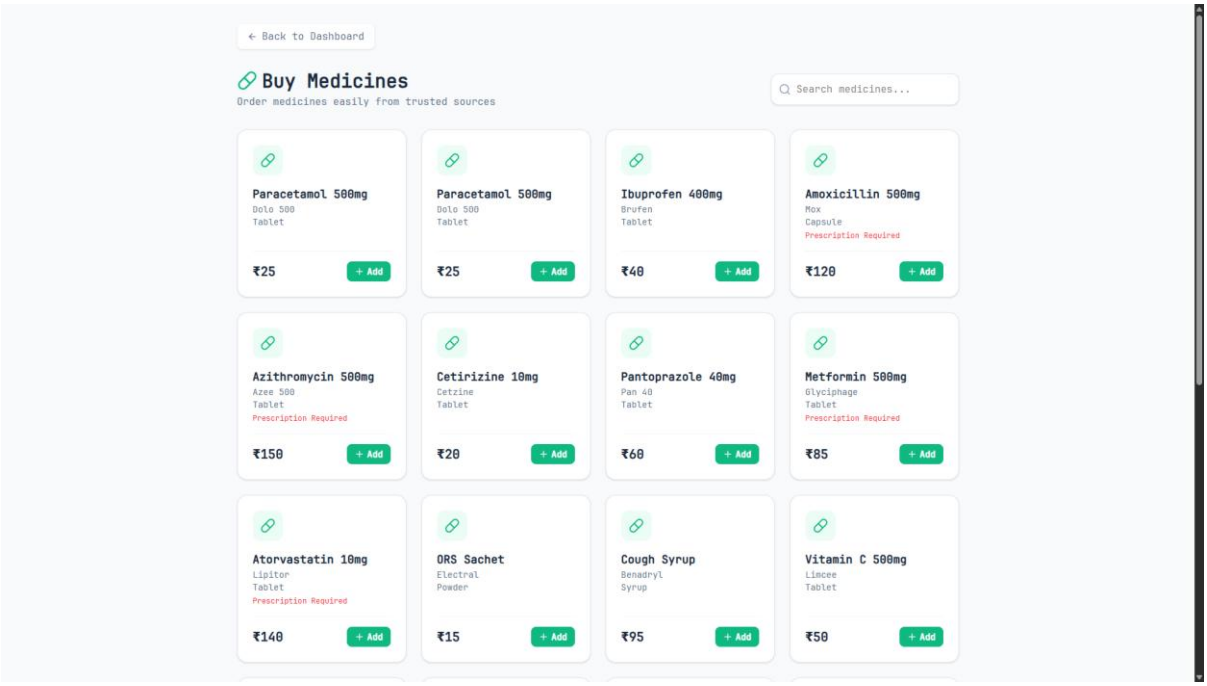

Fig 9: Buy Medicines Page

7. **Health Records Page** – Displays user medical records and prescriptions.

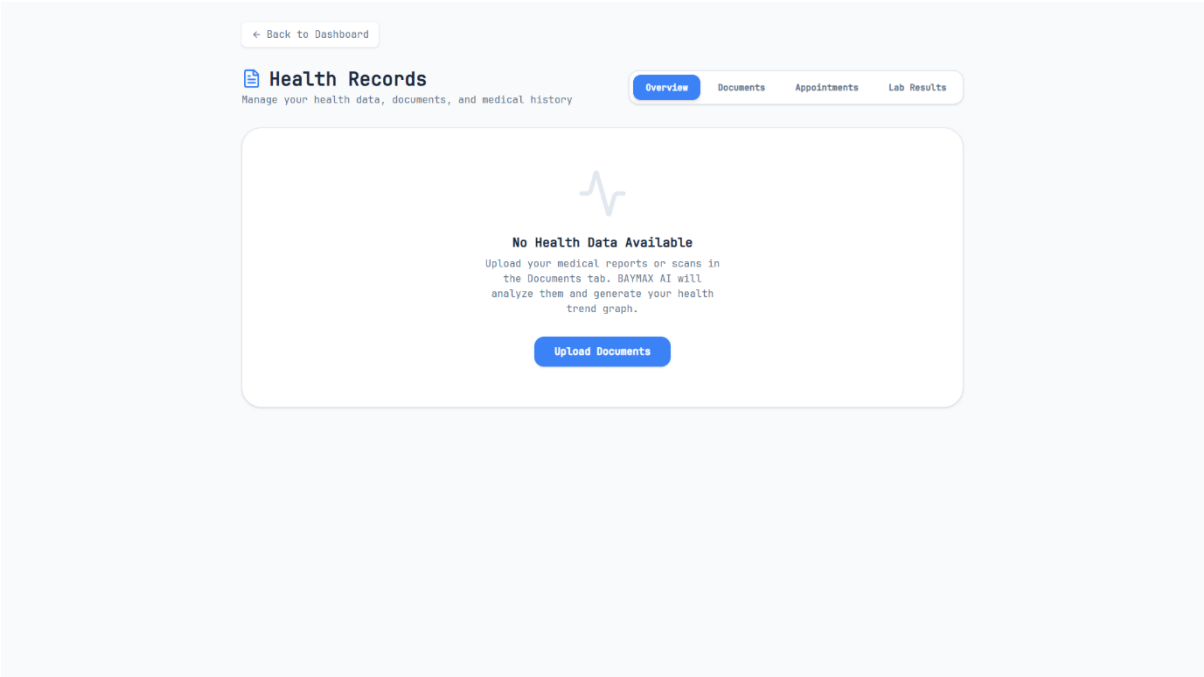

Fig 10: Health Records Page

8. **AI Health Assistant Interface** – Shows the chatbot or AI interface that provides basic healthcare guidance.

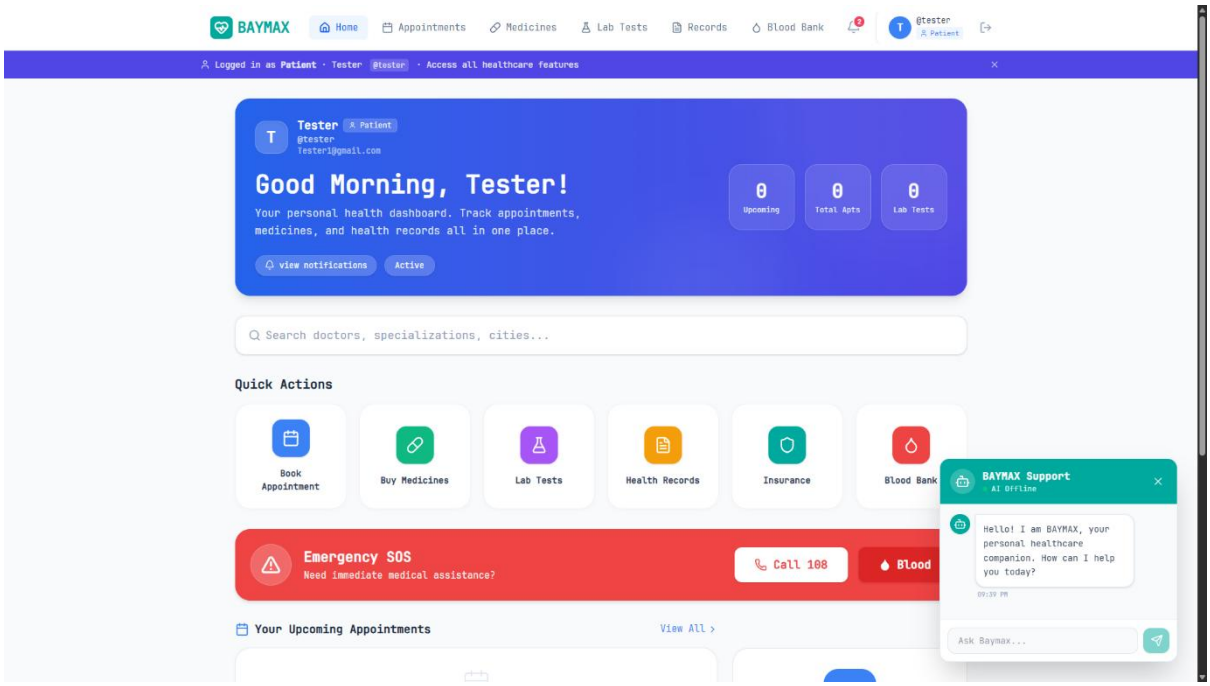

Fig 11: AI Health Assistant Interface

9. **Emergency Assistance Page** – Displays emergency contact numbers and healthcare support services.

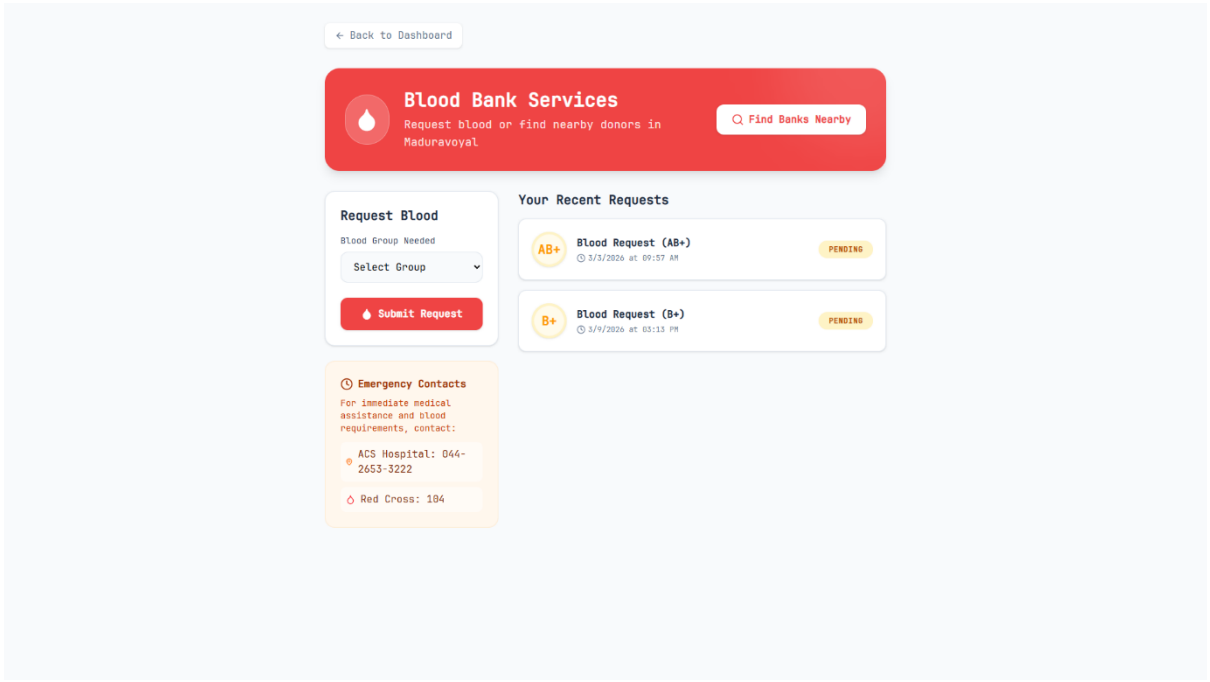

Fig 12: Emergency Assistance Page

APPENDIX B – SAMPLE TEST CASES

The following table shows sample test cases used to verify the functionality of the system.

Table 4: Test Case

Test Case ID	Module	Test Description	Expected Result	Status
TC01	User Registration	User enters valid details and registers	User account created successfully	Passed
TC02	User Login	User enters valid login credentials	User logged into the system	Passed
TC03	Doctor Search	User searches for doctors by specialization	List of doctors displayed	Passed
TC04	Appointment Booking	User selects doctor and books appointment	Appointment confirmed	Passed
TC05	Medicine Search	User searches for medicine	Medicine information displayed	Passed
TC06	Health Records	User views health records	Records displayed correctly	Passed
TC07	AI Health Assistant	User enters health query	System provides health suggestion	Passed
TC08	Emergency Assistance	User accesses emergency module	Emergency contacts displayed	Passed

The test results indicate that all system modules function correctly and meet the required system objectives.

APPENDIX C – SAMPLE SOURCE CODE

This section presents selected portions of the source code used in the development of the BAYMAX healthcare system. The code snippets demonstrate the implementation of key functionalities such as user authentication, doctor management, appointment booking, and AI-based health assistance.

The purpose of including source code is to provide technical insight into the system development and to illustrate how different modules are implemented using modern web technologies.

C1. Back – End

C1.1 config

C1.1.1 db.js:

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);

    console.log("MongoDB Connected");
    console.log("Host:", conn.connection.host);
    console.log("Database:", conn.connection.name);

  } catch (error) {
    console.error("DB Error:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

C1.2 controllers

C1.2.1 adminController.js

```
const User = require('../models/User');
const DoctorProfile = require('../models/DoctorProfile');

exports.getPendingDoctors = async (req, res) => {
  try {
    const pendingDoctors = await User.find({ role: 'doctor', isApproved: false
}).select('-password');

    // Fetch profiles for these doctors
    const doctorsWithProfiles = await Promise.all(pendingDoctors.map(async (doc)
=> {
      const profile = await DoctorProfile.findOne({ user: doc._id });
      return { ...doc.toObject(), profile };
    }));

    res.json(doctorsWithProfiles);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

exports.approveDoctor = async (req, res) => {
  try {
    const doctor = await User.findById(req.params.id);
    if (!doctor || doctor.role !== 'doctor') {
      return res.status(404).json({ message: 'Doctor not found' });
    }
    doctor.isApproved = true;
    await doctor.save();
    res.json({ message: 'Doctor approved successfully' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
```

```
};

exports.rejectDoctor = async (req, res) => {
  try {
    const doctor = await User.findById(req.params.id);
    if (!doctor || doctor.role !== 'doctor') {
      return res.status(404).json({ message: 'Doctor not found' });
    }
    await User.findByIdAndDelete(req.params.id);
    await DoctorProfile.findOneAndDelete({ user: req.params.id });
    res.json({ message: 'Doctor rejected and removed' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

C1.2.2 authController.js

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const DoctorProfile = require("../models/DoctorProfile");

/**
 * Generate JWT Token
 */
const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET || "fallback_secret", {
    expiresIn: "30d",
  });
};

/**
 * @desc    Register new user
 * @route   POST /api/auth/register
 * @access  Public
 */
```

```
exports.register = async (req, res) => {
  try {
    const {
      name,
      email,
      password,
      role,
      specialization,
      experience,
      hospital,
    } = req.body;

    // Check if user already exists
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: "User already exists" });
    }

    // Auto-approve patients & admins
    const isApproved = role === "patient" || role === "admin";

    // Create user
    const user = await User.create({
      name,
      email,
      password,
      role,
      isApproved,
    });

    // If role is doctor, create doctor profile
    if (role === "doctor") {
      await DoctorProfile.create({
        user: user._id,
        specialization,
```

```
        experience,
        hospital,
        address: "Maduravoyal, Chennai", // As per your requirement
        isApproved: false,
      });
    }

    // Send response
    return res.status(201).json({
      _id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      isApproved: user.isApproved,
      token: generateToken(user._id),
    });
  } catch (error) {
    console.error(" FULL ERROR STACK:\n", error.stack); // VERY IMPORTANT
    return res.status(500).json({ message: error.message });
  }
};

/**
 * @desc    Login user
 * @route   POST /api/auth/login
 * @access  Public
 */
exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;

    /**
     * Hardcoded Admin Login (Keeping your logic unchanged)
     */
    if (
```

```
  (email === "admin" || email === "admin@baymax.com") &&
  password === "admin@baymax"
) {
  let adminUser = await User.findOne({ role: "admin" });

  if (!adminUser) {
    adminUser = await User.create({
      name: "Admin",
      email: "admin@baymax.com",
      password: "admin@baymax",
      role: "admin",
      isApproved: true,
    });
  }

  return res.json({
    _id: adminUser._id,
    name: adminUser.name,
    email: adminUser.email,
    role: adminUser.role,
    isApproved: adminUser.isApproved,
    token: generateToken(adminUser._id),
  });
}

// Find user by email
const user = await User.findOne({ email });

if (user && (await user.matchPassword(password))) {
  // Doctor approval check
  if (user.role === "doctor" && !user.isApproved) {
    return res.status(403).json({
      message: "Your account is pending admin approval.",
    });
  }
```

```
      return res.json({
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        isApproved: user.isApproved,
        token: generateToken(user._id),
      });
    } else {
      return res.status(401).json({
        message: "Invalid email or password",
      });
    }
  } catch (error) {
    console.error("Login Error:", error.message);
    return res.status(500).json({ message: "Server Error" });
  }
};
```

C1.2.3 doctorController.js

```
const Appointment = require("../models/Appointment");
const DoctorProfile = require("../models/DoctorProfile");

exports.getAppointments = async (req, res) => {
  try {
    const appointments = await Appointment.find({ doctor: req.user._id })
      .populate("patient", "name email")
      .sort({ createdAt: -1 });
    res.json(appointments);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```
exports.updateAppointmentStatus = async (req, res) => {
  try {
    const { status } = req.body;

    const appointment = await Appointment.findById(req.params.id);
    if (!appointment) {
      return res.status(404).json({ message: "Appointment not found" });
    }

    // Authorization check
    if (appointment.doctor.toString() !== req.user._id.toString()) {
      return res.status(401).json({ message: "Not authorized" });
    }

    const previousStatus = appointment.status;

    // Update status
    appointment.status = status;
    await appointment.save();

    //  Handle waiting count properly
    const profile = await DoctorProfile.findOne({ user: req.user._id });

    if (profile) {
      //  Increase waiting only when newly approved
      if (status === "approved" && previousStatus !== "approved") {
        profile.waitingCount += 1;
      }

      //  Optional: if rejected after being approved
      if (
        status === "rejected" &&
        previousStatus === "approved" &&
        profile.waitingCount > 0
      ) {
```

```
      profile.waitingCount -= 1;
    }

    await profile.save();
  }

  res.json({
    message: "Appointment status updated",
    appointment,
    profile,
  });
 } catch (error) {
  res.status(500).json({ message: error.message });
 }
};

exports.markAppointmentDone = async (req, res) => {
 try {
  const appointment = await Appointment.findById(req.params.id);
  if (!appointment)
    return res.status(404).json({ message: "Appointment not found" });

  if (appointment.doctor.toString() !== req.user._id.toString()) {
    return res.status(401).json({ message: "Not authorized" });
  }

  if (appointment.status === "completed") {
    return res.status(400).json({ message: "Already completed" });
  }

  appointment.status = "completed";
  await appointment.save();

  const profile = await DoctorProfile.findOne({ user: req.user._id });
  if (profile.waitingCount > 0) profile.waitingCount -= 1;
```

```
    profile.completedCount += 1;
    await profile.save();

    res.json({ message: "Appointment marked as done", profile });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

exports.getProfile = async (req, res) => {
  try {
    const profile = await DoctorProfile.findOne({ user: req.user._id });
    res.json(profile);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

exports.updateWaitingCount = async (req, res) => {
  try {
    const { action } = req.body;

    const profile = await DoctorProfile.findOne({ user: req.user._id });
    if (!profile) {
      return res.status(404).json({ message: "Profile not found" });
    }

    if (action === "increment") {
      profile.waitingCount += 1;
    }

    if (action === "decrement" && profile.waitingCount > 0) {
      profile.waitingCount -= 1;
      profile.completedCount += 1; //  nice feature
    }
```

```
    await profile.save();

    res.json(profile);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

C1.2.4 medicineController.js

```
const Medicine = require('../models/Medicine');

// GET all medicines
const getMedicines = async (req, res) => {
  try {
    const medicines = await Medicine.find();

    console.log(" Medicines fetched:", medicines.length);

    res.json(medicines);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
// ADD medicine (for testing)
const addMedicine = async (req, res) => {
  try {
    const medicine = new Medicine(req.body);
    const saved = await medicine.save();
    res.status(201).json(saved);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};
module.exports = { getMedicines, addMedicine };
```

C1.2.5 patientController.js

```
const User = require("../models/User");
const DoctorProfile = require("../models/DoctorProfile");
const Appointment = require("../models/Appointment");
const Medicine = require("../models/Medicine");
const Pharmacy = require("../models/Pharmacy");
const Cart = require("../models/Cart");
const LabTest = require("../models/LabTest");
const LabBooking = require("../models/LabBooking");
const BloodRequest = require("../models/BloodRequest");

// --- DOCTOR & APPOINTMENT ---
const getDoctors = async (req, res) => {
  try {
    const doctors = await User.find({
      role: "doctor",
      isApproved: true,
    }).select("-password");
    const doctorProfiles = await DoctorProfile.find({
      user: { $in: doctors.map((d) => d._id) },
    }).populate("user", "name email");
    res.json(doctorProfiles);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const bookAppointment = async (req, res) => {
  try {
    const { doctorId, doctor, date, time } = req.body;

    // Accept both doctorId or doctor (frontend safe)
    const finalDoctorId = doctorId || doctor;

    // Debug logs (important)
```

```
    console.log("Incoming Body:", req.body);
    console.log("Final Doctor ID:", finalDoctorId);
    console.log("Logged User:", req.user);

    // Check doctor exists
    const doctorData = await User.findById(finalDoctorId);
    if (!doctorData || doctorData.role !== "doctor") {
      return res.status(404).json({ message: "Doctor not found" });
    }

    // Update waiting count (optional)
    const profile = await DoctorProfile.findOne({ user: finalDoctorId });
    if (profile) {
      profile.waitingCount += 1;
      await profile.save();
    }

    // Create appointment
    const appointment = new Appointment({
      patient: req.user?.id, // safe access
      doctor: finalDoctorId,
      date: date || new Date().toISOString().split("T")[0],
      time: time || "10:00 AM",
      status: "pending",
    });

    const savedAppointment = await appointment.save();

    res.status(201).json(savedAppointment);
  } catch (error) {
    console.log("BOOKING ERROR:", error);
    res.status(500).json({ message: "Booking failed", error: error.message });
  }
};
const getAppointments = async (req, res) => {
```

```
  try {
    const appointments = await Appointment.find({ patient: req.user.id })
      .populate("doctor", "name email")
      .sort({ createdAt: -1 });

    const enhancedAppointments = await Promise.all(
      appointments.map(async (apt) => {
        const docProfile = await DoctorProfile.findOne({
          user: apt.doctor._id,
        });
        return { ...apt._doc, doctorProfile: docProfile };
      }),
    );
    res.json(enhancedAppointments);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// --- MEDICINES & CART ---
const getMedicines = async (req, res) => {
  try {
    const search = req.query.search || "";
    const query = search ? { name: { $regex: search, $options: "i" } } : {};
    const medicines = await Medicine.find(query).populate("pharmacy");
    res.json(medicines);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const getCart = async (req, res) => {
  try {
    let cart = await Cart.findOne({ patient: req.user.id }).populate(
      "items.medicine",
```

```
    );
    if (!cart) {
      cart = await Cart.create({ patient: req.user.id, items: [] });
    }
    res.json(cart);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const addToCart = async (req, res) => {
  const { medicineId } = req.body;
  try {
    let cart = await Cart.findOne({ patient: req.user.id });
    if (!cart) cart = new Cart({ patient: req.user.id, items: [] });

    const itemIndex = cart.items.findIndex(
      (item) => item.medicine.toString() === medicineId,
    );
    if (itemIndex > -1) {
      cart.items[itemIndex].quantity += 1;
    } else {
      cart.items.push({ medicine: medicineId, quantity: 1 });
    }
    await cart.save();
    cart = await cart.populate("items.medicine");
    res.json(cart);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const removeFromCart = async (req, res) => {
  try {
    let cart = await Cart.findOne({ patient: req.user.id });
```

```
    if (!cart) return res.status(404).json({ message: "Cart not found" });

    cart.items = cart.items.filter(
      (item) => item.medicine.toString() !== req.params.medicineId,
    );
    await cart.save();
    cart = await cart.populate("items.medicine");
    res.json(cart);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const checkoutCart = async (req, res) => {
  try {
    let cart = await Cart.findOne({ patient: req.user.id });
    if (cart) {
      cart.items = [];
      await cart.save();
    }
    res.json({ message: "Checkout successful" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// --- LAB TESTS ---
const getLabTests = async (req, res) => {
  try {
    const tests = await LabTest.find();
    res.json(tests);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```
const bookLabTest = async (req, res) => {
  const { labTestId } = req.body;
  try {
    const booking = new LabBooking({
      patient: req.user.id,
      labTest: labTestId,
      status: "pending",
    });
    await booking.save();
    res.status(201).json(booking);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const getLabBookings = async (req, res) => {
  try {
    const bookings = await LabBooking.find({ patient: req.user.id }).populate(
      "labTest",
    );
    res.json(bookings);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// --- BLOOD BANK ---
const getBloodRequests = async (req, res) => {
  try {
    const requests = await BloodRequest.find({ patient: req.user.id }).sort({
      createdAt: -1,
    });
    res.json(requests);
  } catch (error) {
```

```
    res.status(500).json({ message: error.message });
  }
};

const createBloodRequest = async (req, res) => {
  const { bloodGroup } = req.body;
  try {
    const request = new BloodRequest({
      bloodGroup,
      patient: req.user.id,
      status: "pending",
    });
    await request.save();
    res.status(201).json(request);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

module.exports = {
  getDoctors,
  bookAppointment,
  getAppointments,
  getMedicines,
  getCart,
  addToCart,
  removeFromCart,
  checkoutCart,
  getLabTests,
  bookLabTest,
  getLabBookings,
  getBloodRequests,
  createBloodRequest,
};
```

C1.3 middleware

C1.3.1 auth.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')
  ) {
    try {
      token = req.headers.authorization.split(' ')[1];

      const decoded = jwt.verify(
        token,
        process.env.JWT_SECRET || 'fallback_secret'
      );

      req.user = await User.findById(decoded.id).select('-password');

      return next(); // IMPORTANT
    } catch (error) {
      return res.status(401).json({
        message: 'Not authorized, token failed',
      });
    }
  }

  return res.status(401).json({
    message: 'Not authorized, no token',
  });
};
```

```
const admin = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    return next(); //  IMPORTANT
  }

  return res.status(403).json({
    message: 'Not authorized as admin',
  });
};

const doctor = (req, res, next) => {
  if (req.user && req.user.role === 'doctor') {
    if (!req.user.isApproved) {
      return res.status(403).json({
        message: 'Your account is pending admin approval.',
      });
    }

    return next(); //  IMPORTANT
  }

  return res.status(403).json({
    message: 'Not authorized as doctor',
  });
};

module.exports = { protect, admin, doctor };
```

C.1.3.2 authMiddleware.js

```
const jwt = require("jsonwebtoken");

const protect = async (req, res, next) => {
  let token;

  if (
```

```
    req.headers.authorization &&
    req.headers.authorization.startsWith("Bearer")
  ) {
    try {
      token = req.headers.authorization.split(" ")[1];

      const decoded = jwt.verify(
        token,
        process.env.JWT_SECRET || "fallback_secret",
      );

      const User = require("../models/User");
      req.user = await User.findById(decoded.id).select("-password");

      return next(); // always return next
    } catch (error) {
      return res.status(401).json({ message: "Not authorized, token failed" });
    }
  }

  //  Ensure function ALWAYS returns response
  return res.status(401).json({ message: "Not authorized, no token" });
};

const admin = (req, res, next) => {
  if (req.user && req.user.role === "admin") {
    return next(); //  good practice
  } else {
    return res.status(401).json({ message: "Not authorized as an admin" });
  }
};

module.exports = { protect, admin };
```

C.1.3.3 errorMidleware.js

```
const notFound = (req, res, next) => {
  const error = new Error(`Not Found - ${req.originalUrl}`);
  res.status(404);
  next(error);
};

const errorHandler = (err, req, res, next) =>
{
  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;

  res.status(statusCode).json({
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? null : err.stack,
  });
};

module.exports = { notFound, errorHandler };
```

C.1.3.4 validationMiddleware.js

```
const { validationResult } = require('express-validator');

const validateRequest = (req, res, next) =>
{
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  next();
};

module.exports = { validateRequest };
```

C.1.4 Models

C.1.4.1 Appointment.js

```
const mongoose = require('mongoose');

const appointmentSchema = new mongoose.Schema({
  patient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  doctor: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  date: { type: String, required: true },
  time: { type: String, required: true },
  status: { type: String, enum: ['pending', 'approved', 'rejected', 'completed'], default:
'pending' }
}, { timestamps: true });

module.exports = mongoose.model('Appointment', appointmentSchema);
```

C.1.4.2 BloodRequest.js

```
const mongoose = require('mongoose');

const bloodRequestSchema = new mongoose.Schema({
  bloodGroup: { type: String, required: true },
  patient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  status: { type: String, enum: ['pending', 'fulfilled', 'rejected'], default: 'pending' },
}, { timestamps: true });

module.exports = mongoose.model('BloodRequest', bloodRequestSchema);
```

C.1.4.3 Cart.js

```
const mongoose = require('mongoose');

const cartSchema = new mongoose.Schema({
  patient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  items: [
    {
      medicine: { type: mongoose.Schema.Types.ObjectId, ref: 'Medicine', required:
true },
```

```
    quantity: { type: Number, default: 1 }
   }
  ]
}, { timestamps: true });
module.exports = mongoose.model('Cart', cartSchema);
```

C.1.4.4 DoctorProfile.js

```
const mongoose = require('mongoose');

const doctorProfileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  specialization: { type: String, required: true },
  experience: { type: Number, required: true },
  hospital: { type: String, required: true },
  address: { type: String, default: 'Maduravoyal, Chennai' },
  latitude: { type: Number, default: 13.0645 },
  longitude: { type: Number, default: 80.1654 },
  waitingCount: { type: Number, default: 0 },
  completedCount: { type: Number, default: 0 }
}, { timestamps: true });

module.exports = mongoose.model('DoctorProfile', doctorProfileSchema);
```

C.1.4.5 LabBooking.js

```
const mongoose = require('mongoose');

const labBookingSchema = new mongoose.Schema({
  patient: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  labTest: { type: mongoose.Schema.Types.ObjectId, ref: 'LabTest', required: true
},
  status: { type: String, enum: ['pending', 'completed'], default: 'pending' }
}, { timestamps: true });

module.exports = mongoose.model('LabBooking', labBookingSchema);
```

C.1.4.6 LabTest.js

```
const mongoose = require('mongoose');

const labTestSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, required: true },
  description: { type: String },
}, { timestamps: true });

module.exports = mongoose.model('LabTest', labTestSchema);
```

C.1.4.7 Medicine.js

```
const mongoose = require('mongoose');

const medicineSchema = new mongoose.Schema({
  name: { type: String, required: true },
  type: String,
  price: Number,
  availability: { type: Boolean, default: true }
}, {
  timestamps: true
});

module.exports = mongoose.model('Medicine', medicineSchema);
```

C.1.4.8 Pharmacy.js

```
const mongoose = require('mongoose');

const pharmacySchema = new mongoose.Schema({
  name: { type: String, required: true },
  address: { type: String, default: 'Maduravoyal, Chennai' },
  latitude: { type: Number, required: true },
  longitude: { type: Number, required: true },
}, { timestamps: true });
```

```
module.exports = mongoose.model('Pharmacy', pharmacySchema);
```

C.1.4.9 User.js

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    role: {
      type: String,
      enum: ["patient", "doctor", "admin"],
      default: "patient",
    },
    isApproved: { type: Boolean, default: false },
  },
  { timestamps: true },
);

userSchema.pre("save", async function ()
{
  if (!this.isModified("password")) return;

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
userSchema.methods.matchPassword = async function (enteredPassword)
 {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model("User", userSchema);
```

C.1.5 routes

C.1.5.1 adminRoutes.js

```
const express = require('express');
const { getPendingDoctors, approveDoctor, rejectDoctor } =
require('../controllers/adminController');
const { protect, admin } = require('../middleware/auth');
const router = express.Router();

router.use(protect, admin);

router.get('/pending-doctors', getPendingDoctors);
router.put('/approve-doctor/:id', approveDoctor);
router.delete('/reject-doctor/:id', rejectDoctor);

module.exports = router;
```

C.1.5.2 aiRoutes.js

```
const express = require("express");
const router = express.Router();
const axios = require("axios");

router.post("/chat", async (req, res) => {
  try {
    const { message } = req.body;

    console.log("API KEY:", process.env.OPENROUTER_API_KEY);

    const response = await axios.post(
      "https://openrouter.ai/api/v1/chat/completions",
      {
        model: "meta-llama/llama-3-8b-instruct", // ✅ FIXED MODEL
        messages: [
          {
            role: "system",
            content: `
```

```
You are a professional healthcare assistant designed to provide general medical guidance.

Guidelines:
- Use a clear, calm, and professional tone at all times.
- Do NOT use roleplay expressions (e.g., *beep*, *smile*, *sad*) or informal language.
- Do NOT use emojis.
- Provide accurate, easy-to-understand, and structured responses.
- Present information in short paragraphs or bullet points where appropriate.
- Offer general health advice based on symptoms, but avoid making definitive diagnoses.
- Always include a disclaimer encouraging the user to consult a qualified medical professional for proper diagnosis and treatment.
- If symptoms appear serious or urgent, clearly advise seeking immediate medical attention.

Your goal is to assist users with helpful, safe, and professional health information while maintaining clarity and reliability.
`,
        },
        {
          role: "user",
          content: message,
        },
      ],
    },
    {
      headers: {
        Authorization: `Bearer ${process.env.OPENROUTER_API_KEY}`,
        "Content-Type": "application/json",
        "HTTP-Referer": "http://localhost:5000",
        "X-Title": "BAYMAX AI",
      },
    },
```

```
    );
    res.json({
      content: response.data.choices[0].message.content,
    });
  } catch (error) {
    console.error("FULL ERROR:", JSON.stringify(error.response?.data, null, 2));

    res.status(500).json({
      error: error.response?.data || error.message,
    });
  }
});

module.exports = router;
```

C1.5.3 authRoutes.js

```
const express = require('express');
const { register, login } = require('../controllers/authController');
const router = express.Router();

router.post('/register', register);
router.post('/login', login);

module.exports = router;
```

C.1.5.4 doctorRoutes.js

```
const express = require('express');
const { getAppointments, updateAppointmentStatus, markAppointmentDone,
getProfile, updateWaitingCount } = require('../controllers/doctorController');
const { protect, doctor } = require('../middleware/auth');
const router = express.Router();

router.use(protect, doctor);

router.get('/appointments', getAppointments);
```

```
router.put('/appointment/:id/status', updateAppointmentStatus);
router.put('/appointment/:id/done', markAppointmentDone);
router.get('/profile', getProfile);
router.put('/waiting-count', updateWaitingCount);

module.exports = router;
```

C.1.5.5 labtestsRoute.js

```
const express = require('express');
const router = express.Router();
const LabTest = require('../models/LabTest');

// GET all lab tests
router.get('/', async (req, res) => {
  try {
    const tests = await LabTest.find();
    res.json(tests);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

C.1.5.6 medicineRoutes.js

```
const express = require('express');
const router = express.Router();

const { getMedicines, addMedicine } = require('../controllers/medicineController');

router.get('/', getMedicines);
router.post('/', addMedicine);

module.exports = router;
```

C.1.5.7 patientRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getDoctors,
  bookAppointment,
  getAppointments,
  getMedicines,
  getCart,
  addToCart,
  removeFromCart,
  checkoutCart,
  getLabTests,
  bookLabTest,
  getLabBookings,
  getBloodRequests,
  createBloodRequest
} = require('../controllers/patientController');
const { protect } = require('../middleware/authMiddleware');

router.use(protect);

router.get('/doctors', getDoctors);
router.post('/appointments', bookAppointment);
router.get('/appointments', getAppointments);

router.get('/medicines', getMedicines);
router.get('/cart', getCart);
router.post('/cart', addToCart);
router.delete('/cart/:medicineId', removeFromCart);
router.post('/cart/checkout', checkoutCart);

router.get('/labs', getLabTests);
router.post('/labs', bookLabTest);
router.get('/lab-bookings', getLabBookings);
```

```
router.get('/blood-requests', getBloodRequests);
router.post('/blood-requests', createBloodRequest);

module.exports = router;
```

C.1.6 .env (Format)

```
MONGO_URI=mongodb+srv://<username>:<password>@baymax.cwbubxv.mong
odb.net/baymax?appName=baymax
NODE_ENV=development
PORT=5000
OPENROUTER_API_KEY=sk-or-v1-API_KEY
```

C.1.7 server.js (Core Back – End)

```
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const path = require('path');
const http = require('http');
const { Server } = require('socket.io');

// Load environment variables
dotenv.config({ path: path.join(__dirname, '.env') });

// DB connection
const connectDB = require('./config/db');

// Route imports
const authRoutes = require('./routes/authRoutes');
const aiRoutes = require('./routes/aiRoutes');
const adminRoutes = require('./routes/adminRoutes');
const doctorRoutes = require('./routes/doctorRoutes');
const patientRoutes = require('./routes/patientRoutes');
const labRoutes = require('./routes/labtestsRoutes');
const medicineRoutes = require('./routes/medicineRoutes');
```

```
// Middleware imports
const { errorHandler, notFound } = require('./middleware/errorMiddleware');

// Initialize app
const app = express();

// Connect DB
connectDB();

// Middleware
app.use(cors({
  origin: '*',
}));
app.use(express.json());

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/doctor', doctorRoutes);
app.use('/api/patients', patientRoutes);
app.use('/api/labs', labRoutes);
app.use('/api/medicines', medicineRoutes);
app.use('/api/ai', aiRoutes);

// Test route
app.get('/api/test', (req, res) => {
  res.json({ message: ' API is working perfectly' });
});

// Error handling
app.use(notFound);
app.use(errorHandler);

// Create HTTP server
```

```
const server = http.createServer(app);

// Setup Socket.IO
const io = new Server(server, {
  cors: {
    origin: '*',
    methods: ['GET', 'POST'],
  },
});

//  Store users (VERY IMPORTANT for video call)
const users = {};

// Socket connection
io.on('connection', (socket) => {
  console.log(' User connected:', socket.id);

  // Register user
  socket.on('register', (userId) => {
    users[userId] = socket.id;
    console.log(`User ${userId} registered with socket ${socket.id}`);
  });

  // Call user
  socket.on('call-user', ({ to, offer, from }) => {
    const targetSocket = users[to];
    if (targetSocket) {
      io.to(targetSocket).emit('call-made', {
        offer,
        from,
      });
    }
  });

  // Answer call
```

```
  socket.on('make-answer', ({ to, answer }) => {
    const targetSocket = users[to];
    if (targetSocket) {
      io.to(targetSocket).emit('answer-made', {
        answer,
      });
    }
  });

  // ICE candidate
  socket.on('ice-candidate', ({ to, candidate }) => {
    const targetSocket = users[to];
    if (targetSocket) {
      io.to(targetSocket).emit('ice-candidate', {
        candidate,
      });
    }
  });

  // Disconnect
  socket.on('disconnect', () => {
    console.log(' User disconnected:', socket.id);

    // Remove user from list
    for (let userId in users) {
      if (users[userId] === socket.id) {
        delete users[userId];
        break;
      }
    }
  });
});

// Start server
const PORT = process.env.PORT || 5000;
```

```
server.listen(PORT, () => {
  console.log(` Server running on port ${PORT}`);
});
```

C2. Front - End

C2.1 src

C2.1.1 components

C2.1.1.1 ChatBot.tsx

```
import React, { useState, useEffect, useRef } from "react";
import { MessageCircle, X, Send, Loader2 } from "lucide-react";
import axios from "axios";

export default function ChatBot() {
  const [isOpen, setIsOpen] = useState(false);
  const [input, setInput] = useState("");
  const [isTyping, setIsTyping] = useState(false);
  const messagesEndRef = useRef<HTMLDivElement>(null);

  const [messages, setMessages] = useState([
    {
      id: 1,
      sender: "bot",
      text: "Hello! I am BAYMAX, your personal healthcare companion. How can I
help you today?",
      time: new Date().toLocaleTimeString([], {
        hour: "2-digit",
        minute: "2-digit",
      }),
    },
  ]);

  // ✅ Smooth auto scroll (FIXED)
  useEffect(() => {
```

```
  messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
}, [messages, isTyping]);

const handleSend = async () => {
  if (!input.trim()) return;

  const userText = input.trim();

  setMessages((prev) => [
    ...prev,
    {
      id: Date.now(),
      sender: "user",
      text: userText,
      time: new Date().toLocaleTimeString([], {
        hour: "2-digit",
        minute: "2-digit",
      }),
    },
  ]);

  setInput("");
  setIsTyping(true);

  try {
    const res = await axios.post("http://localhost:5000/api/ai/chat", {
      message: userText,
    });

    const aiReply =
      res.data?.content ||
      res.data?.message?.content ||
      "No response from AI";

    setMessages((prev) => [
```

```
      ...prev,
      {
        id: Date.now() + 1,
        sender: "bot",
        text: aiReply,
        time: new Date().toLocaleTimeString([], {
          hour: "2-digit",
          minute: "2-digit",
        }),
      },
    ]);
  } catch (error: any) {
    console.error("Frontend Error:", error);

    setMessages((prev) => [
      ...prev,
      {
        id: Date.now(),
        sender: "bot",
        text:
          error?.response?.data?.error ||
          " Failed to connect AI. Please try again.",
        time: new Date().toLocaleTimeString([], {
          hour: "2-digit",
          minute: "2-digit",
        }),
      },
    ]);
  } finally {
    setIsTyping(false);
  }
};

return (
  <>
```

```
      {/* Floating Button */}
      <button
        onClick={() => setIsOpen(true)}
        className={`fixed bottom-8 right-8 w-16 h-16 bg-[#00A99D] text-white
rounded-full flex items-center justify-center shadow-lg transition ${
          isOpen ? "scale-0" : "scale-100"
        }`}
      >
        <MessageCircle size={26} />
      </button>

      {/* Chat Window */}
      {isOpen && (
        <div className="fixed bottom-8 right-8 w-[380px] h-[500px] bg-white
rounded-2xl shadow-2xl flex flex-col overflow-hidden">
          {/* Header */}
          <div className="bg-[#00A99D] text-white p-4 flex justify-between items-
center">
            <div>
              <h2 className="font-semibold">BAYMAX</h2>
              <p className="text-xs opacity-80">AI Healthcare Assistant </p>
            </div>
            <button
              onClick={() => setIsOpen(false)}
              className="p-2 rounded-full hover:bg-white/20 transition z-50"
            >
              <X size={20} />
            </button>
          </div>

          {/* Messages (SCROLL FIXED) */}
          <div ref="flex-1 overflow-y-auto min-h-0 p-4 bg-gray-50">
            {messages.map((msg) => (
              <div
                key={msg.id}
```

```
            className={`flex mb-3 ${
              msg.sender === "user" ? "justify-end" : "justify-start"
            }`}
          >
            <div
              className={`max-w-[80%] break-words px-4 py-2 rounded-2xl text-sm
shadow ${
                msg.sender === "user"
                  ? "bg-[#00A99D] text-white rounded-br-none"
                  : "bg-white text-gray-800 border rounded-bl-none"
              }`}
            >
              {/*  TEXT FORMATTING FIX */}
              {msg.text.split("\n").map((line, index) => (
                <p key={index} className="mb-1 leading-relaxed">
                  {line}
                </p>
              ))}

              <div className="text-[10px] mt-1 opacity-70 text-right">
                {msg.time}
              </div>
            </div>
          </div>
        ))}

        {/* Typing Indicator */}
        {isTyping && (
          <div className="flex justify-start mb-2">
            <div className="bg-white px-4 py-2 rounded-xl shadow text-sm">
              <Loader2 className="animate-spin inline mr-2" size={14} />
              Thinking...
            </div>
          </div>
        )}
```

```
          <div ref={messagesEndRef} />
        </div>

        {/* Input */}
        <div className="p-3 border-t flex items-center gap-2 bg-white">
          <input
            value={input}
            onChange={(e) => setInput(e.target.value)}
            onKeyDown={(e) => e.key === "Enter" && handleSend()}
            placeholder="Describe your symptoms..."
            className="flex-1 border rounded-full px-4 py-2 focus:outline-none
focus:ring-2 focus:ring-[#00A99D]"
          />
          <button
            onClick={handleSend}
            className="bg-[#00A99D] text-white p-2 rounded-full flex items-center
justify-center shadow"
          >
            {isTyping ? (
              <Loader2 className="animate-spin" size={18} />
            ) : (
              <Send size={18} />
            )}
          </button>
        </div>
        {/*  Warning Message */}
        <div className="px-3 pb-2 text-xs text-red-500 flex items-center gap-1">
          AI is not 100% accurate. Please consult a doctor.
        </div>
      </div>
    )}
  </>
 );
}
```

C2.1.1.2 NotificationBell.tsx

```
import React, { useState, useRef, useEffect } from "react";
import { Bell, CheckCircle, Clock, Info } from "lucide-react";

export default function NotificationBell() {
  const [isOpen, setIsOpen] = useState(false);
  const [notifications, setNotifications] = useState([
    {
      id: 1,
      title: "Welcome to BAYMAX!",
      message:
        "Your account has been successfully created. Explore our healthcare
services.",
      time: "Just now",
      read: false,
      type: "info",
    },
    {
      id: 2,
      title: "Appointment Reminder",
      message: "You have an upcoming appointment scheduled for tomorrow.",
      time: "2 hours ago",
      read: false,
      type: "reminder",
    },
    {
      id: 3,
      title: "System Update",
      message: "New features have been added to the Blood Bank module.",
      time: "1 day ago",
      read: true,
      type: "system",
    },
  ]);
```

```
  const dropdownRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (
        dropdownRef.current &&
        !dropdownRef.current.contains(event.target as Node)
      ) {
        setIsOpen(false);
      }
    }
    document.addEventListener("mousedown", handleClickOutside);
    return () => document.removeEventListener("mousedown",
handleClickOutside);
  }, []);

  const unreadCount = notifications.filter((n) => !n.read).length;

  const markAllAsRead = () => {
    setNotifications(notifications.map((n) => ({ ...n, read: true })));
  };

  const markAsRead = (id: number) => {
    setNotifications(
      notifications.map((n) => (n.id === id ? { ...n, read: true } : n)),
    );
  };

  return (
    <div className="relative" ref={dropdownRef}>
      <button
        onClick={() => setIsOpen(!isOpen)}
        className="text-slate-400 hover:text-slate-600 transition-colors relative p-1"
      >
        <Bell className="w-[22px] h-[22px]" />
```

```
        {unreadCount > 0 && (
          <span className="absolute -top-1 -right-1 bg-[#F43F5E] text-white text-[10px] font-bold px-[5px] py-[1px] rounded-full border-2 border-white">
            {unreadCount}
          </span>
        )}
      </button>

      {isOpen && (
        <div className="absolute right-0 mt-3 w-80 bg-white rounded-2xl shadow-xl border border-slate-100 z-50 overflow-hidden transform origin-top-right transition-all">
          <div className="p-4 border-b border-slate-100 flex items-center justify-between bg-slate-50">
            <h3 className="font-bold text-slate-800">Notifications</h3>
            {unreadCount > 0 && (
              <button
                onClick={markAllAsRead}
                className="text-xs text-[#3B82F6] font-medium hover:underline"
              >
                Mark all read
              </button>
            )}
          </div>

          <div className="max-h-[350px] overflow-y-auto">
            {notifications.length > 0 ? (
              notifications.map((notif) => (
                <div
                  key={notif.id}
                  onClick={() => markAsRead(notif.id)}
                  className={`p-4 border-b border-slate-50 hover:bg-slate-50 cursor-pointer transition-colors ${!notif.read ? "bg-blue-50/30" : ""}`}
                >
                  <div className="flex gap-3">
```

```
                <div className="mt-0.5">
                 {notif.type === "info" && (
                  <Info className="w-5 h-5 text-blue-500" />
                 )}
                 {notif.type === "reminder" && (
                  <Clock className="w-5 h-5 text-amber-500" />
                 )}
                 {notif.type === "system" && (
                  <CheckCircle className="w-5 h-5 text-emerald-500" />
                 )}
                </div>
                <div className="flex-1 space-y-1">
                 <div className="flex items-center justify-between">
                  <p
                   className={`text-sm font-semibold ${!notif.read ? "text-slate-800"
: "text-slate-600"}`}
                  >
                   {notif.title}
                  </p>
                  {!notif.read && (
                   <div className="w-2 h-2 bg-blue-500 rounded-full"></div>
                  )}
                 </div>
                 <p className="text-xs text-slate-500 leading-relaxed">
                  {notif.message}
                 </p>
                 <p className="text-[10px] text-slate-400 font-medium">
                  {notif.time}
                 </p>
                </div>
               </div>
              </div>
             ))
            ) : (
             <div className="p-8 text-center text-slate-500">
```

```
            <Bell className="w-8 h-8 mx-auto mb-2 text-slate-300 opacity-50" />
            <p className="text-sm">No notifications yet</p>
          </div>
        )}
      </div>

      {notifications.length > 0 && (
        <div className="p-3 border-t border-slate-100 text-center bg-slate-50">
          <button className="text-xs font-semibold text-slate-500 hover:text-slate-
800 transition-colors">
            View all notifications
          </button>
        </div>
      )}
    </div>
  );
}
```

C2.1.2 context

C2.1.2.1 AuthContext.tsx

```
import {
  createContext,
  useContext,
  useState,
  useEffect,
  ReactNode,
} from "react";
import api from "../services/api";

interface User {
  _id: string;
  name: string;
  email: string;
```

```
  role: "patient" | "doctor" | "admin";
  isApproved: boolean;
  token: string;
}

interface AuthContextType {
  user: User | null;
  login: (data: User) => void;
  logout: () => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider = ({ children }: { children: ReactNode }) => {
  const [user, setUser] = useState<User | null>(() => {
    const saved = localStorage.getItem("user");
    return saved ? JSON.parse(saved) : null;
  });

  useEffect(() => {
    if (user) {
      localStorage.setItem("user", JSON.stringify(user));
      api.defaults.headers.common["Authorization"] = `Bearer ${user.token}`;
    } else {
      localStorage.removeItem("user");
      delete api.defaults.headers.common["Authorization"];
    }
  }, [user]);

  const login = (userData: User) => setUser(userData);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
```

```
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be used within an AuthProvider");
  return context;
};
```

C2.1.3 pages

C2.1.3.1 AdminDashboard.tsx

```
import { useEffect, useState } from "react";
import api from "../services/api";
import { useAuth } from "../context/AuthContext";
import { LogOut, Activity, UserCheck, XCircle } from "lucide-react";
import { useNavigate } from "react-router-dom";

export default function AdminDashboard() {
  const [doctors, setDoctors] = useState([]);
  const { logout } = useAuth();
  const navigate = useNavigate();

  const fetchDoctors = async () => {
    try {
      const { data } = await api.get("/admin/pending-doctors");
      setDoctors(data);
    } catch (err) {
      console.error(err);
    }
  };

  useEffect(() => {
    fetchDoctors();
  }, []);
```

```
  const handleApprove = async (id: string) => {
    if (!window.confirm("Approve this doctor?")) return;
    try {
      await api.put(`/admin/approve-doctor/${id}`);
      fetchDoctors();
    } catch (err) {
      console.error(err);
    }
  };

  const handleReject = async (id: string) => {
    if (!window.confirm("Reject and delete this doctor?")) return;
    try {
      await api.delete(`/admin/reject-doctor/${id}`);
      fetchDoctors();
    } catch (err) {
      console.error(err);
    }
  };

  return (
    <div className="min-h-screen bg-slate-50">
      <nav className="bg-white border-b border-slate-200 sticky top-0 z-50">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 h-16 flex items-
center justify-between">
          <div className="flex items-center gap-2">
            <div className="bg-[#06B6D4] p-1.5 rounded-lg">
              <Activity className="w-5 h-5 text-white" />
            </div>
            <span className="text-xl font-bold text-slate-800 tracking-tight">
              BAYMAX Admin
            </span>
          </div>
          <button
```

```
          onClick={() => {
            logout();
            navigate("/login");
          }}
          className="flex items-center gap-2 text-slate-600 hover:text-red-500
transition-colors font-medium text-sm"
        >
          <LogOut className="w-4 h-4" />
          Sign Out
        </button>
      </div>
    </nav>

    <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
      <h1 className="text-2xl font-bold text-slate-800 mb-6">
        Pending Doctor Approvals
      </h1>

      {doctors.length === 0 ? (
        <div className="bg-white rounded-xl shadow-sm border border-slate-200
p-8 text-center text-slate-500">
          No pending doctors awaiting approval.
        </div>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {doctors.map((doc: any) => (
            <div
              key={doc._id}
              className="bg-white rounded-xl shadow-sm border border-slate-200 p-
6 flex flex-col"
            >
              <div className="mb-4">
                <h3 className="text-lg font-bold text-slate-800">
                  {doc.name}
                </h3>
```

```
  <p className="text-sm text-slate-500">{doc.email}</p>
</div>

{doc.profile && (
  <div className="space-y-2 mb-6 flex-1 text-sm">
    <p>
      <span className="font-semibold text-slate-700">
        Specialization:
      </span>{" "}
      {doc.profile.specialization}
    </p>
    <p>
      <span className="font-semibold text-slate-700">
        Experience:
      </span>{" "}
      {doc.profile.experience} years
    </p>
    <p>
      <span className="font-semibold text-slate-700">
        Hospital:
      </span>{" "}
      {doc.profile.hospital}
    </p>
    <p>
      <span className="font-semibold text-slate-700">
        Location:
      </span>{" "}
      {doc.profile.address}
    </p>
  </div>
)}

<div className="flex gap-3 mt-auto pt-4 border-t border-slate-100">
  <button
    onClick={() => handleApprove(doc._id)}
```

```
                  className="flex-1 flex items-center justify-center gap-2 bg-green-500
text-white py-2 rounded-lg font-medium hover:bg-green-600 transition-colors text-
sm"
                >
                  <UserCheck className="w-4 h-4" />
                  Approve
                </button>
                <button
                  onClick={() => handleReject(doc._id)}
                  className="flex-1 flex items-center justify-center gap-2 bg-red-500
text-white py-2 rounded-lg font-medium hover:bg-red-600 transition-colors text-sm"
                >
                  <XCircle className="w-4 h-4" />
                  Reject
                </button>
              </div>
            </div>
          ))}
        </div>
      )}
    </main>
  </div>
 );
}
```

C2.1.3.2 BloodBank.tsx

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import api from "../services/api";
import { ArrowLeft, Droplet, Clock, Search, MapPin } from "lucide-react";

export default function BloodBank() {
  const [requests, setRequests] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);
  const [bloodGroup, setBloodGroup] = useState("");
```

```
const navigate = useNavigate();

useEffect(() => {
  fetchRequests();
}, []);

const fetchRequests = async () => {
  try {
    const res = await api.get("/patients/blood-requests");
    setRequests(res.data);
  } catch (error) {
    console.error(error);
  } finally {
    setLoading(false);
  }
};

const handleRequest = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!bloodGroup) return;

  try {
    await api.post("/patients/blood-requests", { bloodGroup });
    setBloodGroup("");
    alert("Blood Request Submitted!");
    fetchRequests();
  } catch (error) {
    console.error(error);
    alert("Failed to submit request");
  }
};

const bloodGroups = ["A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"];

return (
```

```
    <div className="min-h-screen bg-[#F8FAFC] pb-20 p-4 md:p-8">
      <div className="max-w-4xl mx-auto space-y-8">
        <button
          onClick={() => navigate("/")}
          className="flex items-center gap-2 text-slate-500 hover:text-slate-800
transition-colors bg-white px-4 py-2 rounded-lg shadow-sm w-fit"
        >
          <ArrowLeft className="w-4 h-4" /> Back to Dashboard
        </button>

        <div className="bg-[#EF4444] rounded-[24px] p-8 text-white shadow-lg
relative overflow-hidden">
          <div className="absolute top-0 right-0 w-64 h-64 bg-white/10 rounded-full -
translate-y-1/2 translate-x-1/3 blur-2xl pointer-events-none"></div>

          <div className="relative z-10 flex flex-col md:flex-row justify-between
items-center gap-6">
            <div className="flex items-center gap-6">
              <div className="w-20 h-20 bg-white/20 rounded-full flex items-center
justify-center border border-white/30 backdrop-blur-sm">
                <Droplet className="w-10 h-10 fill-white text-white" />
              </div>
              <div>
                <h1 className="text-3xl font-bold">Blood Bank Services</h1>
                <p className="text-red-100 mt-2 text-lg">
                  Request blood or find nearby donors in Maduravoyal
                </p>
              </div>
            </div>

            <a
              href="https://www.google.com/maps/search/blood+bank+near+maduravo
yal+chennai"
              target="_blank"
              rel="noreferrer"
```

```
            className="bg-white text-[#EF4444] px-6 py-3 rounded-xl font-bold
hover:bg-slate-50 transition-colors flex items-center gap-2 shadow-sm whitespace-
nowrap"
          >
            <Search className="w-5 h-5" /> Find Banks Nearby
          </a>
        </div>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
        <div className="md:col-span-1 space-y-6">
          <div className="bg-white p-6 rounded-2xl shadow-sm border border-
slate-200">
            <h2 className="text-xl font-bold text-slate-800 mb-4">
              Request Blood
            </h2>
            <form onSubmit={handleRequest} className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-slate-700 mb-2">
                  Blood Group Needed
                </label>
                <select
                  value={bloodGroup}
                  onChange={(e) => setBloodGroup(e.target.value)}
                  required
                  className="w-full px-4 py-3 bg-slate-50 border border-slate-200
rounded-xl focus:outline-none focus:border-red-500 focus:ring-1 focus:ring-red-
500"
                >
                  <option value="">Select Group</option>
                  {bloodGroups.map((bg) => (
                    <option key={bg} value={bg}>
                      {bg}
                    </option>
                  ))}
```

```
            </select>
          </div>
          <div className="pt-2">
            <button
              type="submit"
              className="w-full bg-[#EF4444] text-white font-bold py-3.5 px-4
rounded-xl hover:bg-red-600 transition-colors shadow-sm flex items-center justify-
center gap-2"
            >
              <Droplet className="w-5 h-5 fill-current" /> Submit Request
            </button>
          </div>
        </form>
      </div>

      <div className="bg-orange-50 p-6 rounded-2xl border border-orange-
100">
        <h3 className="font-bold text-orange-800 flex items-center gap-2 mb-
2">
          <Clock className="w-5 h-5" /> Emergency Contacts
        </h3>
        <p className="text-sm text-orange-700 mb-3">
          For immediate medical assistance and blood requirements,
          contact:
        </p>
        <div className="space-y-2 font-medium text-orange-900">
          <div className="flex items-center gap-2 bg-white/50 p-2 rounded-lg">
            <MapPin className="w-4 h-4 text-orange-500" /> ACS Hospital:
            044-2653-3222
          </div>
          <div className="flex items-center gap-2 bg-white/50 p-2 rounded-lg">
            <Droplet className="w-4 h-4 text-red-500" /> Red Cross: 104
          </div>
        </div>
      </div>
```

```
        </div>

        <div className="md:col-span-2">
          <h2 className="text-xl font-bold text-slate-800 mb-4">
            Your Recent Requests
          </h2>

          {loading ? (
            <div className="flex justify-center p-8">
              <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-
red-500"></div>
            </div>
          ) : requests.length > 0 ? (
            <div className="space-y-4">
              {requests.map((req, idx) => (
                <div
                  key={idx}
                  className="bg-white p-5 rounded-xl border border-slate-200
shadow-sm flex items-center justify-between hover:border-red-200 transition-
colors group"
                >
                  <div className="flex items-center gap-4">
                    <div
                      className={`w-14 h-14 rounded-full flex items-center justify-center
text-xl font-bold border-4 shadow-sm
                        ${
                          req.status === "pending"
                            ? "bg-amber-50 text-amber-500 border-amber-100"
                            : req.status === "fulfilled"
                              ? "bg-emerald-50 text-emerald-500 border-emerald-100"
                              : "bg-red-50 text-red-500 border-red-100"
                        }
                      `}
                      >
                        {req.bloodGroup}
```

```
                </div>
                <div>
                  <p className="font-semibold text-slate-800">
                    Blood Request ({req.bloodGroup})
                  </p>
                  <p className="text-xs text-slate-500 flex items-center gap-1 mt-
1">
                    <Clock className="w-3.5 h-3.5" />
                    {new Date(req.createdAt).toLocaleDateString()} at{" "}
                    {new Date(req.createdAt).toLocaleTimeString([], {
                      hour: "2-digit",
                      minute: "2-digit",
                    })}
                  </p>
                </div>
              </div>

              <span
                className={`px-4 py-1.5 rounded-full text-xs font-bold uppercase
tracking-wider
                ${
                  req.status === "pending"
                    ? "bg-amber-100 text-amber-700"
                    : req.status === "fulfilled"
                      ? "bg-emerald-100 text-emerald-700"
                      : "bg-red-100 text-red-700"
                }
              `}
              >
                {req.status}
              </span>
            </div>
          ))}
        </div>
      ) : (
```

```
            <div className="bg-white border border-slate-200 border-dashed
rounded-2xl p-12 text-center text-slate-500">
              <Droplet className="w-12 h-12 text-slate-200 mx-auto mb-3" />
              <p className="font-medium">No blood requests made yet.</p>
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
```

C2.1.3.3 BuyMedicines.tsx

```
import React, { useEffect, useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import api from "../services/api";
import { ArrowLeft, Search, Pill, Plus } from "lucide-react";

export default function BuyMedicines() {
  const [medicines, setMedicines] = useState<any[]>([]);
  const [filtered, setFiltered] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState("");
  const navigate = useNavigate();

  useEffect(() => {
    fetchMedicines();
  }, []);

  useEffect(() => {
    if (search.trim() === "") {
      setFiltered(medicines);
    } else {
```

```
    const filteredData = medicines.filter((med) =>
      med.name.toLowerCase().includes(search.toLowerCase()),
    );
    setFiltered(filteredData);
  }
 }, [search, medicines]);

 const fetchMedicines = async () => {
  try {
    const res = await axios.get("http://localhost:5000/api/medicines");
    console.log("DIRECT FETCH DATA:", res.data);
    setMedicines(res.data);
    setFiltered(res.data);
  } catch (error) {
    console.error("DIRECT FETCH ERROR:", error);
  } finally {
    setLoading(false);
  }
 };
 const handleAddToCart = (medicineId: string) => {
  alert("Cart system not implemented yet.");
 };

 return (
  <div className="min-h-screen bg-[#F8FAFC] pb-20 p-4 md:p-8">
    <div className="max-w-6xl mx-auto space-y-8">
      {/* Header */}
      <div className="flex justify-between items-center">
        <button
          onClick={() => navigate("/")}
          className="flex items-center gap-2 text-slate-500 hover:text-slate-800
transition-colors bg-white px-4 py-2 rounded-lg shadow-sm"
        >
          <ArrowLeft className="w-4 h-4" /> Back to Dashboard
        </button>
```

```
      </div>

      {/* Title + Search */}
      <div className="flex flex-col md:flex-row justify-between items-start
md:items-center gap-4">
        <div>
          <h1 className="text-3xl font-bold text-slate-800 flex items-center gap-2">
            <Pill className="w-8 h-8 text-[#10B981]" /> Buy Medicines
          </h1>
          <p className="text-slate-500 mt-1">
            Order medicines easily from trusted sources
          </p>
        </div>

        <div className="relative w-full md:w-auto min-w-[300px]">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400
w-5 h-5" />
          <input
            type="text"
            placeholder="Search medicines..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-10 pr-4 py-3 bg-white border border-slate-200
rounded-xl focus:outline-none focus:border-[#10B981] shadow-sm"
          />
        </div>
      </div>

      {/* Medicines Grid */}
      {loading ? (
        <div className="flex justify-center p-12">
          <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-
[#10B981]"></div>
        </div>
      ) : (
```

```
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">
          {filtered.map((med) => (
            <div
              key={med._id}
              className="bg-white p-6 rounded-2xl shadow-sm border border-slate-
200 hover:shadow-md transition-shadow flex flex-col"
            >
              <div className="flex-grow">
                <div className="w-12 h-12 bg-emerald-50 rounded-xl flex items-
center justify-center mb-4">
                  <Pill className="w-6 h-6 text-[#10B981]" />
                </div>

                <h3 className="text-lg font-bold text-slate-800">
                  {med.name}
                </h3>

                {med.brand && (
                  <p className="text-sm text-slate-500">{med.brand}</p>
                )}

                <p className="text-sm text-slate-500">{med.type}</p>

                {med.requiresPrescription && (
                  <p className="text-xs text-red-500 mt-1">
                    Prescription Required
                  </p>
                )}
              </div>

              <div className="flex items-center justify-between mt-4 pt-4 border-t
border-slate-100">
                <span className="text-xl font-bold text-slate-800">
                  ₹{med.price}
                </span>
```

```
                <button
                  onClick={() => handleAddToCart(med._id)}
                  className="flex items-center gap-1 bg-[#10B981] text-white px-3 py-
1.5 rounded-lg hover:bg-emerald-600 transition-colors text-sm font-semibold"
                >
                  <Plus className="w-4 h-4" /> Add
                </button>
              </div>
            </div>
          ))}

          {filtered.length === 0 && (
            <div className="col-span-full text-center py-12 text-slate-500">
              No medicines found.
            </div>
          )}
        </div>
      )}
    </div>
  </div>
 );
}
```

C2.1.3.4 CartView.tsx

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import api from "../services/api";
import {
  ArrowLeft,
  ShoppingCart,
  Trash2,
  ShieldCheck,
  Pill,
} from "lucide-react";
```

```
export default function CartView() {
  const [cart, setCart] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  useEffect(() => {
    fetchCart();
  }, []);

  const fetchCart = async () => {
    try {
      const res = await api.get("/patients/cart");
      setCart(res.data);
    } catch (error) {
      console.error(error);
    } finally {
      setLoading(false);
    }
  };

  const handleRemove = async (medicineId: string) => {
    try {
      const res = await api.delete(`/patients/cart/${medicineId}`);
      setCart(res.data);
    } catch (error) {
      console.error(error);
      alert("Failed to remove item");
    }
  };

  const handleCheckout = async () => {
    try {
      await api.post("/patients/cart/checkout");
      alert("Order Placed Successfully!");
      setCart({ ...cart, items: [] });
```

```
      navigate("/medicines");
    } catch (error) {
      console.error(error);
      alert("Checkout Failed");
    }
  };

  const calculateTotal = () => {
    if (!cart || !cart.items) return 0;
    return cart.items.reduce((total: number, item: any) => {
      return total + item.medicine.price * item.quantity;
    }, 0);
  };

  return (
    <div className="min-h-screen bg-[#F8FAFC] pb-20 p-4 md:p-8">
      <div className="max-w-4xl mx-auto space-y-8">
        <button
          onClick={() => navigate("/medicines")}
          className="flex items-center gap-2 text-slate-500 hover:text-slate-800
transition-colors bg-white px-4 py-2 rounded-lg shadow-sm w-fit"
        >
          <ArrowLeft className="w-4 h-4" /> Back to Medicines
        </button>

        <div>
          <h1 className="text-3xl font-bold text-slate-800 flex items-center gap-2">
            <ShoppingCart className="w-8 h-8 text-[#00A99D]" /> Your Cart
          </h1>
          <p className="text-slate-500 mt-1">
            Review your medicines before checkout
          </p>
        </div>

        {loading ? (
```

```
        <div className="flex justify-center p-12">
          <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-[#00A99D]"></div>
        </div>
      ) : (
        <div className="bg-white rounded-2xl shadow-sm border border-slate-200 overflow-hidden">
          {cart?.items?.length > 0 ? (
            <div className="divide-y divide-slate-100">
              {cart.items.map((item: any, idx: number) => (
                <div
                  key={idx}
                  className="p-6 flex flex-col md:flex-row items-center justify-between gap-4"
                >
                  <div className="flex items-center gap-4 w-full md:w-auto">
                    <div className="w-16 h-16 bg-slate-50 rounded-xl flex items-center justify-center shrink-0 border border-slate-100">
                      <Pill className="w-8 h-8 text-slate-400" />
                    </div>
                    <div>
                      <h3 className="text-lg font-bold text-slate-800">
                        {item.medicine.name}
                      </h3>
                      <p className="text-sm text-slate-500">
                        Qty: {item.quantity}
                      </p>
                    </div>
                  </div>

                  <div className="flex items-center justify-between w-full md:w-auto gap-8">
                    <span className="text-xl font-bold text-slate-800">
                      ₹{item.medicine.price * item.quantity}
                    </span>
```

```
                <button
                  onClick={() => handleRemove(item.medicine._id)}
                  className="text-red-400 hover:text-red-600 bg-red-50 hover:bg-
red-100 p-2.5 rounded-lg transition-colors"
                >
                  <Trash2 className="w-5 h-5" />
                </button>
              </div>
            </div>
          ))}

          <div className="p-6 bg-slate-50 flex flex-col items-end">
            <div className="text-right space-y-2 mb-6">
              <div className="text-slate-500 flex justify-between gap-12">
                <span>Subtotal</span>
                <span className="font-semibold">₹{calculateTotal()}</span>
              </div>
              <div className="text-slate-500 flex justify-between gap-12">
                <span>Delivery</span>
                <span className="font-semibold text-emerald-500">
                  Free
                </span>
              </div>
              <div className="text-2xl font-bold text-slate-800 flex justify-between
gap-12 pt-4 border-t border-slate-200">
                <span>Total</span>
                <span>₹{calculateTotal()}</span>
              </div>
            </div>

            <button
              onClick={handleCheckout}
              className="flex items-center gap-2 bg-[#00A99D] text-white px-8 py-
4 rounded-xl hover:bg-teal-600 transition-colors shadow-lg font-bold text-lg w-full
md:w-auto"
```

```
              >
                <ShieldCheck className="w-6 h-6" /> Place Order
              </button>
            </div>
          </div>
        ) : (
          <div className="text-center py-20 px-4">
            <ShoppingCart className="w-16 h-16 text-slate-200 mx-auto mb-4" />
            <h3 className="text-xl font-bold text-slate-800 mb-2">
              Your cart is empty
            </h3>
            <p className="text-slate-500 mb-6">
              Looks like you haven't added any medicines yet.
            </p>
            <button
              onClick={() => navigate("/medicines")}
              className="bg-[#00A99D] text-white px-6 py-3 rounded-xl font-
semibold hover:bg-teal-600 transition-colors inline-flex items-center gap-2"
            >
              <Pill className="w-5 h-5" /> Browse Medicines
            </button>
          </div>
        )}
      </div>
    )}
  </div>
 );
}
```

C2.1.3.5 Dashboard.tsx

```
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import api from "../services/api";
```

```
import NotificationBell from "../components/NotificationBell";
import ChatBot from "../components/ChatBot";
import {
  HeartPulse,
  Home,
  Calendar,
  Pill,
  FlaskConical,
  FileText,
  Shield,
  Droplet,
  Bell,
  User,
  X,
  Search,
  AlertTriangle,
  Phone,
  ChevronRight,
  LogOut,
  Video,
  Clock,
  CheckCircle2,
} from "lucide-react";

export default function Dashboard() {
  const navigate = useNavigate();
  const { user, logout } = useAuth();
  const [showBanner, setShowBanner] = useState(true);
  const [appointments, setAppointments] = useState<any[]>([]);
  const [labBookings, setLabBookings] = useState<any[]>([]);
  const [stats, setStats] = useState({
    upcoming: 0,
    total: 0,
    labs: 0,
    reminders: 0,
```

```
});

useEffect(() => {
  if (user) {
    api
      .get("/patients/records")
      .then((res) => {
        const appts = res.data.appointments || [];
        setAppointments(appts);
        const upcoming = appts.filter(
          (a: any) => a.status === "approved" || a.status === "pending",
        ).length;
        setStats((s) => ({ ...s, total: appts.length, upcoming }));
      })
      .catch(console.error);

    api
      .get("/patients/labs")
      .then((res) => {
        setStats((s) => ({ ...s, labs: res.data.length || 0 }));
      })
      .catch(console.error);

    api
      .get("/patients/lab-bookings")
      .then((res) => {
        const bookings = res.data || [];
        setLabBookings(bookings);
        setStats((s) => ({ ...s, reminders: bookings.length }));
      })
      .catch(console.error);
  }
}, [user]);

if (!user) return null;
```

```
  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <div className="min-h-screen bg-[#F8FAFC] font-sans text-slate-800 pb-20">
      {/* Navbar */}
      <nav className="bg-white border-b border-slate-200 sticky top-0 z-50">
        <div className="max-w-7xl mx-auto px-4 flex items-center justify-between
h-[72px]">
          {/* Logo */}
          <div className="flex items-center gap-2 text-[#00A99D] cursor-pointer">
            <div className="bg-[#00A99D] p-1.5 rounded-lg flex items-center justify-
center">
              <HeartPulse className="text-white w-6 h-6" strokeWidth={2.5} />
            </div>
            <span className="font-bold text-2xl tracking-tight">BAYMAX</span>
          </div>

          {/* Nav Links */}
          <div className="hidden lg:flex items-center gap-1.5 ml-8 mr-auto">
            <button
              onClick={() => navigate("/")}
              className="flex items-center gap-2 px-4 py-2 bg-blue-50 text-blue-600
rounded-lg text-[15px] font-medium transition-colors"
            >
              <Home className="w-[18px] h-[18px]" strokeWidth={2.5} /> Home
            </button>
            <button
              onClick={() => navigate("/appointments")}
              className="flex items-center gap-2 px-4 py-2 text-slate-500 hover:bg-
slate-50 hover:text-slate-800 rounded-lg text-[15px] font-medium transition-colors"
            >
```

```
            <Calendar className="w-[18px] h-[18px]" /> Appointments
          </button>
          <button
            onClick={() => navigate("/medicines")}
            className="flex items-center gap-2 px-4 py-2 text-slate-500 hover:bg-
slate-50 hover:text-slate-800 rounded-lg text-[15px] font-medium transition-colors"
          >
            <Pill className="w-[18px] h-[18px]" /> Medicines
          </button>
          <button
            onClick={() => navigate("/labs")}
            className="flex items-center gap-2 px-4 py-2 text-slate-500 hover:bg-
slate-50 hover:text-slate-800 rounded-lg text-[15px] font-medium transition-colors"
          >
            <FlaskConical className="w-[18px] h-[18px]" /> Lab Tests
          </button>
          <button
            onClick={() => navigate("/health-records")}
            className="flex items-center gap-2 px-4 py-2 text-slate-500 hover:bg-
slate-50 hover:text-slate-800 rounded-lg text-[15px] font-medium transition-colors"
          >
            <FileText className="w-[18px] h-[18px]" /> Records
          </button>
          <button
            onClick={() => navigate("/blood-bank")}
            className="flex items-center gap-2 px-4 py-2 text-slate-500 hover:bg-
slate-50 hover:text-slate-800 rounded-lg text-[15px] font-medium transition-colors"
          >
            <Droplet className="w-[18px] h-[18px]" /> Blood Bank
          </button>
        </div>

        {/* Right Actions */}
        <div className="flex items-center gap-5">
          <NotificationBell />
```

```
          <div className="flex items-center gap-2 border-l border-slate-200 pl-5
cursor-pointer hover:bg-slate-50 p-1.5 rounded-lg transition-colors">
            <div className="w-9 h-9 rounded-full bg-[#3B82F6] text-white flex items-
center justify-center font-bold shadow-sm">
              {user.name.charAt(0).toUpperCase()}
            </div>
            <div className="hidden md:flex flex-col">
              <div className="text-[13px] font-medium text-slate-700 leading-tight">
                @{user.name.toLowerCase().split(" ")[0]}
              </div>
              <div className="text-[#3B82F6] bg-blue-50 inline-flex items-center
gap-1 px-1.5 py-0.5 rounded text-[11px] font-medium mt-0.5 w-max capitalize">
                <User className="w-3 h-3" /> {user.role}
              </div>
            </div>
          </div>
          <button
            onClick={handleLogout}
            className="text-slate-400 hover:text-red-500 transition-colors"
            title="Logout"
          >
            <LogOut className="w-5 h-5" />
          </button>
        </div>
      </div>
    </nav>

    {/* Info Banner */}
    {showBanner && (
      <div className="bg-[#4F46E5] text-white px-4 py-2.5 flex items-center
justify-center text-[13.5px]">
        <div className="max-w-7xl mx-auto w-full flex items-center justify-
between">
          <div className="flex items-center gap-2 opacity-90 font-light">
```

```
            <User className="w-4 h-4" />
            <span>
              Logged in as{" "}
              <strong className="font-semibold capitalize">
                {user.role}
              </strong>{" "}
              · {user.name}{" "}
              <span className="bg-white/20 px-1.5 py-0.5 rounded text-xs ml-1 mr-
1">
                @{user.name.toLowerCase().split(" ")[0]}
              </span>{" "}
              · Access all healthcare features
            </span>
          </div>
          <button
            onClick={() => setShowBanner(false)}
            className="opacity-70 hover:opacity-100 transition-opacity p-1"
          >
            <X className="w-4 h-4" />
          </button>
        </div>
      </div>
    )}

    {/* Main Content */}
    <main className="max-w-[1200px] mx-auto px-4 py-8 space-y-8">
      {/* Hero Section */}
      <div className="bg-gradient-to-r from-[#2563EB] to-[#4F46E5] rounded-
[24px] p-8 text-white shadow-lg relative overflow-hidden">
        {/* Decorative shapes */}
        <div className="absolute top-0 right-0 w-[500px] h-[500px] bg-white/5
rounded-full -translate-y-1/2 translate-x-1/3 blur-3xl pointer-events-none"></div>
        <div className="absolute bottom-0 right-1/4 w-64 h-64 bg-white/5
rounded-full translate-y-1/2 blur-2xl pointer-events-none"></div>
```

```
        <div className="relative z-10 flex flex-col md:flex-row justify-between
items-start md:items-center gap-8">
          <div className="space-y-5">
           {/* Profile Tag */}
           <div className="flex items-center gap-4">
             <div className="w-[52px] h-[52px] bg-white/20 rounded-2xl flex items-
center justify-center text-2xl font-bold backdrop-blur-md shadow-inner border
border-white/20">
               {user.name.charAt(0).toUpperCase()}
             </div>
             <div className="space-y-0.5">
              <div className="flex items-center gap-2">
               <h3 className="font-bold text-lg leading-tight">
                {user.name}
               </h3>
               <span className="bg-white/20 px-2 py-0.5 rounded-md flex items-
center gap-1.5 text-xs font-medium backdrop-blur-md border border-white/10
capitalize">
                 <User className="w-3 h-3" /> {user.role}
               </span>
              </div>
              <div className="text-blue-100 text-sm opacity-90 leading-tight">
               @{user.name.toLowerCase().split(" ")[0]}
              </div>
              <div className="text-blue-100 text-[13px] opacity-75 leading-tight">
               {user.email}
              </div>
             </div>
            </div>

            {/* Greeting */}
            <div className="space-y-2">
             <h1 className="text-[32px] md:text-[40px] font-bold leading-tight flex
items-center gap-3">
               Good Morning, {user.name.split(" ")[0]}!
```

```
            </h1>
            <p className="text-blue-50/90 max-w-xl text-[15px] md:text-[17px]
leading-relaxed font-light">
              Your personal health dashboard. Track appointments, medicines,
              and health records all in one place.
            </p>
          </div>

          {/* Badges */}
          <div className="flex items-center gap-3 pt-1">
            <button
              onClick={() =>
                window.scrollTo({ top: 0, behavior: "smooth" })
              }
              className="bg-white/20 backdrop-blur-md border border-white/10 px-
3.5 py-1.5 rounded-full flex items-center gap-2 text-[13px] font-medium shadow-
sm hover:bg-white/30 transition-colors cursor-pointer"
            >
              <Bell className="w-4 h-4" /> view notifications
            </button>
            <span className="bg-white/20 backdrop-blur-md border border-
white/10 px-4 py-1.5 rounded-full text-[13px] font-medium shadow-sm">
              Active
            </span>
          </div>
        </div>

        {/* Stats Grid */}
        <div className="flex gap-4 w-full md:w-auto mt-6 md:mt-0 overflow-x-
auto pb-2 md:pb-0 hide-scrollbar">
          {[
            {
              label: "Upcoming",
              value: stats.upcoming.toString(),
              action: () => navigate("/appointments"),
```

```
            },
            {
              label: "Total Apts",
              value: stats.total.toString(),
              action: () => navigate("/health-records"),
            },
            {
              label: "Lab Tests",
              value: stats.reminders.toString(),
              action: () => {
                document
                  .getElementById("reminders-section")
                  ?.scrollIntoView({ behavior: "smooth" });
              },
            },
          ].map((stat, i) => (
            <div
              key={i}
              onClick={stat.action}
              className="bg-white/10 backdrop-blur-xl border border-white/20
rounded-[20px] p-4 flex flex-col items-center justify-center min-w-[90px] md:min-w-
[105px] h-[105px] shadow-sm hover:bg-white/15 transition-colors cursor-pointer"
            >
              <span className="text-3xl font-bold mb-1">{stat.value}</span>
              <span className="text-[12px] text-blue-50 font-medium text-center">
                {stat.label}
              </span>
            </div>
          ))}
        </div>
      </div>
    </div>

    {/* Search */}
```

```
      <div className="relative shadow-sm group bg-white rounded-2xl border
border-slate-200 hover:border-blue-400 transition-colors">
        <div className="absolute inset-y-0 left-0 pl-5 flex items-center pointer-
events-none">
          <Search className="h-5 w-5 text-slate-400 group-focus-within:text-blue-
500 transition-colors" />
        </div>
        <input
          type="text"
          className="block w-full pl-12 pr-5 py-4 bg-transparent outline-none text-
slate-800 placeholder-slate-400 text-lg transition-all"
          placeholder="Search doctors, specializations, cities..."
        />
      </div>

      {/* Quick Actions */}
      <div className="space-y-4">
        <h2 className="text-[20px] font-bold text-slate-800">
          Quick Actions
        </h2>
        <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-5">
          {[
            {
              label: "Book Appointment",
              icon: Calendar,
              color: "bg-[#3B82F6]",
              path: "/appointments",
            },
            {
              label: "Buy Medicines",
              icon: Pill,
              color: "bg-[#10B981]",
              path: "/medicines",
            },
            {
```

```
            label: "Lab Tests",
            icon: FlaskConical,
            color: "bg-[#A855F7]",
            path: "/labs",
          },
          {
            label: "Health Records",
            icon: FileText,
            color: "bg-[#F59E0B]",
            path: "/health-records",
          },
          {
            label: "Insurance",
            icon: Shield,
            color: "bg-[#00A99D]",
            path: "/insurance",
          },
          {
            label: "Blood Bank",
            icon: Droplet,
            color: "bg-[#EF4444]",
            path: "/blood-bank",
          },
        ].map((action, i) => (
          <button
            key={i}
            onClick={() => action.path !== "#" && navigate(action.path)}
            className="bg-white p-6 rounded-[24px] border border-slate-100
shadow-[0_2px_10px_-4px_rgba(0,0,0,0.05)] hover:shadow-md hover:border-
slate-200 transition-all flex flex-col items-center justify-center gap-4 group"
          >
            <div
              className={`${action.color} p-4 rounded-2xl shadow-sm group-
hover:scale-110 transition-transform duration-300`}
            >
```

```
              <action.icon className="w-7 h-7 text-white" />
            </div>
            <span className="text-[14px] font-semibold text-slate-700 text-center">
              {action.label}
            </span>
          </button>
        ))}
      </div>
    </div>

    {/* Emergency SOS */}
    <div className="bg-[#EF4444] rounded-[24px] p-6 shadow-md text-white
flex flex-col md:flex-row items-center justify-between gap-6">
      <div className="flex items-center gap-5">
        <div className="bg-white/20 p-4 rounded-full border border-white/20
backdrop-blur-sm">
          <AlertTriangle className="w-8 h-8 text-white" strokeWidth={2.5} />
        </div>
        <div className="space-y-0.5">
          <h2 className="text-[22px] font-bold">Emergency SOS</h2>
          <p className="text-red-50 text-[15px] font-medium">
            Need immediate medical assistance?
          </p>
        </div>
      </div>
      <div className="flex items-center gap-4 w-full md:w-auto">
        <a
          href="tel:108"
          className="flex-1 md:flex-none flex items-center justify-center gap-2.5
bg-white text-[#EF4444] font-bold py-3.5 px-8 rounded-xl hover:bg-slate-50
transition-colors shadow-sm text-lg"
        >
          <Phone className="w-5 h-5" /> Call 108
        </a>
        <button
```

```
            onClick={() => navigate("/blood-bank")}
            className="flex-1 md:flex-none flex items-center justify-center gap-2.5
bg-[#DC2626] text-white font-bold py-3.5 px-8 rounded-xl hover:bg-red-800
transition-colors shadow-sm border border-red-400/30 text-lg"
          >
            <Droplet className="w-5 h-5" fill="currentColor" /> Blood
          </button>
        </div>
      </div>

      {/* Lower Section */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        {/* Appointments & Reminders */}
        <div className="lg:col-span-2 space-y-8" id="reminders-section">
          {/* Appointments */}
          <div className="space-y-4">
            <div className="flex items-center justify-between">
              <h2 className="text-[18px] font-bold text-slate-800 flex items-center
gap-2.5">
                <Calendar className="w-[22px] h-[22px] text-[#3B82F6]" /> Your
                Upcoming Appointments
              </h2>
              <button
                onClick={() => navigate("/health-records")}
                className="text-[#3B82F6] text-[15px] font-medium hover:underline
flex items-center gap-1"
              >
                View All <ChevronRight className="w-4 h-4" />
              </button>
            </div>

            {appointments.filter(
              (a) => a.status === "pending" || a.status === "approved",
            ).length > 0 ? (
              <div className="space-y-3">
```

```
              {appointments
                .filter(
                  (a) => a.status === "pending" || a.status === "approved",
                )
                .slice(0, 3)
                .map((appt: any, i) => (
                  <div
                    key={i}
                    className="bg-white rounded-2xl border border-slate-200 p-5 flex
items-center justify-between shadow-sm hover:shadow-md transition-shadow"
                  >
                    <div className="flex items-center gap-4">
                      <div className="w-12 h-12 bg-blue-100 rounded-xl flex items-
center justify-center text-blue-600 font-bold text-xl shrink-0">
                        {appt.doctor?.name
                          ? appt.doctor.name.charAt(0)
                          : "D"}
                      </div>
                      <div>
                        <h4 className="font-bold text-slate-800">
                          Dr. {appt.doctor?.name || "Unknown"}
                        </h4>
                        <div className="flex items-center gap-3 text-sm text-slate-
500">
                          <span>
                            {appt.date} at {appt.time}
                          </span>
                          {appt.status === "approved" &&
                            appt.doctor?.waitingCount !== undefined && (
                              <span className="flex items-center gap-1.5 text-orange-
600 bg-orange-50 px-2 py-0.5 rounded-md font-medium text-xs">
                                <div className="w-1.5 h-1.5 bg-orange-500 rounded-full
animate-pulse"></div>
                                {appt.doctor.waitingCount} waiting ahead
                              </span>
```

```
                    )}
                  </div>
                 </div>
                </div>
                <div className="flex flex-col items-end gap-2 shrink-0">
                 <span
                  className={`px-3 py-1 text-xs font-semibold rounded-full
${appt.status === "approved" ? "bg-green-100 text-green-700" : "bg-amber-100
text-amber-700"}`}
                 >
                  {appt.status.toUpperCase()}
                 </span>
                 {appt.status === "approved" && (
                  <button
                   onClick={() =>
                    navigate(`/video-call/${appt._id}`)
                   }
                   className="flex items-center gap-1.5 text-xs bg-[#3B82F6]
hover:bg-blue-600 text-white px-3 py-1.5 rounded-lg transition-colors shadow-sm
font-medium"
                  >
                   <Video className="w-3.5 h-3.5" /> Join Call
                  </button>
                 )}
                </div>
               </div>
              ))}
             </div>
            ) : (
             <div className="bg-white rounded-[24px] border border-slate-200
shadow-[0_2px_10px_-4px_rgba(0,0,0,0.05)] p-8 flex flex-col items-center justify-
center text-slate-400">
              <Calendar
               className="w-12 h-12 mb-3 text-slate-200"
               strokeWidth={1}
```

```
                />
                <p className="text-[14px] font-medium">
                  No upcoming appointments
                </p>
                <button
                  onClick={() => navigate("/appointments")}
                  className="mt-4 px-6 py-2 bg-blue-50 text-[#3B82F6] rounded-lg
font-medium text-[13px] hover:bg-blue-100 transition-colors"
                >
                  Book Now
                </button>
              </div>
            )}
          </div>

          {/* Reminders & Lab Tests */}
          <div className="space-y-4">
            <div className="flex items-center justify-between">
              <h2 className="text-[18px] font-bold text-slate-800 flex items-center
gap-2.5">
                <Bell className="w-[22px] h-[22px] text-[#A855F7]" /> Your
                Reminders & Lab Tests
              </h2>
            </div>

            {labBookings.length > 0 ? (
              <div className="space-y-3">
                {labBookings.map((booking: any, i) => (
                  <div
                    key={i}
                    className="bg-white rounded-2xl border border-slate-200 p-5 flex
items-center justify-between shadow-sm hover:border-[#A855F7]/30 transition-
colors relative overflow-hidden"
                  >
```

```
                <div className="absolute left-0 top-0 bottom-0 w-1.5 bg-
[#A855F7]"></div>
                <div className="flex items-center gap-4">
                 <div className="w-12 h-12 bg-purple-100 rounded-xl flex items-
center justify-center text-purple-600 shrink-0">
                  <FlaskConical className="w-6 h-6" />
                 </div>
                 <div>
                  <h4 className="font-bold text-slate-800">
                   {booking.testDetails?.name || "Lab Test"}
                  </h4>
                  <div className="flex items-center gap-3 text-sm text-slate-500
mt-0.5">
                   <span className="flex items-center gap-1">
                    <Calendar className="w-3.5 h-3.5" />{" "}
                    {booking.date}
                   </span>
                   <span className="flex items-center gap-1">
                    <Clock className="w-3.5 h-3.5" /> {booking.time}
                   </span>
                   <span
                    className={`px-2 py-0.5 text-[11px] font-bold rounded-md
uppercase ${booking.type === "home" ? "bg-indigo-100 text-indigo-700" : "bg-
slate-100 text-slate-700"}`}
                   >
                    {booking.type === "home"
                     ? "Home Collect"
                     : "Walk-In"}
                   </span>
                  </div>
                 </div>
                </div>
                <div className="shrink-0">
```

```
                <span className="px-3 py-1.5 text-xs font-semibold rounded-full
bg-emerald-50 text-emerald-600 border border-emerald-100 flex items-center gap-
1.5">
                  <CheckCircle2 className="w-3.5 h-3.5" /> Confirmed
                </span>
              </div>
            </div>
          ))}
        </div>
      ) : (
        <div className="bg-white rounded-[24px] border border-slate-200
shadow-[0_2px_10px_-4px_rgba(0,0,0,0.05)] p-8 flex flex-col items-center justify-
center text-slate-400">
          <FlaskConical
            className="w-12 h-12 mb-3 text-slate-200"
            strokeWidth={1}
          />
          <p className="text-[14px] font-medium">
            No active lab bookings or reminders
          </p>
          <button
            onClick={() => navigate("/labs")}
            className="mt-4 px-6 py-2 bg-purple-50 text-[#A855F7] rounded-lg
font-medium text-[13px] hover:bg-purple-100 transition-colors"
          >
            Book Lab Test
          </button>
        </div>
      )}
    </div>
  </div>

  {/* Right Sidebar - Profile Widget */}
  <div className="space-y-4 lg:pt-11">
```

```
        <div className="bg-white rounded-[24px] border border-slate-200 shadow-[0_2px_10px_-4px_rgba(0,0,0,0.05)] p-8 flex flex-col items-center text-center">
          <div className="w-[84px] h-[84px] bg-[#3B82F6] rounded-[24px] flex items-center justify-center text-white text-[32px] font-bold shadow-lg mb-5">
            {user.name.charAt(0).toUpperCase()}
          </div>
          <h3 className="font-bold text-slate-800 text-[20px]">
            {user.name}
          </h3>
          <p className="text-[14px] text-slate-500 mt-1 font-medium capitalize">
            {user.role} Profile
          </p>

          <div className="w-full h-px bg-slate-100 my-6"></div>

          <div className="w-full space-y-3.5 text-[14.5px]">
            <div className="flex justify-between items-center bg-slate-50 px-4 py-2.5 rounded-xl">
              <span className="text-slate-500 font-medium">Email</span>
              <span
                className="font-bold text-slate-800 text-sm truncate max-w-[150px]"
                title={user.email}
              >
                {user.email}
              </span>
            </div>
            <div className="flex justify-between items-center bg-slate-50 px-4 py-2.5 rounded-xl">
              <span className="text-slate-500 font-medium">Status</span>
              <span className="font-bold text-emerald-500">Active</span>
            </div>
          </div>

          <button
```

```
              onClick={() => navigate("/health-records")}
              className="mt-6 w-full py-3 border border-slate-200 rounded-xl text-
[14.5px] font-semibold text-slate-700 hover:bg-slate-50 hover:border-slate-300
transition-colors"
            >
              View Full Profile
            </button>
          </div>
        </div>
      </div>
    </main>

    {/* Floating Action Button */}
    <ChatBot />

    {/* Added global styles inline for custom hide-scrollbar */}
    <style>{`
      .hide-scrollbar::-webkit-scrollbar {
        display: none;
      }
      .hide-scrollbar {
        -ms-overflow-style: none;
        scrollbar-width: none;
      }
    `}</style>
  </div>
 );
}
```

C2.1.3.6 DoctorAppointments.tsx

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import api from "../services/api";
import {
 ArrowLeft,
```

```
  MapPin,
  Star,
  Clock,
  Calendar,
  Search,
  Activity,
  Users,
  ShieldCheck,
  HeartPulse,
  X,
  IndianRupee,
  ThumbsUp,
} from "lucide-react";

export default function DoctorAppointments() {
  const [doctors, setDoctors] = useState<any[]>([]);
  const [mapLocation, setMapLocation] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [selectedDoctor, setSelectedDoctor] = useState<any | null>(null);
  const [date, setDate] = useState("");
  const [time, setTime] = useState("");
  const [searchQuery, setSearchQuery] = useState("");
  const navigate = useNavigate();

  useEffect(() => {
    fetchDoctors();
  }, []);

  const fetchDoctors = async () => {
    try {
      const res = await api.get("/patients/doctors");
      setDoctors(res.data);
    } catch (error) {
      console.error("Failed to fetch doctors", error);
    } finally {
```

```
    setLoading(false);
  }
};

const handleBook = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!selectedDoctor || !date || !time) return;

  try {
    await api.post("/patients/appointments", {
      doctorId: selectedDoctor._id,
      date,
      time,
    });
    alert("Appointment booked successfully!");
    setSelectedDoctor(null);
    setDate("");
    setTime("");
    navigate("/health-records");
  } catch (error) {
    console.error("Failed to book appointment", error);
    alert("Failed to book appointment");
  }
};

const filteredDoctors = doctors.filter(
  (doc) =>
    (doc.user?.name || "")
      .toLowerCase()
      .includes(searchQuery.toLowerCase()) ||
    (doc.specialization || "")
      .toLowerCase()
      .includes(searchQuery.toLowerCase()) ||
    (doc.hospital || "").toLowerCase().includes(searchQuery.toLowerCase()),
);
```

```
  return (
    <div className="min-h-screen bg-[#F8FAFC] pb-20">
      {/* Top Navigation Banner */}
      <div className="bg-[#3B82F6] text-white pt-6 pb-24 px-4 relative overflow-hidden">
        <div className="absolute top-0 right-0 w-96 h-96 bg-white/10 rounded-full blur-3xl -translate-y-1/2 translate-x-1/2"></div>
        <div className="max-w-7xl mx-auto relative z-10">
          <button
            onClick={() => navigate("/")}
            className="flex items-center gap-2 text-white/90 hover:text-white transition-colors bg-white/10 backdrop-blur-md px-4 py-2 rounded-xl text-sm font-medium border border-white/20 shadow-sm w-fit mb-8"
          >
            <ArrowLeft className="w-4 h-4" /> Back to Dashboard
          </button>

          <div className="flex flex-col md:flex-row justify-between items-start md:items-end gap-6">
            <div className="space-y-3 max-w-2xl">
              <div className="flex items-center gap-2 text-blue-100 font-medium text-sm mb-2">
                <HeartPulse className="w-4 h-4" /> BAYMAX Healthcare
              </div>
              <h1 className="text-4xl md:text-5xl font-bold tracking-tight">
                Book an Appointment
              </h1>
              <p className="text-blue-100 text-lg md:text-xl font-light leading-relaxed max-w-xl">
                Find top-rated specialists in{" "}
                <strong className="font-semibold text-white">
                  Maduravoyal, Chennai
                </strong>{" "}
                and book your consultation instantly.
```

```
            </p>
          </div>

          <div className="flex items-center gap-3 bg-white/10 backdrop-blur-md
border border-white/20 p-3 rounded-2xl shadow-sm w-full md:w-auto">
            <div className="flex -space-x-3">
              {[1, 2, 3].map((i) => (
                <div
                  key={i}
                  className={`w-10 h-10 rounded-full border-2 border-[#3B82F6] flex
items-center justify-center font-bold text-sm ${i === 1 ? "bg-blue-100 text-blue-
600" : i === 2 ? "bg-emerald-100 text-emerald-600" : "bg-amber-100 text-amber-
600"}`}
                >
                  D
                </div>
              ))}
            </div>
            <div className="pl-2">
              <div className="text-xs text-blue-100 font-medium">
                Verified Doctors
              </div>
              <div className="text-sm font-bold">
                {doctors.length}+ Available
              </div>
            </div>
          </div>
        </div>
      </div>

      <div className="max-w-7xl mx-auto px-4 -mt-10 relative z-20">
        {/* Search Bar */}
        <div className="bg-white rounded-2xl shadow-lg border border-slate-100 p-
3 md:p-4 mb-10 flex flex-col md:flex-row items-center gap-4 max-w-4xl mx-auto">
```

```
        <div className="relative w-full flex-1">
          <Search className="absolute left-4 top-1/2 -translate-y-1/2 text-slate-400
w-5 h-5" />
          <input
            type="text"
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            placeholder="Search by doctor name, specialization, or hospital..."
            className="w-full pl-12 pr-4 py-3.5 bg-slate-50 border border-
transparent rounded-xl focus:bg-white focus:border-blue-500 focus:ring-4
focus:ring-blue-50 outline-none transition-all text-slate-700 font-medium"
          />
        </div>
        <button className="w-full md:w-auto bg-[#3B82F6] hover:bg-blue-600 text-
white font-bold py-3.5 px-8 rounded-xl transition-colors shadow-sm whitespace-
nowrap">
          Search Doctors
        </button>
      </div>

      {loading ? (
        <div className="flex flex-col items-center justify-center py-20">
          <div className="animate-spin rounded-full h-14 w-14 border-4 border-
slate-200 border-t-[#3B82F6]"></div>
          <p className="mt-4 text-slate-500 font-medium">
            Finding the best doctors for you...
          </p>
        </div>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {filteredDoctors.map((doc, idx) => (
            <div
              key={idx}
```

```
            className="bg-white rounded-[24px] shadow-[0_2px_15px_-
4px_rgba(0,0,0,0.05)] border border-slate-100 hover:shadow-xl hover:-translate-y-
1 transition-all duration-300 flex flex-col overflow-hidden group"
          >
            {/* Top Banner Context */}
            <div
              className={`h-2 w-full ${doc.waitingCount > 10 ? "bg-orange-400" :
doc.waitingCount > 5 ? "bg-blue-400" : "bg-emerald-400"}`}
            ></div>

            <div className="p-6 md:p-8 flex flex-col h-full">
              <div className="flex items-start justify-between mb-6">
                <div className="flex items-center gap-4">
                  <div className="w-16 h-16 rounded-2xl bg-gradient-to-br from-
blue-50 to-indigo-50 border border-blue-100 text-blue-600 flex items-center justify-
center text-2xl font-bold shadow-inner group-hover:scale-110 transition-
transform">
                    {doc.user?.name
                      ? doc.user.name.charAt(0).toUpperCase()
                      : "D"}{" "}
                  </div>
                  <div>
                    <h3 className="text-[19px] font-bold text-slate-800 leading-tight
mb-1">
                      {doc.user?.name}
                    </h3>
                    <span className="inline-flex items-center px-2.5 py-1 rounded-md
bg-blue-50 text-blue-600 text-xs font-bold">
                      {doc.specialization}
                    </span>
                  </div>
                </div>
                <div
                  className="bg-emerald-50 p-2 rounded-xl text-emerald-600"
                  title="Verified Specialist"
```

```
            >
              <ShieldCheck className="w-5 h-5" />
            </div>
          </div>

          <div className="space-y-4 mb-8 flex-grow">
            <div className="flex items-start gap-3">
              <div className="bg-slate-50 p-2 rounded-lg mt-0.5">
                <MapPin className="w-4 h-4 text-slate-500" />
              </div>
              <div>
                <p className="text-sm font-semibold text-slate-700">
                  {doc.hospital}
                </p>
                <p className="text-xs text-slate-500 mt-0.5 leading-relaxed">
                  {doc.address}
                </p>
                <button
                  onClick={() =>
                    setMapLocation(
                      `${doc.hospital}, ${doc.address || "Maduravoyal, Chennai"}`,
                    )
                  }
                  className="inline-block text-[#3B82F6] hover:text-blue-700
hover:underline mt-1.5 font-semibold text-[13px]"
                >
                  View on Map →
                </button>
              </div>
            </div>

            <div className="grid grid-cols-2 gap-3">
              <div className="bg-slate-50 rounded-xl p-3 flex items-center gap-
2.5 border border-slate-100">
                <Star className="w-4 h-4 text-amber-400 fill-amber-400" />
```

```
                <div className="flex flex-col">
                  <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-wider">
                    Experience
                  </span>
                  <span className="text-sm font-bold text-slate-700">
                    {doc.experience} Yrs
                  </span>
                </div>
              </div>

              <div className="bg-slate-50 rounded-xl p-3 flex items-center gap-
2.5 border border-slate-100">
                <Users className="w-4 h-4 text-blue-500" />
                <div className="flex flex-col">
                  <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-wider">
                    Completed
                  </span>
                  <span className="text-sm font-bold text-slate-700">
                    {doc.completedCount || 0}+
                  </span>
                </div>
              </div>

              <div className="bg-slate-50 rounded-xl p-3 flex items-center gap-
2.5 border border-slate-100">
                <IndianRupee className="w-4 h-4 text-emerald-500" />
                <div className="flex flex-col">
                  <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-wider">
                    Consult Fee
                  </span>
                  <span className="text-sm font-bold text-slate-700">
                    ₹{doc.fee || 500}
```

```
                    </span>
                  </div>
                </div>

                <div className="bg-slate-50 rounded-xl p-3 flex items-center gap-
2.5 border border-slate-100">
                  <ThumbsUp className="w-4 h-4 text-purple-500" />
                  <div className="flex flex-col">
                    <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-wider">
                      Rating
                    </span>
                    <span className="text-sm font-bold text-slate-700">
                      {doc.recommendation || "90%"}
                    </span>
                  </div>
                </div>
              </div>

              {/* Waiting Queue Status */}
              <div className="mb-5 flex items-center justify-between bg-slate-50
px-4 py-3 rounded-xl border border-slate-100">
                <div className="flex items-center gap-2">
                  <Activity
                    className={`w-4 h-4 ${doc.waitingCount > 10 ? "text-orange-500"
: "text-emerald-500"}`}
                  />
                  <span className="text-sm font-semibold text-slate-700">
                    Live Queue
                  </span>
                </div>
                <div className="flex items-center gap-2">
                  {doc.waitingCount > 0 ? (
                    <>
```

```
                    <div
                      className={`w-2 h-2 rounded-full animate-pulse
${doc.waitingCount > 10 ? "bg-orange-500" : "bg-emerald-500"}`}
                    ></div>
                    <span
                      className={`text-sm font-bold ${doc.waitingCount > 10 ? "text-
orange-600" : "text-emerald-600"}`}
                    >
                      {doc.waitingCount} Waiting
                    </span>
                  </>
                ) : (
                  <span className="text-sm font-bold text-slate-400">
                    No Wait Time
                  </span>
                )}
              </div>
            </div>

            <button
              onClick={() => setSelectedDoctor(doc)}
              className="w-full bg-[#3B82F6] text-white font-bold py-3.5 rounded-
xl hover:bg-blue-600 transition-colors shadow-sm flex items-center justify-center
gap-2"
            >
              <Calendar className="w-4 h-4" /> Book Appointment
            </button>
          </div>
        </div>
      ))}

      {filteredDoctors.length === 0 && (
        <div className="col-span-full bg-white rounded-3xl p-12 text-center
border border-slate-200 shadow-sm">
```

```
            <div className="w-20 h-20 bg-slate-50 rounded-full flex items-center
justify-center mx-auto mb-4">
              <Search className="w-8 h-8 text-slate-400" />
            </div>
            <h3 className="text-xl font-bold text-slate-800 mb-2">
              No doctors found
            </h3>
            <p className="text-slate-500 max-w-md mx-auto">
              We couldn't find any doctors in Maduravoyal matching "
              {searchQuery}". Try adjusting your search criteria.
            </p>
            <button
              onClick={() => setSearchQuery("")}
              className="mt-6 text-[#3B82F6] font-semibold hover:underline"
            >
              Clear Search
            </button>
          </div>
        )}
      </div>
    )}

    {/* Improved Booking Modal */}
    {selectedDoctor && (
      <div className="fixed inset-0 bg-slate-900/40 backdrop-blur-sm z-50 flex
items-center justify-center p-4 overflow-y-auto">
        <div className="bg-white rounded-[24px] w-full max-w-md overflow-
hidden shadow-2xl animate-in fade-in zoom-in duration-200">
          <div className="bg-gradient-to-r from-[#2563EB] to-[#4F46E5] p-8 text-
white text-center relative">
            <button
              onClick={() => setSelectedDoctor(null)}
              className="absolute top-4 right-4 bg-white/20 p-2 rounded-full
hover:bg-white/30 transition-colors backdrop-blur-md"
            >
```

```
            <ArrowLeft className="w-5 h-5" />
          </button>
          <div className="w-24 h-24 bg-white text-[#3B82F6] rounded-[24px]
flex items-center justify-center text-4xl font-bold mx-auto mb-4 shadow-inner">
            {selectedDoctor.user?.name
              ? selectedDoctor.user.name.charAt(0).toUpperCase()
              : "D"}
          </div>
          <h3 className="text-2xl font-bold mb-1">
            {selectedDoctor.user?.name}
          </h3>
          <span className="inline-block bg-white/20 px-3 py-1 rounded-lg text-
sm font-medium backdrop-blur-sm border border-white/10">
            {selectedDoctor.specialization}
          </span>
        </div>

        <form onSubmit={handleBook} className="p-8 space-y-6">
          <div>
            <label className="block text-sm font-bold text-slate-700 mb-2">
              Preferred Date
            </label>
            <div className="relative group">
              <Calendar className="absolute left-4 top-1/2 -translate-y-1/2 w-5 h-
5 text-slate-400 group-focus-within:text-[#3B82F6] transition-colors" />
              <input
                type="date"
                required
                min={new Date().toISOString().split("T")[0]}
                value={date}
                onChange={(e) => setDate(e.target.value)}
                className="w-full pl-12 pr-4 py-3.5 bg-slate-50 border border-slate-
200 rounded-xl focus:bg-white focus:ring-2 focus:ring-[#3B82F6] focus:border-
[#3B82F6] outline-none transition-all font-medium text-slate-700"
              />
```

```
            </div>
          </div>

          <div>
            <label className="block text-sm font-bold text-slate-700 mb-2">
              Preferred Time
            </label>
            <div className="relative group">
              <Clock className="absolute left-4 top-1/2 -translate-y-1/2 w-5 h-5
text-slate-400 group-focus-within:text-[#3B82F6] transition-colors" />
              <input
                type="time"
                required
                value={time}
                onChange={(e) => setTime(e.target.value)}
                className="w-full pl-12 pr-4 py-3.5 bg-slate-50 border border-slate-
200 rounded-xl focus:bg-white focus:ring-2 focus:ring-[#3B82F6] focus:border-
[#3B82F6] outline-none transition-all font-medium text-slate-700"
              />
            </div>
          </div>

          <div className="bg-blue-50 p-4 rounded-xl flex items-start gap-3">
            <Activity className="w-5 h-5 text-blue-600 mt-0.5 shrink-0" />
            <p className="text-[13px] text-blue-800 font-medium leading-
relaxed">
              By confirming this booking, you will be added to the
              doctor's queue. Currently, there are{" "}
              <strong className="font-bold">
                {selectedDoctor.waitingCount} patients
              </strong>{" "}
              waiting.
            </p>
          </div>
```

```
            <div className="pt-2 flex gap-3">
              <button
                type="button"
                onClick={() => setSelectedDoctor(null)}
                className="flex-1 py-3.5 border border-slate-200 text-slate-700 font-bold rounded-xl hover:bg-slate-50 transition-colors"
              >
                Cancel
              </button>
              <button
                type="submit"
                className="flex-[2] py-3.5 bg-[#3B82F6] text-white font-bold rounded-xl hover:bg-blue-600 transition-colors shadow-md"
              >
                Confirm Booking
              </button>
            </div>
          </form>
        </div>
      </div>
    )}

    {/* Map Modal */}
    {mapLocation && (
      <div className="fixed inset-0 bg-slate-900/60 backdrop-blur-sm z-50 flex items-center justify-center p-4">
        <div className="bg-white rounded-3xl w-full max-w-4xl h-[80vh] overflow-hidden shadow-2xl flex flex-col animate-in fade-in zoom-in duration-200">
          <div className="p-4 bg-white border-b border-slate-100 flex items-center justify-between">
            <div>
              <h3 className="text-lg font-bold text-slate-800">
                Hospital Location
              </h3>
              <p className="text-sm text-slate-500">{mapLocation}</p>
```

```
              </div>
              <button
                onClick={() => setMapLocation(null)}
                className="p-2 hover:bg-slate-100 rounded-full transition-colors"
              >
                <X className="w-6 h-6 text-slate-500" />
              </button>
            </div>
            <div className="flex-grow bg-slate-100 relative">
              <iframe
                title="Google Maps Location"
                width="100%"
                height="100%"
                style={{ border: 0 }}
                loading="lazy"
                allowFullScreen
                referrerPolicy="no-referrer-when-downgrade"
                src={`https://maps.google.com/maps?q=${encodeURIComponent(map
Location + ", Chennai")}&t=&z=15&ie=UTF8&iwloc=&output=embed`}
              ></iframe>
            </div>
          </div>
        </div>
      )}
    </div>
  </div>
 );
}
```

C2.1.3.7 DoctorDashboard.tsx

```
import { useEffect, useState } from "react";
import api from "../services/api";
import { useAuth } from "../context/AuthContext";
import {
  LogOut,
```

```
  Activity,
  CheckCircle2,
  Users,
  Minus,
  Plus,
  Video,
} from "lucide-react";
import { useNavigate } from "react-router-dom";

export default function DoctorDashboard() {
  const [appointments, setAppointments] = useState([]);
  const [profile, setProfile] = useState<any>(null);
  const { logout, user } = useAuth();
  const navigate = useNavigate();

  const fetchData = async () => {
    try {
      const [aptRes, profRes] = await Promise.all([
        api.get("/doctor/appointments"),
        api.get("/doctor/profile"),
      ]);
      setAppointments(aptRes.data);
      setProfile(profRes.data);
    } catch (err) {
      console.error(err);
    }
  };

  useEffect(() => {
    fetchData();
  }, []);

  const handleStatusChange = async (id: string, status: string) => {
    try {
      await api.put(`/doctor/appointment/${id}/status`, { status });
```

```
    fetchData();
  } catch (err) {
    console.error(err);
  }
};

const handleMarkDone = async (id: string) => {
  try {
    await api.put(`/doctor/appointment/${id}/done`);
    fetchData();
  } catch (err) {
    console.error(err);
  }
};

const updateWaitingCount = async (action: "increment" | "decrement") => {
  if (
    action === "decrement" &&
    (!profile?.waitingCount || profile.waitingCount <= 0)
  )
    return;
  try {
    const { data } = await api.put("/doctor/waiting-count", { action });
    setProfile(data);
  } catch (err) {
    console.error(err);
  }
};

return (
  <div className="min-h-screen bg-slate-50">
    <nav className="bg-white border-b border-slate-200 sticky top-0 z-50">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 h-16 flex items-
center justify-between">
        <div className="flex items-center gap-2">
```

```
            <div className="bg-[#06B6D4] p-1.5 rounded-lg">
              <Activity className="w-5 h-5 text-white" />
            </div>
            <span className="text-xl font-bold text-slate-800 tracking-tight">
              BAYMAX Doctor
            </span>
          </div>
          <button
            onClick={() => {
              logout();
              navigate("/login");
            }}
            className="flex items-center gap-2 text-slate-600 hover:text-red-500
transition-colors font-medium text-sm"
          >
            <LogOut className="w-4 h-4" />
            Sign Out
          </button>
        </div>
      </nav>

      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <div className="mb-8">
          <h1 className="text-2xl font-bold text-slate-800">
            Welcome, {user?.name}
          </h1>
          <p className="text-slate-500">
            Manage your appointments and waiting list.
          </p>
        </div>

        <div
          className={`p-6 rounded-2xl transition-all duration-500 mb-8 border
shadow-lg ${
            (profile?.waitingCount || 0) >= 15
```

```
            ? "bg-gradient-to-r from-red-500 to-red-600 border-red-500 shadow-red-
200/50"
            : (profile?.waitingCount || 0) >= 10
              ? "bg-gradient-to-r from-orange-500 to-orange-600 border-orange-500
shadow-orange-200/50"
              : (profile?.waitingCount || 0) >= 5
                ? "bg-gradient-to-r from-blue-500 to-blue-600 border-blue-600 shadow-
blue-200/50"
                : (profile?.waitingCount || 0) >= 1
                  ? "bg-gradient-to-r from-emerald-400 to-emerald-500 border-emerald-
500 shadow-emerald-200/50"
                  : "bg-slate-100 border-slate-200 shadow-slate-200/50"
          }`}
        >
          <div className="mb-4">
            <h2
              className={`text-lg font-bold flex items-center gap-2 ${
                (profile?.waitingCount || 0) > 0
                  ? "text-white"
                  : "text-slate-700"
              }`}
            >
              <Activity className="w-5 h-5" />
              Live Dashboard Status
            </h2>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
            <div className="bg-white rounded-xl shadow-sm border border-slate-100
p-6 flex items-center justify-between transform transition-transform hover:scale-
105 duration-300">
              <div>
                <p className="text-sm font-medium text-slate-500 mb-1">
                  Live Waiting
                </p>
                <div className="flex items-center gap-4">
```

```
                <button
                  onClick={() => updateWaitingCount("decrement")}
                  className={`w-8 h-8 rounded-full flex items-center justify-center
transition-colors ${profile?.waitingCount > 0 ? "bg-slate-100 hover:bg-slate-200" :
"bg-slate-50 opacity-50 cursor-not-allowed"}`}
                  disabled={
                    !profile?.waitingCount || profile.waitingCount <= 0
                  }
                  title="Mark a manual patient as done"
                >
                  <Minus className="w-4 h-4 text-slate-600" />
                </button>
                <span className="text-3xl font-bold text-slate-800">
                  {profile?.waitingCount || 0}
                </span>
                <button
                  onClick={() => updateWaitingCount("increment")}
                  className="w-8 h-8 rounded-full bg-[#06B6D4] text-white flex items-
center justify-center hover:bg-cyan-600"
                  title="Add a manual patient to waiting list"
                >
                  <Plus className="w-4 h-4" />
                </button>
              </div>
            </div>
            <div className="bg-cyan-50 p-3 rounded-full hidden lg:block">
              <Users className="w-6 h-6 text-[#06B6D4]" />
            </div>
          </div>

          <div className="bg-white rounded-xl shadow-sm border border-slate-100
p-6 flex items-center justify-between transform transition-transform hover:scale-
105 duration-300">
            <div>
              <p className="text-sm font-medium text-slate-500 mb-1">
```

```
              Done Today
            </p>
            <h3 className="text-3xl font-bold text-slate-800">
              {profile?.completedCount || 0}
            </h3>
          </div>
          <div className="bg-green-50 p-3 rounded-full hidden lg:block">
            <CheckCircle2 className="w-6 h-6 text-green-500" />
          </div>
        </div>

        <div className="bg-white rounded-xl shadow-sm border border-slate-100
p-6 flex items-center justify-between transform transition-transform hover:scale-
105 duration-300">
          <div>
            <p className="text-sm font-medium text-slate-500 mb-1">
              Total Completed
            </p>
            <h3 className="text-3xl font-bold text-slate-800">
              {profile?.totalCompleted || profile?.completedCount || 0}
            </h3>
          </div>
          <div className="bg-purple-50 p-3 rounded-full hidden lg:block">
            <CheckCircle2 className="w-6 h-6 text-purple-500" />
          </div>
        </div>
      </div>

      <div className="bg-white rounded-xl shadow-sm border border-slate-200
overflow-hidden">
        <div className="p-6 border-b border-slate-200">
          <h2 className="text-xl font-bold text-slate-800">
            Appointment Requests
          </h2>
```

```
        </div>
        <div className="overflow-x-auto">
          <table className="w-full">
            <thead className="bg-slate-50 text-slate-500 text-sm font-medium text-
left">
              <tr>
                <th className="px-6 py-4">Patient</th>
                <th className="px-6 py-4">Date</th>
                <th className="px-6 py-4">Time</th>
                <th className="px-6 py-4">Status</th>
                <th className="px-6 py-4 text-right">Actions</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-200">
              {appointments.map((apt: any) => (
                <tr key={apt._id} className="hover:bg-slate-50">
                  <td className="px-6 py-4 font-medium text-slate-800">
                    {apt.patient?.name}
                  </td>
                  <td className="px-6 py-4 text-slate-600">{apt.date}</td>
                  <td className="px-6 py-4 text-slate-600">{apt.time}</td>
                  <td className="px-6 py-4">
                    <span
                      className={`inline-flex items-center px-2.5 py-0.5 rounded-full
text-xs font-medium ${
                        apt.status === "pending"
                          ? "bg-yellow-100 text-yellow-800"
                          : apt.status === "approved"
                            ? "bg-blue-100 text-blue-800"
                            : apt.status === "completed"
                              ? "bg-green-100 text-green-800"
                              : "bg-red-100 text-red-800"
                      }`}
                    >
```

```
                {apt.status.charAt(0).toUpperCase() +
                  apt.status.slice(1)}
                </span>
              </td>
              <td className="px-6 py-4 text-right space-x-2">
                {apt.status === "pending" && (
                  <>
                    <button
                      onClick={() =>
                        handleStatusChange(apt._id, "approved")
                      }
                      className="text-sm bg-blue-500 text-white px-3 py-1.5 rounded
hover:bg-blue-600"
                    >
                      Accept
                    </button>
                    <button
                      onClick={() =>
                        handleStatusChange(apt._id, "rejected")
                      }
                      className="text-sm bg-red-100 text-red-600 px-3 py-1.5
rounded hover:bg-red-200"
                    >
                      Reject
                    </button>
                  </>


                )}
                {apt.status === "approved" && (
                  <div className="flex items-center justify-end gap-2">
                    <button
                      onClick={() => navigate(`/video-call/${apt._id}`)}
                      className="text-sm bg-[#06B6D4] text-white px-3 py-1.5
rounded hover:bg-cyan-600 flex items-center gap-1.5 font-medium"
```

```
                    >
                      <Video className="w-4 h-4" /> Start Call
                    </button>
                    <button
                      onClick={() => handleMarkDone(apt._id)}
                      className="text-sm bg-green-500 text-white px-3 py-1.5
rounded hover:bg-green-600"
                    >
                      Mark Done
                    </button>
                  </div>
                )}
              </td>
            </tr>
          ))}
          {appointments.length === 0 && (
            <tr>
              <td
                colSpan={5}
                className="px-6 py-8 text-center text-slate-500"
              >
                No appointments found.
              </td>
            </tr>
          )}

        </tbody>
      </table>
    </div>
  </div>
</main>
</div>
);
}
```

C2.1.3.8 HealthRecords.tsx

```
import React, { useEffect, useState, useRef, useMemo } from "react";
import { useNavigate } from "react-router-dom";
import api from "../services/api";
import {
  ArrowLeft,
  Calendar,
  FileText,
  CheckCircle2,
  Clock,
  XCircle,
  FlaskConical,
  TestTube2,
  Upload,
  File,
  Image as ImageIcon,
  Activity,
  HeartPulse,
  Trash2,
  Droplet,
  Loader2,
} from "lucide-react";
import {
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  ResponsiveContainer,
  AreaChart,
  Area,
} from "recharts";

export default function HealthRecords() {
  const [appointments, setAppointments] = useState<any[]>([]);
  const [labBookings, setLabBookings] = useState<any[]>([]);
```

```
  const [documents, setDocuments] = useState<any[]>([]);

  const [loading, setLoading] = useState(true);
  const [uploading, setUploading] = useState(false);
  const [activeTab, setActiveTab] = useState<
    "overview" | "documents" | "appointments" | "labs"
  >("overview");

  const navigate = useNavigate();
  const fileInputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    fetchRecords();
  }, []);

  const fetchRecords = async () => {
    try {
      const [aptRes, labRes, docRes] = await Promise.all([
        api.get("/patients/appointments"),
        api.get("/patients/lab-bookings"),
        api.get("/patients/records"),
      ]);
      setAppointments(aptRes.data);
      setLabBookings(labRes.data);
      setDocuments(docRes.data?.documents || []);
    } catch (error) {
      console.error("Failed to fetch records", error);
    } finally {
      setLoading(false);
    }
  };

  const handleFileUpload = async (e: React.ChangeEvent<HTMLInputElement>)
=> {
    if (e.target.files && e.target.files.length > 0) {
```

```
    const file = e.target.files[0];
    setUploading(true);

    try {
      // Simulating upload and AI Data Extraction processing delay
      await new Promise((resolve) => setTimeout(resolve, 2000));

      const payload = {
        fileName: file.name,
        fileType: file.type,
        fileSize: (file.size / (1024 * 1024)).toFixed(2) + " MB",
      };

      const res = await api.post("/patients/records/upload", payload);

      // Add new document to state to reflect UI change instantly
      setDocuments([res.data, ...documents]);

      // Switch to overview to show them the new graph points automatically
      setActiveTab("overview");
    } catch (error) {
      console.error("Upload failed", error);
      alert("Failed to upload and analyze document. Please try again.");
    } finally {
      setUploading(false);
    }
  }
};

const handleDeleteFile = async (id: string) => {
  try {
    await api.delete(`/patients/records/${id}`);
    setDocuments(documents.filter((f) => f._id !== id));
  } catch (error) {
    console.error("Delete failed", error);
```

```
  }
};

const getStatusColor = (status: string) => {
  switch (status?.toLowerCase()) {
    case "completed":
      return "text-emerald-500 bg-emerald-50 border-emerald-200";
    case "approved":
      return "text-blue-500 bg-blue-50 border-blue-200";
    case "rejected":
      return "text-red-500 bg-red-50 border-red-200";
    default:
      return "text-orange-500 bg-orange-50 border-orange-200";
  }
};

const getStatusIcon = (status: string) => {
  switch (status?.toLowerCase()) {
    case "completed":
      return <CheckCircle2 className="w-4 h-4" />;
    case "approved":
      return <CheckCircle2 className="w-4 h-4" />;
    case "rejected":
      return <XCircle className="w-4 h-4" />;
    default:
      return <Clock className="w-4 h-4" />;
  }
};

// Dynamically compute graph data from uploaded documents
const graphData = useMemo(() => {
  if (!documents || documents.length === 0) return [];

  // Extract the AI analyzed points and sort by date
  const dataPoints = documents
```

```
    .filter((doc) => doc.extractedData)
    .map((doc) => ({
      month: doc.extractedData.date,
      bp: doc.extractedData.bloodPressure,
      sugar: doc.extractedData.bloodSugar,
      heartRate: doc.extractedData.heartRate,
      timestamp: new Date(doc.uploadDate).getTime(),
    }))
    .sort((a, b) => a.timestamp - b.timestamp);

  // Remove the raw timestamp before passing to Recharts
  return dataPoints.map(({ timestamp, ...rest }) => rest);
}, [documents]);

const avgStats = useMemo(() => {
  if (graphData.length === 0) return { bp: 0, sugar: 0, hr: 0 };
  const sums = graphData.reduce(
    (acc, curr) => ({
      bp: acc.bp + curr.bp,
      sugar: acc.sugar + curr.sugar,
      hr: acc.hr + curr.heartRate,
    }),
    { bp: 0, sugar: 0, hr: 0 },
  );

  return {
    bp: Math.round(sums.bp / graphData.length),
    sugar: Math.round(sums.sugar / graphData.length),
    hr: Math.round(sums.hr / graphData.length),
  };
}, [graphData]);

return (
  <div className="min-h-screen bg-[#F8FAFC] pb-20 p-4 md:p-8">
    <div className="max-w-6xl mx-auto space-y-8">
```

```
        <button
          onClick={() => navigate("/")}
          className="flex items-center gap-2 text-slate-500 hover:text-slate-800
transition-colors bg-white px-4 py-2 rounded-lg shadow-sm w-fit"
        >
          <ArrowLeft className="w-4 h-4" /> Back to Dashboard
        </button>

        <div className="flex flex-col lg:flex-row justify-between items-start lg:items-
center gap-6">
          <div>
            <h1 className="text-3xl font-bold text-slate-800 flex items-center gap-3">
              <FileText className="w-8 h-8 text-[#3B82F6]" />
              Health Records
            </h1>
            <p className="text-slate-500 mt-1">
              Manage your health data, documents, and medical history
            </p>
          </div>

          <div className="flex flex-wrap items-center gap-2 bg-white p-1.5 rounded-
2xl shadow-sm border border-slate-200">
            <button
              onClick={() => setActiveTab("overview")}
              className={`px-5 py-2.5 text-sm font-semibold rounded-xl transition-all
${activeTab === "overview" ? "bg-[#3B82F6] text-white shadow-md" : "text-slate-
500 hover:bg-slate-50"}`}
            >
              Overview
            </button>
            <button
              onClick={() => setActiveTab("documents")}
              className={`px-5 py-2.5 text-sm font-semibold rounded-xl transition-all
${activeTab === "documents" ? "bg-[#10B981] text-white shadow-md" : "text-slate-
500 hover:bg-slate-50"}`}
```

```
          >
            Documents
          </button>
          <button
            onClick={() => setActiveTab("appointments")}
            className={`px-5 py-2.5 text-sm font-semibold rounded-xl transition-all
${activeTab === "appointments" ? "bg-[#F59E0B] text-white shadow-md" : "text-
slate-500 hover:bg-slate-50"}`}
          >
            Appointments
          </button>
          <button
            onClick={() => setActiveTab("labs")}
            className={`px-5 py-2.5 text-sm font-semibold rounded-xl transition-all
${activeTab === "labs" ? "bg-[#A855F7] text-white shadow-md" : "text-slate-500
hover:bg-slate-50"}`}
          >
            Lab Results
          </button>
        </div>
      </div>

      {loading ? (
        <div className="flex justify-center p-12">
          <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-
[#3B82F6]"></div>
        </div>
      ) : (
        <div className="space-y-6 animate-in fade-in slide-in-from-bottom-4
duration-500">
          {/* OVERVIEW TAB */}
          {activeTab === "overview" && (
            <div className="space-y-6">
              {graphData.length > 0 ? (
                <>
```

```
            <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
              <div className="bg-white p-6 rounded-[24px] border border-slate-
200 shadow-sm flex items-center gap-4">
                <div className="w-14 h-14 bg-rose-100 text-rose-500 rounded-2xl
flex items-center justify-center">
                  <HeartPulse className="w-7 h-7" />
                </div>
                <div>
                  <p className="text-slate-500 text-sm font-medium">
                    Avg Heart Rate
                  </p>
                  <h4 className="text-2xl font-bold text-slate-800">
                    {avgStats.hr}{" "}
                    <span className="text-sm text-slate-400 font-normal">
                      bpm
                    </span>
                  </h4>
                </div>
              </div>
              <div className="bg-white p-6 rounded-[24px] border border-slate-
200 shadow-sm flex items-center gap-4">
                <div className="w-14 h-14 bg-blue-100 text-blue-500 rounded-2xl
flex items-center justify-center">
                  <Activity className="w-7 h-7" />
                </div>
                <div>
                  <p className="text-slate-500 text-sm font-medium">
                    Avg Blood Pressure (Sys)
                  </p>
                  <h4 className="text-2xl font-bold text-slate-800">
                    {avgStats.bp}{" "}
                    <span className="text-sm text-slate-400 font-normal">
                      mmHg
                    </span>
                  </h4>
```

```
                </div>
              </div>
              <div className="bg-white p-6 rounded-[24px] border border-slate-
200 shadow-sm flex items-center gap-4">
                <div className="w-14 h-14 bg-emerald-100 text-emerald-500
rounded-2xl flex items-center justify-center">
                  <Droplet className="w-7 h-7" />
                </div>
                <div>
                  <p className="text-slate-500 text-sm font-medium">
                    Avg Blood Sugar
                  </p>
                  <h4 className="text-2xl font-bold text-slate-800">
                    {avgStats.sugar}{" "}
                    <span className="text-sm text-slate-400 font-normal">
                      mg/dL
                    </span>
                  </h4>
                </div>
              </div>
            </div>

            <div className="bg-white p-6 md:p-8 rounded-[32px] border border-
slate-200 shadow-sm">
              <div className="flex items-center justify-between mb-8">
                <div>
                  <h2 className="text-xl font-bold text-slate-800 flex items-center
gap-2">
                    <Activity className="w-6 h-6 text-indigo-500" />
                    AI Health Analysis
                  </h2>
                  <p className="text-slate-500 mt-1 text-sm">
                    Trends extracted dynamically from your uploaded
                    documents
                  </p>
```

```
  </div>
</div>

<div className="h-[350px] w-full mt-4">
  <ResponsiveContainer width="100%" height="100%">
    <AreaChart
      data={graphData}
      margin={{
        top: 10,
        right: 10,
        left: -20,
        bottom: 0,
      }}
    >
      <defs>
        <linearGradient
          id="colorBp"
          x1="0"
          y1="0"
          x2="0"
          y2="1"
        >
          <stop
            offset="5%"
            stopColor="#3B82F6"
            stopOpacity={0.4}
          />
          <stop
            offset="95%"
            stopColor="#3B82F6"
            stopOpacity={0}
          />
        </linearGradient>
        <linearGradient
          id="colorSugar"
```

```
    x1="0"
    y1="0"
    x2="0"
    y2="1"
  >
    <stop
      offset="5%"
      stopColor="#10B981"
      stopOpacity={0.4}
    />
    <stop
      offset="95%"
      stopColor="#10B981"
      stopOpacity={0}
    />
  </linearGradient>
</defs>
<CartesianGrid
  strokeDasharray="3 3"
  vertical={false}
  stroke="#F1F5F9"
/>
<XAxis
  dataKey="month"
  stroke="#94A3B8"
  fontSize={12}
  tickLine={false}
  axisLine={false}
/>
<YAxis
  stroke="#94A3B8"
  fontSize={12}
  tickLine={false}
  axisLine={false}
/>
```

```
                    <Tooltip
                      contentStyle={{
                        borderRadius: "16px",
                        border: "none",
                        boxShadow: "0 10px 15px -3px rgb(0 0 0 / 0.1)",
                        padding: "12px 20px",
                      }}
                      itemStyle={{ fontWeight: "bold" }}
                    />
                    <Area
                      type="monotone"
                      name="Blood Pressure"
                      dataKey="bp"
                      stroke="#3B82F6"
                      strokeWidth={4}
                      fillOpacity={1}
                      fill="url(#colorBp)"
                    />
                    <Area
                      type="monotone"
                      name="Blood Sugar"
                      dataKey="sugar"
                      stroke="#10B981"
                      strokeWidth={4}
                      fillOpacity={1}
                      fill="url(#colorSugar)"
                    />
                  </AreaChart>
                </ResponsiveContainer>
              </div>
            </div>
          </>
        ) : (
          <div className="bg-white p-16 rounded-[32px] border border-slate-
200 shadow-sm text-center flex flex-col items-center justify-center">
```

```
                <Activity className="w-20 h-20 text-slate-200 mb-6" />
                <h3 className="text-xl font-bold text-slate-800">
                  No Health Data Available
                </h3>
                <p className="text-slate-500 mt-2 max-w-sm">
                  Upload your medical reports or scans in the Documents tab.
                  BAYMAX AI will analyze them and generate your health trend
                  graph.
                </p>
                <button
                  onClick={() => setActiveTab("documents")}
                  className="mt-8 px-8 py-3 bg-[#3B82F6] text-white font-semibold
rounded-2xl hover:bg-blue-600 shadow-sm transition-all hover:shadow-md"
                >
                  Upload Documents
                </button>
              </div>
            )}
          </div>
        )}

        {/* DOCUMENTS TAB */}
        {activeTab === "documents" && (
          <div className="space-y-6">
            <div
              onClick={() => !uploading && fileInputRef.current?.click()}
              className={`border-2 border-dashed border-emerald-200 bg-emerald-
50/30 transition-all rounded-[32px] p-12 text-center flex flex-col items-center
justify-center shadow-sm ${uploading ? "opacity-70 cursor-not-allowed" : "cursor-
pointer hover:bg-emerald-50/80 group"}`}
            >
              <div className="w-20 h-20 bg-white rounded-full shadow-md flex
items-center justify-center mb-5 group-hover:scale-110 group-hover:shadow-lg
transition-all duration-300">
                {uploading ? (
```

```
                <Loader2 className="w-10 h-10 text-emerald-500 animate-spin" />
              ) : (
                <Upload className="w-10 h-10 text-emerald-500" />
              )}
            </div>
            <h3 className="text-xl font-bold text-slate-800">
              {uploading
                ? "BAYMAX is Analyzing Document..."
                : "Upload Scans & Medical Reports"}
            </h3>
            <p className="text-slate-500 mt-2 max-w-sm mx-auto">
              {uploading
                ? "Extracting vital signs and health data using AI..."
                : "Click or drag and drop your physical records, x-rays, or prescriptions here to digitize and extract insights."}
            </p>
            {!uploading && (
              <div className="mt-4 px-4 py-1.5 bg-emerald-100 text-emerald-700 text-xs font-semibold rounded-full">
                PDF, JPG, PNG up to 10MB
              </div>
            )}
            <input
              type="file"
              ref={fileInputRef}
              onChange={handleFileUpload}
              className="hidden"
              accept=".pdf,.jpg,.jpeg,.png"
              disabled={uploading}
            />
          </div>

          <div className="bg-white rounded-[32px] border border-slate-200 shadow-sm overflow-hidden">
```

```
            <div className="p-6 md:p-8 border-b border-slate-100 flex items-
center gap-3">
              <div className="w-10 h-10 bg-slate-100 text-slate-600 rounded-xl
flex items-center justify-center">
                <FileText className="w-5 h-5" />
              </div>
              <h3 className="text-xl font-bold text-slate-800">
                Your Documents
              </h3>
            </div>
            <div className="divide-y divide-slate-100">
              {documents.map((file) => (
                <div
                  key={file._id}
                  className="p-4 md:p-6 flex items-center justify-between hover:bg-
slate-50/80 transition-colors"
                >
                  <div className="flex items-center gap-4">
                    <div
                      className={`w-14 h-14 rounded-2xl flex items-center justify-
center flex-shrink-0 shadow-sm ${file.fileType.includes("image") ? "bg-purple-100
text-purple-600" : "bg-rose-100 text-rose-600"}`}
                    >
                      {file.fileType.includes("image") ? (
                        <ImageIcon className="w-7 h-7" />
                      ) : (
                        <File className="w-7 h-7" />
                      )}
                    </div>
                    <div>
                      <p className="font-bold text-slate-800 line-clamp-1">
                        {file.fileName}
                      </p>
                      <p className="text-xs sm:text-sm text-slate-500 flex gap-2 font-
medium mt-1">
```

```
                    <span>
                      Uploaded on{" "}
                      {new Date(file.uploadDate).toLocaleDateString()}
                    </span>
                    <span className="text-slate-300">•</span>
                    <span>{file.fileSize}</span>
                  </p>
                  {file.extractedData && (
                    <p className="text-xs text-emerald-600 mt-1 font-semibold
flex items-center gap-1">
                      <Activity className="w-3 h-3" /> Data Extracted
                      Successfully
                    </p>
                  )}
                </div>
              </div>
              <button
                onClick={() => handleDeleteFile(file._id)}
                className="w-10 h-10 flex items-center justify-center text-slate-
400 hover:text-red-500 hover:bg-red-50 rounded-xl transition-colors"
                title="Delete Document"
              >
                <Trash2 className="w-5 h-5" />
              </button>
            </div>
          ))}
          {documents.length === 0 && (
            <div className="p-16 text-center text-slate-500 flex flex-col items-
center">
              <FileText className="w-16 h-16 text-slate-200 mb-4" />
              <p className="text-lg font-medium text-slate-700">
                No documents uploaded yet.
              </p>
            </div>
          )}
```

```
            </div>
          </div>
        </div>
      )}

      {/* APPOINTMENTS TAB */}
      {activeTab === "appointments" && (
        <div className="bg-white rounded-[32px] border border-slate-200
shadow-sm overflow-hidden">
          {appointments.length > 0 ? (
            <div className="divide-y divide-slate-100">
              {appointments.map((apt, idx) => (
                <div
                  key={idx}
                  className="p-6 flex flex-col md:flex-row items-start md:items-
center gap-6 hover:bg-slate-50/50 transition-colors"
                >
                  <div className="w-14 h-14 bg-orange-100 text-orange-600
rounded-full flex items-center justify-center text-xl font-bold flex-shrink-0">
                    {apt.doctor?.name?.charAt(0).toUpperCase() || "D"}
                  </div>

                  <div className="flex-grow space-y-1">
                    <h3 className="text-lg font-bold text-slate-800">
                      Dr. {apt.doctor?.name}
                    </h3>
                    <p className="text-slate-500 text-sm flex items-center gap-2">
                      {apt.doctorProfile?.specialization || "Specialist"}{" "}
                      • {apt.doctorProfile?.hospital || "Hospital"}
                    </p>
                  </div>

                  <div className="flex flex-col sm:flex-row items-start sm:items-
center gap-4 sm:gap-8 w-full md:w-auto mt-4 md:mt-0">
                    <div className="space-y-1">
```

```
                    <div className="flex items-center gap-2 text-slate-600 font-
medium">
                      <Calendar className="w-4 h-4 text-slate-400" />
                      {new Date(apt.date).toLocaleDateString()}
                    </div>
                    <div className="flex items-center gap-2 text-slate-600 font-
medium">
                      <Clock className="w-4 h-4 text-slate-400" />
                      {apt.time}
                    </div>
                  </div>

                  <div
                    className={`flex items-center gap-1.5 px-4 py-2 rounded-full
text-sm font-semibold border capitalize whitespace-nowrap
${getStatusColor(apt.status)}`}
                  >
                    {getStatusIcon(apt.status)}
                    {apt.status}
                  </div>
                </div>
              </div>
            ))}
          </div>
        ) : (
          <div className="p-16 text-center flex flex-col items-center justify-
center min-h-[300px]">
            <FileText className="w-20 h-20 text-slate-200 mb-6" />
            <h3 className="text-xl font-bold text-slate-800">
              No appointments found
            </h3>
            <p className="text-slate-500 mt-2 max-w-sm">
              You haven't booked any appointments yet. Book one to start
              tracking your history.
            </p>
```

```
                <button
                  onClick={() => navigate("/appointments")}
                  className="mt-8 px-8 py-3 bg-[#F59E0B] text-white font-semibold
rounded-2xl hover:bg-orange-500 shadow-sm transition-all hover:shadow-md"
                >
                  Book Appointment
                </button>
              </div>
            )}
          </div>
        )}

        {/* LABS TAB */}
        {activeTab === "labs" && (
          <div className="bg-white rounded-[32px] border border-slate-200
shadow-sm overflow-hidden">
            {labBookings.length > 0 ? (
              <div className="divide-y divide-slate-100">
                {labBookings.map((booking, idx) => (
                  <div
                    key={idx}
                    className="p-6 flex flex-col md:flex-row items-start md:items-
center gap-6 hover:bg-slate-50/50 transition-colors"
                  >
                    <div className="w-14 h-14 bg-purple-100 text-purple-600
rounded-2xl flex items-center justify-center text-xl flex-shrink-0">
                      <TestTube2 className="w-7 h-7" />
                    </div>

                    <div className="flex-grow space-y-1">
                      <h3 className="text-lg font-bold text-slate-800">
                        {booking.testDetails?.name || booking.labTestId}
                      </h3>
                      <p className="text-slate-500 text-sm font-medium">
                        Booked on:{" "}
```

```
                {new Date(booking.createdAt).toLocaleDateString()}
              </p>
            </div>

            <div className="flex flex-col sm:flex-row items-start sm:items-
center gap-4 sm:gap-8 w-full md:w-auto mt-4 md:mt-0">
              <div
              className={`flex items-center gap-1.5 px-4 py-2 rounded-full
text-sm font-semibold border capitalize whitespace-nowrap
${getStatusColor(booking.status)}`}
              >
              {getStatusIcon(booking.status)}
              {booking.status}
              </div>

              {booking.status === "completed" && (
                <button className="text-[#A855F7] hover:bg-purple-50 px-4 py-
2 rounded-xl transition-colors text-sm font-bold whitespace-nowrap">
                  Download Report
                </button>
              )}
            </div>
          </div>
        ))}
      </div>
    ) : (
      <div className="p-16 text-center flex flex-col items-center justify-
center min-h-[300px]">
        <FlaskConical className="w-20 h-20 text-slate-200 mb-6" />
        <h3 className="text-xl font-bold text-slate-800">
          No lab tests found
        </h3>
        <p className="text-slate-500 mt-2 max-w-sm">
          You haven't booked any lab tests yet.
        </p>
```

```
                <button
                  onClick={() => navigate("/labs")}
                  className="mt-8 px-8 py-3 bg-[#A855F7] text-white font-semibold
rounded-2xl hover:bg-purple-600 shadow-sm transition-all hover:shadow-md"
                >
                  Book Lab Test
                </button>
              </div>
            )}
          </div>
        )}
      </div>
    )}
  </div>
 );
}
```

C2.1.3.9 Insurance.tsx

```
import React from "react";
import { useNavigate } from "react-router-dom";
import {
  ArrowLeft,
  Shield,
  AlertCircle,
  FileText,
  CheckCircle,
} from "lucide-react";

export default function Insurance() {
  const navigate = useNavigate();

  return (
    <div className="min-h-screen bg-[#F8FAFC] pb-20 p-4 md:p-8">
      <div className="max-w-6xl mx-auto space-y-8">
```

```
        <button
          onClick={() => navigate("/")}
          className="flex items-center gap-2 text-slate-500 hover:text-slate-800
transition-colors bg-white px-4 py-2 rounded-lg shadow-sm w-max"
        >
          <ArrowLeft className="w-4 h-4" /> Back to Dashboard
        </button>

        <div className="bg-gradient-to-r from-[#00A99D] to-[#047857] p-8 rounded-
[24px] text-white shadow-lg">
          <div className="flex items-center gap-4 mb-4">
            <div className="bg-white/20 p-3 rounded-2xl backdrop-blur-sm">
              <Shield className="w-8 h-8 text-white" />
            </div>
            <div>
              <h1 className="text-3xl font-bold">Health Insurance</h1>
              <p className="text-teal-50">
                Manage your policies and claims seamlessly.
              </p>
            </div>
          </div>
        </div>

        <div className="bg-white p-12 rounded-[24px] shadow-sm border border-
slate-200 text-center">
          <Shield className="w-20 h-20 text-[#00A99D] mx-auto mb-6 opacity-80" />
          <h2 className="text-2xl font-bold text-slate-800 mb-2">
            No Active Policies Found
          </h2>
          <p className="text-slate-500 max-w-md mx-auto mb-8">
            You currently don't have any health insurance policies linked to
            your BAYMAX account. Link your existing policy or explore new plans
            to secure your health.
          </p>
```

```
        <div className="flex flex-col sm:flex-row items-center justify-center gap-4">
          <button className="bg-[#00A99D] hover:bg-teal-600 text-white px-8 py-3 rounded-xl font-semibold shadow-sm transition-colors w-full sm:w-auto flex items-center justify-center gap-2">
            <CheckCircle className="w-5 h-5" /> Link Existing Policy
          </button>
          <button className="bg-slate-100 hover:bg-slate-200 text-slate-700 px-8 py-3 rounded-xl font-semibold transition-colors w-full sm:w-auto flex items-center justify-center gap-2">
            <FileText className="w-5 h-5" /> Explore Plans
          </button>
        </div>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div className="bg-white p-6 rounded-[24px] border border-slate-200 shadow-sm flex items-start gap-4">
          <div className="bg-blue-50 p-3 rounded-xl text-blue-500">
            <AlertCircle className="w-6 h-6" />
          </div>
          <div>
            <h3 className="font-bold text-slate-800 text-lg mb-1">
              Need Help?
            </h3>
            <p className="text-slate-500 text-sm mb-3">
              Our support team is available 24/7 to assist you with insurance queries.
            </p>
            <button className="text-blue-500 font-semibold text-sm hover:underline">
              Contact Support
            </button>
          </div>
        </div>
```

```
        <div className="bg-white p-6 rounded-[24px] border border-slate-200
shadow-sm flex items-start gap-4">
          <div className="bg-amber-50 p-3 rounded-xl text-amber-500">
            <FileText className="w-6 h-6" />
          </div>
          <div>
            <h3 className="font-bold text-slate-800 text-lg mb-1">
              Claim Status
            </h3>
            <p className="text-slate-500 text-sm mb-3">
              Track your recent reimbursement requests and pre-authorization
              status.
            </p>
            <button className="text-amber-500 font-semibold text-sm
hover:underline">
              Track Claims
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}
```

C2.1.3.10 LabTestsView.tsx

```
import React, { useEffect, useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import api from "../services/api";
import {
  ArrowLeft,
  FlaskConical,
  TestTube2,
```

```
  CheckCircle2,
  Calendar,
  Clock,
  MapPin,
  MailCheck,
  X,
} from "lucide-react";

export default function LabTestsView() {
  const [tests, setTests] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  // Modal state
  const [selectedTest, setSelectedTest] = useState<any | null>(null);
  const [bookingData, setBookingData] = useState({
    date: "",
    time: "",
    type: "home", // 'home' or 'walk-in'
    address: "",
  });
  const [bookingLoading, setBookingLoading] = useState(false);
  const [successMsg, setSuccessMsg] = useState("");

  useEffect(() => {
    fetchTests();
  }, []);

  const fetchTests = async () => {
    try {
      const res = await axios.get("http://localhost:5000/api/labs");
      console.log("DATA RECEIVED:", res.data);
      setTests(res.data);
    } catch (error: any) {
      console.log("ERROR:", error);
```

```
  } finally {
    setLoading(false);
  }
};

const handleBookSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!selectedTest) return;

  setBookingLoading(true);
  try {
    const payload = {
      labTestId: selectedTest._id,
      ...bookingData,
    };
    const res = await api.post("/patients/book-lab", payload);
    setSuccessMsg(
      res.data.message ||
        "Lab booked successfully. Notification sent to your email ID.",
    );

    // Reset after 3 seconds
    setTimeout(() => {
      setSuccessMsg("");
      setSelectedTest(null);
      navigate("/"); // Redirect to dashboard to see reminders
    }, 3000);
  } catch (error) {
    console.error(error);
    alert("Failed to book test. Please try again.");
  } finally {
    setBookingLoading(false);
  }
};
```

```
  return (
    <div className="min-h-screen bg-[#F8FAFC] pb-20 p-4 md:p-8">
      <div className="max-w-6xl mx-auto space-y-8 relative">
        <button
          onClick={() => navigate("/")}
          className="flex items-center gap-2 text-slate-500 hover:text-slate-800
transition-colors bg-white px-4 py-2 rounded-lg shadow-sm w-fit"
        >
          <ArrowLeft className="w-4 h-4" /> Back to Dashboard
        </button>

        <div>
          <h1 className="text-3xl font-bold text-slate-800 flex items-center gap-2">
            <FlaskConical className="w-8 h-8 text-[#A855F7]" /> Lab Tests &
            Diagnostics
          </h1>
          <p className="text-slate-500 mt-1">
            Book diagnostic tests. Choose Home Collection or Walk-In.
          </p>
        </div>

        {loading ? (
          <div className="flex justify-center p-12">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-
[#A855F7]"></div>
          </div>
        ) : (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            {tests.map((test, idx) => (
              <div
                key={idx}
                className="bg-white p-6 rounded-2xl shadow-sm border border-slate-
200 hover:shadow-md transition-shadow flex flex-col h-full group"
              >
                <div className="flex items-start gap-4 mb-4">
```

```
            <div className="w-12 h-12 bg-purple-50 rounded-xl flex items-center
justify-center shrink-0 border border-purple-100 group-hover:scale-110 transition-
transform">
              <TestTube2 className="w-6 h-6 text-[#A855F7]" />
            </div>
            <div>
              <h3 className="text-lg font-bold text-slate-800">
                {test.name}
              </h3>
              <p className="text-sm text-slate-500 line-clamp-2 mt-1">
                {test.description}
              </p>
            </div>
          </div>

          <div className="mt-auto space-y-4 pt-4 border-t border-slate-100">
            <div className="flex items-center gap-2 text-sm text-emerald-600
font-medium bg-emerald-50 px-3 py-1.5 rounded-lg w-fit">
              <CheckCircle2 className="w-4 h-4" /> Reports in 24 hrs
            </div>

            <div className="flex items-center justify-between">
              <span className="text-2xl font-bold text-slate-800">
                ₹{test.price}
              </span>
              <button
                onClick={() => setSelectedTest(test)}
                className="bg-[#A855F7] text-white px-5 py-2.5 rounded-xl
hover:bg-purple-700 transition-colors font-semibold shadow-sm"
              >
                Book Test
              </button>
            </div>
          </div>
        </div>
```

```
        ))}
      </div>
    )}

    {/* Booking Modal */}
    {selectedTest && (
      <div className="fixed inset-0 bg-slate-900/50 backdrop-blur-sm z-50 flex
items-center justify-center p-4">
        <div className="bg-white rounded-2xl w-full max-w-lg shadow-2xl
overflow-hidden animate-in fade-in zoom-in-95 duration-200">
          <div className="px-6 py-4 border-b border-slate-100 flex items-center
justify-between bg-slate-50">
            <h3 className="font-bold text-lg text-slate-800">
              Book Lab Test
            </h3>
            <button
              onClick={() => setSelectedTest(null)}
              className="text-slate-400 hover:text-slate-600 transition-colors p-2
rounded-full hover:bg-slate-200"
            >
              <X className="w-5 h-5" />
            </button>
          </div>

          {successMsg ? (
            <div className="p-8 text-center space-y-4">
              <div className="w-16 h-16 bg-emerald-100 rounded-full flex items-
center justify-center mx-auto mb-4">
                <MailCheck className="w-8 h-8 text-emerald-600" />
              </div>
              <h4 className="text-xl font-bold text-slate-800">
                Booking Confirmed!
              </h4>
              <p className="text-emerald-600 font-medium">{successMsg}</p>
              <p className="text-sm text-slate-500">
```

```
              Redirecting to your dashboard to view reminders...
            </p>
          </div>
        ) : (
          <form onSubmit={handleBookSubmit} className="p-6 space-y-6">
            <div className="bg-purple-50 p-4 rounded-xl border border-purple-
100 flex justify-between items-center">
              <div>
                <p className="text-sm text-purple-600 font-medium">
                  Selected Test
                </p>
                <p className="font-bold text-slate-800">
                  {selectedTest.name}
                </p>
              </div>
              <span className="text-xl font-bold text-slate-800">
                ₹{selectedTest.price}
              </span>
            </div>

            <div className="grid grid-cols-2 gap-4">
              <div className="space-y-2">
                <label className="text-sm font-semibold text-slate-700 flex items-
center gap-2">
                  <Calendar className="w-4 h-4 text-slate-400" /> Date
                </label>
                <input
                  type="date"
                  required
                  value={bookingData.date}
                  onChange={(e) =>
                    setBookingData({
                      ...bookingData,
                      date: e.target.value,
                    })
```

```
                }
                 className="w-full px-4 py-2.5 rounded-xl border border-slate-200
focus:outline-none focus:ring-2 focus:ring-purple-500 bg-slate-50"
               />
              </div>
              <div className="space-y-2">
               <label className="text-sm font-semibold text-slate-700 flex items-
center gap-2">
                <Clock className="w-4 h-4 text-slate-400" /> Time Slot
               </label>
               <input
                type="time"
                required
                value={bookingData.time}
                onChange={(e) =>
                 setBookingData({
                  ...bookingData,
                  time: e.target.value,
                 })
                }
                 className="w-full px-4 py-2.5 rounded-xl border border-slate-200
focus:outline-none focus:ring-2 focus:ring-purple-500 bg-slate-50"
               />
              </div>
             </div>

             <div className="space-y-3">
              <label className="text-sm font-semibold text-slate-700">
               Collection Type
              </label>
              <div className="grid grid-cols-2 gap-4">
               <label
                className={`border-2 rounded-xl p-4 cursor-pointer transition-all
flex items-center justify-center gap-2 ${bookingData.type === "home" ? "border-
```

```
[#A855F7] bg-purple-50 text-purple-700" : "border-slate-200 hover:border-slate-
300 text-slate-600"}`}
                >
                  <input
                    type="radio"
                    name="type"
                    className="hidden"
                    value="home"
                    checked={bookingData.type === "home"}
                    onChange={() =>
                      setBookingData({ ...bookingData, type: "home" })
                    }
                  />
                  <MapPin className="w-5 h-5" /> Home Collect
                </label>
                <label
                  className={`border-2 rounded-xl p-4 cursor-pointer transition-all
flex items-center justify-center gap-2 ${bookingData.type === "walk-in" ? "border-
[#A855F7] bg-purple-50 text-purple-700" : "border-slate-200 hover:border-slate-
300 text-slate-600"}`}
                >
                  <input
                    type="radio"
                    name="type"
                    className="hidden"
                    value="walk-in"
                    checked={bookingData.type === "walk-in"}
                    onChange={() =>
                      setBookingData({ ...bookingData, type: "walk-in" })
                    }
                  />
                  <TestTube2 className="w-5 h-5" /> Walk-In
                </label>
              </div>
            </div>
```

```
              <div
                className={`space-y-2 transition-all ${bookingData.type === "walk-in" ? "opacity-50" : ""}`}
              >
                <label className="text-sm font-semibold text-slate-700 flex items-center gap-2">
                  <MapPin className="w-4 h-4 text-slate-400" />{" "}
                  {bookingData.type === "home"
                    ? "Home Address"
                    : "Preferred Lab Center"}
                </label>
                <textarea
                  required
                  placeholder={
                    bookingData.type === "home"
                      ? "Enter your full address in Maduravoyal..."
                      : "Search Maduravoyal Labs..."
                  }
                  value={bookingData.address}
                  onChange={(e) =>
                    setBookingData({
                      ...bookingData,
                      address: e.target.value,
                    })
                  }
                  className="w-full px-4 py-3 rounded-xl border border-slate-200 focus:outline-none focus:ring-2 focus:ring-purple-500 bg-slate-50 resize-none h-24"
                />
              </div>

              <button
                type="submit"
                disabled={bookingLoading}
```

```
                className="w-full bg-[#A855F7] text-white py-3.5 rounded-xl font-
bold shadow-lg hover:bg-purple-700 transition-colors flex items-center justify-
center disabled:opacity-70"
              >
                {bookingLoading ? (
                  <div className="w-6 h-6 border-2 border-white/30 border-t-white
rounded-full animate-spin"></div>
                ) : (
                  "Confirm Booking"
                )}
              </button>
            </form>
          )}
        </div>
      </div>
    )}
  </div>
 );
}
```

C2.1.3.11 Login.tsx

```
import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import api from "../services/api";
import { useAuth } from "../context/AuthContext";
import { Activity } from "lucide-react";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();
  const { login } = useAuth();
```

```
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      const { data } = await api.post("/auth/login", { email, password });
      login(data);
      if (data.role === "admin") navigate("/admin");
      else if (data.role === "doctor") navigate("/doctor");
      else navigate("/");
    } catch (err: any) {
      if (!err.response) {
        setError(
          "Server unreachable. Is the backend server running on port 5000?",
        );
      } else {
        setError(err.response?.data?.message || "Login failed");
      }
    }
  };

  return (
    <div className="min-h-screen bg-slate-50 flex flex-col justify-center items-
center p-4">
      <div className="w-full max-w-md bg-white rounded-2xl shadow-sm border
border-slate-200 p-8">
        <div className="flex items-center justify-center gap-2 mb-8">
          <div className="bg-[#06B6D4] p-2 rounded-xl">
            <Activity className="w-6 h-6 text-white" />
          </div>
          <span className="text-2xl font-bold text-slate-800 tracking-tight">
            BAYMAX
          </span>
        </div>

        <h2 className="text-2xl font-bold text-center mb-6 text-slate-800">
          Welcome Back
```

```
        </h2>

        {error && (
          <div className="bg-red-50 text-red-500 p-3 rounded-lg mb-4 text-sm text-
center">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-1">
              Email / Username
            </label>
            <input
              type="text"
              required
              className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] focus:border-transparent outline-none"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
            />
          </div>
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-1">
              Password
            </label>
            <input
              type="password"
              required
              className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] focus:border-transparent outline-none"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
            />
```

```
          </div>
          <button
            type="submit"
            className="w-full bg-[#2563EB] text-white py-2.5 rounded-lg font-medium
hover:bg-blue-700 transition-colors"
          >
            Sign In
          </button>
        </form>
        <p className="mt-6 text-center text-sm text-slate-600">
          Don't have an account?{" "}
          <Link to="/register" className="text-[#2563EB] font-medium">
            Register here
          </Link>
        </p>
      </div>
    </div>
  );
}
```

C2.1.3.12 Register.tsx

```
import { useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import api from "../services/api";
import { useAuth } from "../context/AuthContext";
import { Activity } from "lucide-react";

export default function Register() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
    role: "patient",
    specialization: "",
    experience: "",
```

```
    hospital: "",
  });
  const [error, setError] = useState("");
  const navigate = useNavigate();
  const { login } = useAuth();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      const { data } = await api.post("/auth/register", formData);
      login(data);
      if (data.role === "doctor") {
        alert(
          "Registration successful! Please wait for admin approval before logging in.",
        );
        navigate("/login");
      } else {
        navigate("/");
      }
    } catch (err: any) {
      if (!err.response) {
        setError(
          "Server unreachable. Is the backend server running on port 5000?",
        );
      } else {
        setError(err.response?.data?.message || "Registration failed");
      }
    }
  };

  return (
    <div className="min-h-screen bg-slate-50 flex flex-col justify-center items-
center p-4 py-12">
      <div className="w-full max-w-md bg-white rounded-2xl shadow-sm border
border-slate-200 p-8">
```

```
        <div className="flex items-center justify-center gap-2 mb-8">
          <div className="bg-[#06B6D4] p-2 rounded-xl">
            <Activity className="w-6 h-6 text-white" />
          </div>
          <span className="text-2xl font-bold text-slate-800 tracking-tight">
            BAYMAX
          </span>
        </div>

        <h2 className="text-2xl font-bold text-center mb-6 text-slate-800">
          Create Account
        </h2>

        {error && (
          <div className="bg-red-50 text-red-500 p-3 rounded-lg mb-4 text-sm text-
center">
            {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-1">
              Full Name
            </label>
            <input
              type="text"
              required
              className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none"
              value={formData.name}
              onChange={(e) =>
                setFormData({ ...formData, name: e.target.value })
              }
            />
```

```
        </div>
        <div>
          <label className="block text-sm font-medium text-slate-700 mb-1">
            Email
          </label>
          <input
            type="email"
            required
            className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none"
            value={formData.email}
            onChange={(e) =>
              setFormData({ ...formData, email: e.target.value })
            }
          />
        </div>
        <div>
          <label className="block text-sm font-medium text-slate-700 mb-1">
            Password
          </label>
          <input
            type="password"
            required
            className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none"
            value={formData.password}
            onChange={(e) =>
              setFormData({ ...formData, password: e.target.value })
            }
          />
        </div>
        <div>
          <label className="block text-sm font-medium text-slate-700 mb-1">
            Role
          </label>
```

```
          <select
            className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none bg-white"
            value={formData.role}
            onChange={(e) =>
              setFormData({ ...formData, role: e.target.value })
            }
          >
            <option value="patient">Patient</option>
            <option value="doctor">Doctor</option>
          </select>
        </div>

        {formData.role === "doctor" && (
          <div className="space-y-4 pt-4 border-t border-slate-100">
            <div className="p-3 bg-blue-50 text-blue-800 rounded-lg text-sm">
              Doctor accounts require admin approval. Address is restricted to
              Maduravoyal, Chennai.
            </div>
            <div>
              <label className="block text-sm font-medium text-slate-700 mb-1">
                Specialization
              </label>
              <input
                type="text"
                required
                className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none"
                value={formData.specialization}
                onChange={(e) =>
                  setFormData({ ...formData, specialization: e.target.value })
                }
              />
            </div>
            <div>
```

```
            <label className="block text-sm font-medium text-slate-700 mb-1">
              Years of Experience
            </label>
            <input
              type="number"
              required
              min="0"
              className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none"
              value={formData.experience}
              onChange={(e) =>
                setFormData({ ...formData, experience: e.target.value })
              }
            />
          </div>
          <div>
            <label className="block text-sm font-medium text-slate-700 mb-1">
              Hospital/Clinic Name
            </label>
            <input
              type="text"
              required
              className="w-full px-4 py-2 border border-slate-200 rounded-lg
focus:ring-2 focus:ring-[#06B6D4] outline-none"
              value={formData.hospital}
              onChange={(e) =>
                setFormData({ ...formData, hospital: e.target.value })
              }
            />
          </div>
        </div>
      )}

      <button
        type="submit"
```

```
          className="w-full bg-[#2563EB] text-white py-2.5 rounded-lg font-medium
hover:bg-blue-700 transition-colors mt-6"
         >
          Create Account
         </button>
        </form>
        <p className="mt-6 text-center text-sm text-slate-600">
         Already have an account?{" "}
         <Link to="/login" className="text-[#2563EB] font-medium">
          Sign in
         </Link>
        </p>
       </div>
     </div>
  );
}
```

C2.1.3.13 VideoCall.tsx

```
import React, { useState, useEffect, useRef } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { Mic, MicOff, Video, VideoOff, PhoneOff } from "lucide-react";
import { useAuth } from "../context/AuthContext";
import socket from "../socket";
import api from "../services/api";

export default function VideoCall() {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const { user } = useAuth();

  const [isMuted, setIsMuted] = useState<boolean>(false);
  const [isVideoOff, setIsVideoOff] = useState<boolean>(false);
  const [appointment, setAppointment] = useState<any>(null);

  const localVideoRef = useRef<HTMLVideoElement | null>(null);
```

```
const remoteVideoRef = useRef<HTMLVideoElement | null>(null);
const streamRef = useRef<MediaStream | null>(null);
const peerConnection = useRef<RTCPeerConnection | null>(null);

// SOCKET CONNECT
useEffect(() => {
  if (!user?._id) return;

  socket.on("connect", () => {
    console.log(" Connected:", socket.id);
    socket.emit("register", user._id);
  });

  return () => {
    socket.off("connect");
  };
}, [user]);

// CAMERA + WEBRTC
useEffect(() => {
  const init = async () => {
    try {
      const stream = await navigator.mediaDevices.getUserMedia({
        video: true,
        audio: true,
      });

      streamRef.current = stream;

      if (localVideoRef.current) {
        localVideoRef.current.srcObject = stream;
      }

      const pc = new RTCPeerConnection();
      peerConnection.current = pc;
```

```
      stream.getTracks().forEach((track) => {
        pc.addTrack(track, stream);
      });

      pc.ontrack = (event: RTCTrackEvent) => {
        console.log(" Remote stream received");

        if (remoteVideoRef.current) {
          remoteVideoRef.current.srcObject = event.streams[0];
        }
      };
    } catch (err) {
      console.error("Camera error:", err);
    }
  };

  init();
}, []);

// START CALL (DOCTOR)
const startCall = async () => {
  if (!peerConnection.current || !appointment || !user?._id) return;

  const offer = await peerConnection.current.createOffer();
  await peerConnection.current.setLocalDescription(offer);

  socket.emit("call-user", {
    to: appointment?.patient?._id,
    offer,
    from: user._id,
  });

  console.log(" Calling...");
};
```

```
// RECEIVE CALL + ANSWER
useEffect(() => {
  socket.on("call-made", async (data: any) => {
    if (!peerConnection.current) return;

    await peerConnection.current.setRemoteDescription(
      new RTCSessionDescription(data.offer),
    );

    const answer = await peerConnection.current.createAnswer();
    await peerConnection.current.setLocalDescription(answer);

    socket.emit("make-answer", {
      to: data.from,
      answer,
    });
  });

  socket.on("answer-made", async (data: any) => {
    if (!peerConnection.current) return;

    await peerConnection.current.setRemoteDescription(
      new RTCSessionDescription(data.answer),
    );
  });

  return () => {
    socket.off("call-made");
    socket.off("answer-made");
  };
}, []);

//  FETCH APPOINTMENT
useEffect(() => {
```

```
  const fetchData = async () => {
    try {
      const endpoint =
        user?.role === "doctor"
          ? "/doctor/appointments"
          : "/patients/records";

      const res = await api.get(endpoint);

      const appts =
        user?.role === "doctor" ? res.data : res.data.appointments;

      const current = appts.find((a: any) => a._id === id);
      setAppointment(current);
    } catch (err) {
      console.error(err);
    }
  };

  if (user) fetchData();
}, [id, user]);

//  AUTO CALL
useEffect(() => {
  if (user?.role === "doctor" && appointment) {
    setTimeout(startCall, 2000);
  }
}, [appointment]);

//  CONTROLS
const toggleMute = () => {
  if (!streamRef.current) return;

  streamRef.current.getAudioTracks().forEach((track) => {
    track.enabled = !track.enabled;
```

```
  });

  setIsMuted(!isMuted);
};

const toggleVideo = () => {
  if (!streamRef.current) return;

  streamRef.current.getVideoTracks().forEach((track) => {
    track.enabled = !track.enabled;
  });

  setIsVideoOff(!isVideoOff);
};

const handleEndCall = () => {
  if (streamRef.current) {
    streamRef.current.getTracks().forEach((track) => track.stop());
  }
  navigate(-1);
};

return (
  <div className="h-screen w-full bg-black relative">
    {/*  REMOTE VIDEO */}
    <video
      ref={remoteVideoRef}
      autoPlay
      playsInline
      className="w-full h-full object-cover"
    />

    {/*  LOCAL VIDEO */}
    <video
      ref={localVideoRef}
```

```
        autoPlay
        muted
        className="absolute bottom-5 right-5 w-40 rounded-lg"
      />

      {/* CONTROLS */}
      <div className="absolute bottom-5 w-full flex justify-center gap-4">
        <button onClick={toggleMute}>{isMuted ? <MicOff /> : <Mic />}</button>

        <button onClick={toggleVideo}>
          {isVideoOff ? <VideoOff /> : <Video />}
        </button>

        <button onClick={handleEndCall}>
          <PhoneOff />
        </button>
      </div>
    </div>
  );
}
```

C2.1.4 services

C2.1.4.1 api.ts

```
import axios from 'axios';

const api = axios.create({
  baseURL: (import.meta as any).env.VITE_API_URL || 'http://localhost:5000/api',
});

// Attach token automatically
api.interceptors.request.use((config) => {
  const userStr = localStorage.getItem('user');

  if (userStr) {
    const user = JSON.parse(userStr);
```

```
    if (user.token) {
      config.headers.Authorization = `Bearer ${user.token}`;
    }
  }

  config.headers['Content-Type'] = 'application/json';

  return config;
}, (error) => {
  return Promise.reject(error);
});

export default api;
```

C2.1.5 utils

C2.1.5.1 cn.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

C2.1.6 App.tsx

```
import React from "react";
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import Dashboard from "./pages/Dashboard";
import Login from "./pages/Login";
import Register from "./pages/Register";
import AdminDashboard from "./pages/AdminDashboard";
import DoctorDashboard from "./pages/DoctorDashboard";
import DoctorAppointments from "./pages/DoctorAppointments";
import HealthRecords from "./pages/HealthRecords";
import BuyMedicines from "./pages/BuyMedicines";
import CartView from "./pages/CartView";
```

```
import LabTestsView from "./pages/LabTestsView";
import BloodBank from "./pages/BloodBank";
import Insurance from "./pages/Insurance";
import VideoCall from "./pages/VideoCall";
import { AuthProvider, useAuth } from "./context/AuthContext";

const ProtectedRoute = ({
  children,
  allowedRoles,
}: {
  children: React.ReactElement;
  allowedRoles?: string[];
}) => {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (allowedRoles && !allowedRoles.includes(user.role))
    return <Navigate to="/" />;
  return children;
};

const RoleRedirect = () => {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (user.role === "admin") return <Navigate to="/admin" />;
  if (user.role === "doctor") return <Navigate to="/doctor" />;
  return <Dashboard />;
};

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<RoleRedirect />} />
          <Route path="/login" element={<Login />} />
```

```
<Route path="/register" element={<Register />} />
<Route
  path="/admin"
  element={
    <ProtectedRoute allowedRoles={["admin"]}>
      <AdminDashboard />
    </ProtectedRoute>
  }
/>
<Route
  path="/doctor"
  element={
    <ProtectedRoute allowedRoles={["doctor"]}>
      <DoctorDashboard />
    </ProtectedRoute>
  }
/>
<Route
  path="/patient"
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <Dashboard />
    </ProtectedRoute>
  }
/>
<Route
  path="/appointments"
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <DoctorAppointments />
    </ProtectedRoute>
  }
/>
<Route
  path="/health-records"
```

```
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <HealthRecords />
    </ProtectedRoute>
  }
/>
<Route
  path="/medicines"
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <BuyMedicines />
    </ProtectedRoute>
  }
/>
<Route
  path="/cart"
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <CartView />
    </ProtectedRoute>
  }
/>
<Route
  path="/labs"
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <LabTestsView />
    </ProtectedRoute>
  }
/>
<Route
  path="/blood-bank"
  element={
    <ProtectedRoute allowedRoles={["patient"]}>
      <BloodBank />
```

```
            </ProtectedRoute>
          }
        />
        <Route
          path="/insurance"
          element={
            <ProtectedRoute allowedRoles={["patient"]}>
              <Insurance />
            </ProtectedRoute>
          }
        />
        <Route
          path="/video-call/:id"
          element={
            <ProtectedRoute allowedRoles={["patient", "doctor"]}>
              <VideoCall />
            </ProtectedRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  </AuthProvider>
 );
}

export { App };
export default App;
```

C2.1.7 index.css

```
@import "tailwindcss";
```

C2.1.8 main.tsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
```

```
import { App } from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

C2.1.9 socket.ts

```
import { io } from "socket.io-client";

const socket = io("http://localhost:5000", {
  transports: ["websocket"],
});

export default socket;
```

C2.1.10 vite-env.d.ts

```
/// <reference types="vite/client" />
```

C2.2 index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>BAYMAX - Healthcare Dashboard</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

C3. Explanation of Backend Server Code (Node.js + Express):

The backend server of the BAYMAX healthcare system is developed using Node.js and Express.js, which provide a scalable and efficient environment for handling server-side operations.
The server begins by importing essential modules such as Express for creating the web server, Mongoose for interacting with the MongoDB database, Dotenv for managing environment variables, and CORS to enable cross-origin resource sharing between frontend and backend.

The application initializes an Express instance and configures middleware functions. The express.json() middleware is used to parse incoming JSON data from client requests, while the CORS middleware ensures that the frontend application can communicate with the backend server without restrictions.

The system follows a modular routing structure by separating functionalities into different route files. The user-related operations are handled through /api/users, doctor-related functionalities through /api/doctors, and appointment-related operations through /api/appointments. This modular approach improves code organization, maintainability, and scalability.

The backend is connected to a MongoDB database using Mongoose. The connection is established using a connection string stored securely in environment variables. Once connected, the system is capable of storing and retrieving healthcare-related data such as user details, doctor information, and appointment records.

Finally, the server listens on port 5000 and handles incoming client requests. This backend architecture ensures efficient data processing, secure communication, and reliable performance for the healthcare system.

C4. Explanation of Frontend Application Code (React + TypeScript)

The frontend of the **BAYMAX** system is developed using **React** along with **TypeScript**, which provides a dynamic and interactive user interface. The routing functionality is implemented using **React Router**, enabling navigation between different pages of the application.
A key feature of the frontend is the implementation of role-based access control using a custom component called ProtectedRoute. This component ensures that only authenticated users can access certain parts of the application.

The ProtectedRoute component retrieves the current user information from the authentication context using the useAuth hook. It performs two important checks:

1. Authentication Check:
   If the user is not logged in, they are redirected to the login page using the Navigate component.

2. Authorization Check:
   If the user is logged in but does not have the required role (such as admin, doctor, or patient), they are redirected to the home page. This ensures that users can only access functionalities permitted for their role.

The application defines both public and protected routes. Public routes such as login and registration are accessible to all users, while protected routes such as admin dashboard, doctor dashboard, and patient dashboard are restricted based on user roles. This routing structure enhances the security of the application and ensures proper separation of functionalities among different types of users.

Overall, the frontend architecture provides a responsive, secure, and user-friendly interface, enabling seamless interaction with the backend system and efficient access to healthcare services.

APPENDIX D – USER MANUAL

D.1 User Registration

1. Open the application in a web browser
2. Click on the “Register” option available on the homepage
3. Enter required details such as name, email, and password
4. Verify the entered details and ensure accuracy
5. Click on “Submit” to create an account
6. User account will be successfully created and ready for login

D.2 User Login

1. Navigate to the login page
2. Enter registered email and password
3. Click on “Login”
4. System validates the credentials
5. User will be redirected to the dashboard upon successful authentication

D.3 Doctor Search and Appointment Booking

1. Enter symptoms or select the required medical specialization
2. View the list of available doctors with relevant details
3. Check waiting count, availability, and consultation timings
4. Select a preferred doctor from the list
5. Click on “Book Appointment”
6. Confirm appointment details and finalize the booking

D.4 Medicine Search

1. Navigate to the “Buy Medicines” section
2. Search for the required medicine using the search bar
3. View availability, price, and medicine details
4. Select the desired medicine
5. Add the item to the cart
6. Proceed to checkout to complete the process

D.5 AI Health Assistant

1. Open the AI chatbot interface
2. Enter symptoms or a health-related query
3. The system processes the input using AI
4. Receive suggestions, possible causes, and basic guidance
5. Use the recommendations for preliminary understanding

D.6 Emergency Assistance

1. Navigate to the Emergency section
2. View important emergency contact numbers
3. Access information about nearby hospitals and healthcare services
4. Use quick access options for immediate assistance
5. Follow provided guidance in case of urgent situations

D.7 Health Records Management

1. Navigate to the “Health Records” section from the dashboard
2. View stored medical records, prescriptions, and reports
3. Upload new documents such as prescriptions or test results
4. Update or edit existing health information if required
5. Download or view records for reference
6. Ensure secure access through authentication mechanisms.